



Early Warning System pada EMS Server Menggunakan Algoritma Long Short-Term Memory

SKRIPSI

Sebagai salah satu syarat untuk meraih
Gelar Sarjana

Oleh:

Gamaliel Aradeya
32220076

Universitas Bunda Mulia
Fakultas Teknologi dan Desain
Program Studi Informatika
Jakarta 2025

IDENTITAS PENULIS

Nama : Gamaliel Aradeya
NIM : 32220076
Tempat/tgl lahir : Jakarta, 30 April 2003
Jenis Kelamin : Pria / ~~Wanita~~
Alamat : Sarusunawa Tower D Lantai 10 No 3,Pluit
Penjaringan : Jakarta Utara RT/RW 007/006
No. Telp : +62 851 7247 6678
E-mail : gamaliel.aradeyay@gmail.com

Riwayat Pendidikan

Tahun 2009 - 2015 : SDS Genesaret, Jakarta
Tahun 2015 - 2016 : SMPN 21 Jakarta, Jakarta
Tahun 2016 - 2018 : SMPN 1 Tirtoyudo, Malang
Tahun 2018 - 2021 : SMKN 56 Jakarta, Jakarta
Tahun 2022 - sekarang : Universitas Bunda Mulia, Jakarta

ABSTRAK

Perangkat server yang beroperasi secara terus-menerus menghasilkan panas yang dapat memengaruhi stabilitas dan kinerja perangkat apabila tidak dipantau dengan baik. Pemantauan suhu dan kelembaban pada lingkungan server

diperlukan untuk mengetahui kondisi aktual serta mendukung deteksi dini terhadap potensi anomali termal. Penelitian ini bertujuan untuk membangun Environment Monitoring System (EMS) server berbasis data sensor time-series dengan penerapan algoritma Long Short-Term Memory (LSTM) untuk memprediksi suhu pada area hotspot server testbed.

Sistem yang dikembangkan menggunakan dua sensor suhu dan kelembaban XY-MD02, yaitu S1 sebagai sensor ambient/reference dan S2 sebagai sensor hotspot/exhaust. Data sensor dibaca oleh Raspberry Pi gateway melalui komunikasi Modbus RTU RS485, kemudian dikirim ke backend API berbasis Go dan disimpan pada database PostgreSQL. Dashboard berbasis React digunakan untuk menampilkan data suhu dan kelembaban aktual, histori pembacaan sensor, status kesehatan sensor, status termal aktual, status prediksi, serta hasil evaluasi model. ML Worker berbasis Python dan TensorFlow digunakan untuk melakukan preprocessing, resampling data menjadi interval satu menit, pembentukan window time-series, pelatihan model LSTM, evaluasi model, dan inferensi prediksi suhu S2 lima menit ke depan.

Pengujian dilakukan terhadap integrasi sensor, gateway, backend, database, dashboard, dan ML Worker. Hasil pengujian menunjukkan bahwa sistem berhasil membaca data dari sensor XY-MD02, mengirim data hardware ke backend, menyimpan data ke database sebagai data time-series, serta menampilkan data aktual pada dashboard. Sistem juga berhasil memisahkan status kesehatan sensor dengan status termal aktual, sehingga sensor yang masih aktif dapat tetap berstatus normal secara teknis meskipun suhu aktual masuk kategori waspada atau anomali. Selain itu, inferensi model dapat dijalankan secara periodik sehingga hasil prediksi terbaru dapat dikirim ke backend dan ditampilkan pada dashboard.

Hasil pelatihan awal menggunakan data hardware menunjukkan bahwa model LSTM berhasil menghasilkan prediksi suhu S2 dengan nilai RMSE sebesar 1,4902, MAE sebesar 0,9049, dan MAPE sebesar 2,6127%. Namun, hasil evaluasi juga menunjukkan bahwa baseline persistence dan moving average masih menghasilkan nilai RMSE yang lebih rendah dibandingkan LSTM. Hal ini

menunjukkan bahwa sistem EMS dan pipeline prediksi telah berhasil diimplementasikan, tetapi performa model LSTM masih memerlukan pengumpulan data yang lebih panjang, variasi kondisi termal yang lebih representatif, serta tuning hyperparameter sebelum dapat diklaim sebagai model prediksi final yang lebih unggul dibandingkan baseline sederhana.

Kata kunci: Environment Monitoring System, LSTM, time-series, anomali termal, Raspberry Pi, XY-MD02.

ABSTRACT

Servers that operate continuously generate heat that may affect device stability and performance if the thermal condition is not properly monitored. Temperature and humidity monitoring in a server environment is required to observe the current condition and support early detection of potential thermal anomalies. This research aims to develop a server Environment Monitoring System (EMS) based on time-series sensor data by applying the Long Short-Term Memory (LSTM) algorithm to predict temperature in the hotspot area of a server testbed.

The developed system uses two XY-MD02 temperature and humidity sensors, where S1 acts as the ambient/reference sensor and S2 acts as the hotspot/exhaust sensor. Sensor data is collected by a Raspberry Pi gateway through Modbus RTU RS485 communication, then transmitted to a Go-based backend API and stored in a PostgreSQL database. A React-based dashboard is used to display current temperature and humidity values, sensor reading history, sensor health status, current thermal status, prediction status, and model evaluation results. The Python and TensorFlow-based ML Worker is used for preprocessing, one-minute data resampling, time-series window generation, LSTM model training, model evaluation, and inference to predict S2 temperature five minutes ahead.

Testing was conducted on the integration of sensors, gateway, backend, database, dashboard, and ML Worker. The results show that the system successfully reads data from XY-MD02 sensors, sends hardware data to the backend, stores the data in the database as time-series records, and displays the current readings on the dashboard. The system also successfully separates sensor health status from current thermal status, allowing a sensor to remain technically normal while its actual temperature is categorized as warning or anomaly. In addition, model inference can run periodically so that updated prediction results can be submitted to the backend and displayed on the dashboard.

The initial hardware-based training result shows that the LSTM model can generate S2 temperature predictions with an RMSE of 1.4902, MAE of 0.9049, and MAPE of 2.6127%. However, the evaluation also shows that the persistence and moving average baselines still produce lower RMSE values than the LSTM model. This indicates that the EMS and prediction pipeline have been successfully implemented, but the LSTM model performance still requires longer data collection, more representative thermal variations, and hyperparameter tuning before it can be claimed as a final prediction model that outperforms simple baseline methods.

Keywords: *Environment Monitoring System, LSTM, time-series, thermal anomaly, Raspberry Pi, XY-MD02.*

PRAKATA

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Kuasa karena atas kasih karunia-nya, skripsi yang berjudul “*Prediksi Anomali Termal pada Environment Monitoring System Server menggunakan Algoritma Long-Short Term Memory*”.

Penyusunan skripsi ini merupakan salah satu syarat untuk memperoleh gelar Sarjana pada jenjang pendidikan Strata Satu (S1).

Dalam penyusunan skripsi ini, hal tersebut tidak lepas dari bantuan, arahan, dan bimbingan oleh banyak pihak yang telah memberi banyak masukan kepada penulis. Oleh karena itu, penulis ingin memberikan ucapan terima kasih kepada semua pihak, khususnya kepada:

1. Bapak **Doddy Surja Bajuadji, M.B.A.**, selaku Rektor Universitas Bunda Mulia.
2. Bapak **Howard S. Giam, S.E., Ak., M.B.A.**, selaku Pelaksana Harian Rektor Universitas Bunda Mulia.
3. Ibu **Dr. Shanty Sudarji, S.Psi., M.Psi.**, selaku Wakil Rektor Bidang Akademik Universitas Bunda Mulia.
4. Bapak **Dr. Adhiguna Mahendra, S.Kom., M.Sc.**, selaku Dekan Fakultas Teknologi dan Desain Universitas Bunda Mulia.
5. Bapak **Agustinus Fritz Wijaya, S.Kom., M.Cs.**, selaku Ketua Program Studi Informatika Universitas Bunda Mulia.
6. Bapak **Teady Matius Surya Mulyana, S.Kom., M.Kom.**, selaku Dosen Pembimbing Akademik yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan serta arahan kepada penulis.

Demikian, penyusunan skripsi ini penulis hadirkan dengan segala kelebihan dan kekurangan akibat keterbatasan ilmu dan kemampuan yang dimiliki.

Jakarta, 17 Januari 2026

Penulis,

Gamaliel Aradeya

DAFTAR ISI

	Halaman
IDENTITAS PENULIS	2
PRAKATA	3
DAFTAR ISI	5
BAB 1	
PENDAHULUAN	6
1.1. Latar Belakang Permasalahan	6
1.2. Pembatasan Masalah	8
1.3. Rumusan Permasalahan	9
1.4. Tujuan dan Manfaat	9
1.5. Sistematika Penulisan	11
BAB 2	
TINJAUAN PUSTAKA	12
2.1 Kerangka Teori	12
2.1.1 Environment Monitoring System	12
2.1.2 Lingkungan Termal Server dan Sensor Suhu-Kelembaban	14
2.1.3 Data Time-Series pada EMS	15
2.1.4 Pra-pemrosesan Data Time-Series	16
2.1.5 Long Short-Term Memory	17
2.1.6 Prediksi Suhu Lingkungan Server	18
2.1.7 Mekanisme Deteksi Anomali Termal Berbasis Ambang Batas	19
2.1.8 Evaluasi Model Prediksi	19
2.1.9 Baseline Sederhana	21
2.2 Penelitian Terdahulu	21
2.3 Kerangka / Skema Berpikir	25
BAB 3	
ANALISIS DAN PERANCANGAN	28
3.1 Rekayasa Kebutuhan	28
3.1.1 Kebutuhan Fungsional	29
3.1.1.1 Fungsi Akuisisi Data Sensor	29
3.1.1.2 Fungsi Pengiriman Data Gateway	30
3.1.1.3 Fungsi Validasi Data Sensor	31
3.1.1.4 Fungsi Penyimpanan Data Time-Series	31
3.1.1.5 Fungsi Monitoring Dashboard Real-Time	32
3.1.1.6 Fungsi Visualisasi Layout Sensor	32
3.1.1.7 Fungsi Pra-pemrosesan Data	33
3.1.1.8 Fungsi Prediksi Suhu Menggunakan LSTM	34
3.1.1.9 Fungsi Klasifikasi Status Termal	34
3.1.1.10 Fungsi Notifikasi Telegram	35
3.1.1.11 Fungsi Evaluasi Model dan Baseline	36

3.1.2 Kebutuhan Non-Fungsional	36
3.1.2.1 Kinerja Waktu-Nyata	36
3.1.2.2 Keandalan Pembacaan Sensor	37
3.1.2.3 Integritas Data Time-Series	37
3.1.2.4 Akurasi Prediksi Model	38
3.1.2.5 Keamanan Dasar API	38
3.1.2.6 Kemudahan Penggunaan Dashboard	39
3.1.3 Kebutuhan Perangkat Keras	39
3.1.3.1 Laptop sebagai Server Testbed	39
3.1.3.2 Raspberry Pi Gateway	40
3.1.3.3 Sensor XY-MD02 S1 Ambient	40
3.1.3.4 Sensor XY-MD02 S2 Hotspot atau Exhaust	40
3.1.3.5 USB RS485 Converter	41
3.1.3.6 Perangkat Pendukung Komunikasi dan Daya	41
3.1.4 Kebutuhan Perangkat Lunak	42
3.1.4.1 Sistem Operasi Raspberry Pi	42
3.1.4.2 Go/Golang Backend	42
3.1.4.3 PostgreSQL dan TimescaleDB	43
3.1.4.4 Python untuk Model LSTM	43
3.1.4.5 React, Vite, dan Tailwind CSS	43
3.1.4.6 Chart.js atau ECharts	44
3.1.4.7 Telegram Bot API	44
3.1.5 Kebutuhan Data	44
3.1.6 Asumsi dan Batasan Implementasi	45
3.2 Pemilihan Metode dan Teknologi	46
3.2.1 Pemilihan Metode Prototyping	48
3.2.2 Pemilihan Raspberry Pi sebagai Gateway	49
3.2.3 Pemilihan Sensor XY-MD02 dan Modbus RS485	49
3.2.4 Pemilihan Go sebagai Backend	50
3.2.5 Pemilihan PostgreSQL dan TimescaleDB	50
3.2.6 Pemilihan Python untuk LSTM	51
3.2.7 Pemilihan Server-Sent Events	51
3.2.8 Pemilihan Telegram Bot API	51
3.2.9 Pemilihan Baseline Sederhana	52
3.3 Perancangan Proses	52
3.3.1 Proses Akuisisi Data Sensor	53
3.3.2 Proses Pengiriman Data Gateway ke Backend	55
3.3.3 Proses Penyimpanan Data Time-Series	55
3.3.4 Proses Pra-pemrosesan Data	56
3.3.5 Proses Training dan Evaluasi LSTM	57
3.3.6 Proses Inferensi Prediksi Suhu	58
3.3.7 Proses Klasifikasi Status Termal	58

3.3.8 Proses Monitoring Dashboard dan Layout Sensor	60
3.4 Perancangan Sistem/Aplikasi	60
3.4.1 Sensor Layer	62
3.4.2 Gateway Layer	62
3.4.3 Backend Layer	63
3.4.4 Database Layer	63
3.4.5 Intelligence Layer	66
3.4.6 Presentation and Notification Layer	67
3.4.7 Rancangan API	67
3.4.8 Rancangan Integrasi LSTM	68
3.4.9 Rancangan Notifikasi Telegram	69
3.5 Perancangan Pengujian	70
3.5.1 Pengujian Pembacaan Sensor	70
3.5.2 Pengujian Gateway	71
3.5.3 Pengujian Backend dan Database	71
3.5.4 Pengujian Dashboard Real-Time	72
3.5.5 Pengujian Visualisasi Layout Sensor	72
3.5.6 Pengujian Model LSTM	73
3.5.7 Pengujian Baseline	73
3.5.8 Pengujian Klasifikasi Status Termal	74
3.5.9 Pengujian Telegram Alert	74
3.5.10 Pengujian Demo Sistem	75
BAB 4	
IMPLEMENTASI DAN PENGUJIAN	76
4.1 Implementasi	77
4.1.1 Lingkungan Implementasi	77
4.1.2 Implementasi Arsitektur Sistem	78
4.1.3 Implementasi Basis Data	80
4.1.4 Implementasi Backend Go API	82
4.1.5 Implementasi Gateway Sensor	83
4.1.5.1 Implementasi Gateway Simulator	84
4.1.5.2 Implementasi Gateway Raspberry Pi	85
4.1.6 Implementasi Dashboard Monitoring	86
4.1.7 Implementasi Modul LSTM	88
4.1.8 Implementasi Deteksi Status Anomali Termal	89
4.1.9 Implementasi Notifikasi Telegram	91
4.2 Pengujian	92
4.2.1 Skenario Pengujian	92
4.2.2 Pengujian Gateway Sensor	92
4.2.3 Pengujian Backend API dan Database	94
4.2.4 Pengujian Dashboard Monitoring	95
4.2.5 Pengujian Model LSTM	97
4.2.6 Pengujian Deteksi Status Termal	98

4.2.7 Pengujian Notifikasi Telegram	99
4.2.8 Pengujian Integrasi End-to-End	100
4.2.9 Pembahasan Hasil Pengujian	101
4.2.10 Keterbatasan Pengujian	103
BAB 5	
KESIMPULAN DAN SARAN	104
5.1 Simpulan	104
5.2 Saran	106
Daftar Pustaka Sementara	109

DAFTAR TABEL

Tabel 2.1 Penelitian Terdahulu 1.....	28
Tabel 2.2 Penelitian Terdahulu 2.....	28
Tabel 2.3 Penelitian Terdahulu 3.....	29
Tabel 2.4 Penelitian Terdahulu 4.....	30
Tabel 2.5 Penelitian Terdahulu 5.....	30
Tabel 2.6 Penelitian Terdahulu 6.....	30
Tabel 3.1 Threshold Status Termal.....	41
Tabel 3.2 Kebutuhan Data Model Prediksi.....	51
Tabel 3.3 Rancangan Tabel Database.....	72
Tabel 3.4 Rancangan API.....	75
Tabel 3.5 Pengujian Pembacaan Sensor.....	77
Tabel 3.6 Pengujian Gateway.....	78
Tabel 3.7 Pengujian Backend dan Database.....	78
Tabel 3.8 Pengujian Dashboard Real-Time.....	79
Tabel 3.9 Pengujian Visualisasi Layout Sensor.....	80
Tabel 3.10 Metrik Evaluasi Model LSTM.....	80
Tabel 3.11 Pengujian Baseline.....	81
Tabel 3.12 Pengujian Klasifikasi Status Termal.....	81
Tabel 3.12 Pengujian Telegram Alert.....	82
Tabel 3.13 Pengujian Demo Sistem.....	82
Tabel 4.1 Lingkungan Implementasi Sistem.....	84
Tabel 4.4 Skenario Gateway Simulator.....	92
Tabel 4.5 Implementasi Halaman Dashboard.....	94
Tabel 4.6 Implementasi Model LSTM.....	96
Tabel 4.7 Kriteria Status Termal.....	97
Tabel 4.8 Implementasi Aturan Notifikasi.....	98

Tabel 4.9 Ringkasan Skenario Pengujian.....	99
Tabel 4.10 Pengujian Gateway Simulator.....	100
Tabel 4.11 Pengujian Gateway Raspberry Pi.....	100
Tabel 4.12 Pengujian Backend API.....	101
Tabel 4.13 Pengujian Database.....	102
Tabel 4.14 Pengujian Dashboard Monitoring.....	103
Tabel 4.15 Konfigurasi Pengujian Model LSTM.....	104
Tabel 4.16 Hasil Evaluasi Model.....	104
Tabel 4.17 Pengujian Status Termal.....	105
Tabel 4.18 Pengujian Notifikasi Telegram.....	106
Tabel 4.19 Pengujian Integrasi End-to-End.....	107
Tabel 4.20 Keterkaitan Hasil Pengujian dengan Rumusan Masalah.	109

DAFTAR GAMBAR

Gambar 3.1 Tata Letak Sensor pada Server Testbed	1
Gambar 3.2 Tahapan Metode Prototyping	1
Gambar 3.3 Flowchart Proses Utama EMS	1
Gambar 3.4 Flowchart Akuisisi Data Sensor	1
Gambar 3.5 Flowchart Pra-pemrosesan Data dan Pembentukan Window	1
Gambar 3.6 Flowchart Klasifikasi Status dan Telegram Alert	1
Gambar 3.7 Arsitektur Sistem EMS Berbasis LSTM	1
Gambar 3.8 Entity Relationship Diagram Database EMS	1

BAB 1

PENDAHULUAN

1.1. Latar Belakang Permasalahan

Pesatnya kemajuan di bidang teknologi informasi mengakibatkan intensitas penggunaan perangkat server kian meningkat demi menopang beragam layanan digital, seperti penyimpanan data, aplikasi internal, serta infrastruktur

jaringan. Operasional perangkat tersebut secara terus-menerus memicu timbulnya panas yang berasal dari beban komputasi serta penggunaan komponen perangkat keras dalam jangka panjang. Tanpa pengawasan kondisi termal yang memadai, eskalasi suhu berisiko menurunkan performa, mengganggu kestabilan sistem, hingga memperbesar potensi kerusakan pada komponen perangkat keras (ASHRAE Technical Committee 9.9, 2021).

Parameter lingkungan pada area server menjadi aspek krusial dalam menjamin kontinuitas operasional sistem informasi secara keseluruhan. Faktor seperti suhu dan kelembaban perlu dipantau secara periodik karena keduanya dapat memengaruhi reliabilitas perangkat elektronik. Temperatur yang melampaui batas operasional berpotensi memicu gangguan termal, sedangkan tingkat kelembaban yang tidak ideal dapat meningkatkan risiko gangguan teknis pada perangkat. Oleh karena itu, diperlukan solusi pemantauan lingkungan yang mampu melakukan akuisisi, penyimpanan, serta penyajian data secara kontinu (ASHRAE Technical Committee 9.9, 2021).

Environment Monitoring System (EMS) hadir sebagai solusi sistematis untuk mengawasi parameter lingkungan seperti suhu dan kelembaban secara terstruktur. Dalam konteks Internet of Things, sistem monitoring dapat melibatkan perangkat sensor, jaringan komunikasi, penyimpanan data, dan antarmuka pengguna untuk menghasilkan informasi yang dapat dimanfaatkan dalam pengambilan keputusan (Gubbi et al., 2013). Dalam lingkup server testbed atau ruang server berskala kecil, EMS berfungsi mengumpulkan data sensor secara berkala, menyimpannya dalam format deret waktu, serta menampilkannya melalui

dashboard monitoring. Akan tetapi, mekanisme pemantauan konvensional sering kali masih bersifat reaktif karena hanya menyajikan kondisi aktual setelah perubahan terjadi, sehingga administrator berisiko terlambat dalam mengantisipasi gangguan termal.

Implementasi early warning system (EWS) diperlukan untuk meminimalisir keterlambatan respons terhadap fluktuasi kondisi termal yang berpotensi mengganggu operasional sistem. Dalam konteks penelitian ini, early warning system dipahami sebagai mekanisme yang tidak sekadar menyajikan data aktual, tetapi juga memberikan peringatan dini melalui estimasi kondisi suhu pada periode mendatang. Pemanfaatan deep learning, termasuk Long Short-Term Memory, pada early warning system dapat digunakan untuk mengenali pola data historis dan memperkirakan potensi kondisi kritis sebelum gangguan terjadi (Cassano et al., 2025). Melalui pendekatan tersebut, administrator diharapkan mampu mengidentifikasi potensi kondisi waspada atau anomali secara lebih awal sebelum berkembang menjadi gangguan sistem yang lebih serius.

Informasi yang dihasilkan EMS memiliki karakteristik time-series karena setiap pembacaan sensor dicatat berdasarkan urutan waktu tertentu. Karakteristik tersebut memungkinkan pemanfaatan data historis untuk mengenali pola perubahan temperatur dari waktu ke waktu. Pendekatan supervised learning pada data time-series dapat digunakan untuk membangun model estimasi berdasarkan data historis yang telah tersusun secara kronologis (Bontempi et al., 2013). Penelitian ini menerapkan algoritma Long Short-Term Memory (LSTM), yaitu salah satu arsitektur recurrent neural network yang dirancang untuk mempelajari

dependensi temporal pada data berurutan (Hochreiter & Schmidhuber, 1997). Penerapan LSTM juga relevan pada konteks temperatur lingkungan pusat data karena penelitian sebelumnya telah menggunakan LSTM untuk memodelkan prediksi temperatur data center (Kang et al., 2022).

Pada sistem yang dikembangkan, hasil estimasi temperatur sensor S2 kemudian dibandingkan dengan ambang batas operasional untuk menentukan status termal yang mencakup kondisi normal, waspada, anomali, atau trouble. Status tersebut dipantau melalui dashboard dan dapat dikirimkan melalui notifikasi Telegram jika kondisi lingkungan membutuhkan perhatian. Penggunaan Telegram API sebagai media notifikasi pada sistem monitoring ruang server berbasis IoT telah digunakan dalam penelitian sebelumnya untuk mengirimkan informasi kepada pengguna secara cepat (Wicaksono et al., 2022). Dengan demikian, algoritma LSTM diposisikan sebagai komponen pendukung dalam ekosistem EMS untuk menghasilkan mekanisme peringatan dini yang terukur.

Mengacu pada urgensi tersebut, penelitian ini berfokus pada **Early Warning System EMS Server Menggunakan Algoritma Long Short-Term Memory**. Pengembangan sistem ini mencakup integrasi pembacaan sensor suhu dan kelembaban, transmisi data melalui gateway Raspberry Pi, pengelolaan basis data, visualisasi dashboard, penerapan model LSTM untuk estimasi suhu S2, klasifikasi status termal berdasarkan ambang batas, serta pengiriman notifikasi peringatan dini.

1.2. Pembatasan Masalah

Agar penelitian lebih terarah dan tidak melebar dari fokus utama, maka batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Penelitian dilakukan pada lingkungan server testbed atau ruang server skala kecil-menengah sebagai representasi lingkungan operasional server.
2. Sistem yang dikembangkan merupakan *Environment Monitoring System* untuk melakukan pemantauan lingkungan server dan menghasilkan *early warning*, bukan sistem kontrol pendinginan otomatis.
3. Parameter utama yang digunakan dalam penelitian ini adalah suhu dan kelembaban yang diperoleh dari sensor lingkungan.
4. Data yang digunakan berbentuk *time-series* berdasarkan pencatatan sensor secara periodik.
5. Sensor yang digunakan terdiri dari dua sensor suhu dan kelembaban, yaitu S1 sebagai sensor ambient/reference dan S2 sebagai sensor hotspot/exhaust.
6. Gateway sensor menggunakan Raspberry Pi untuk membaca sensor dan mengirimkan data ke backend.
7. Algoritma *Long Short-Term Memory* digunakan untuk mendukung proses estimasi suhu S2 lima menit ke depan.
8. Target estimasi model LSTM adalah suhu sensor S2 sebagai representasi area hotspot pada lingkungan server testbed.
9. Mekanisme *early warning system* dilakukan berdasarkan hasil estimasi suhu, ambang batas operasional, status termal, dan notifikasi.
10. Status sistem dibatasi pada status normal, waspada, anomali, dan trouble.
11. Notifikasi peringatan dini dikirimkan melalui Telegram apabila status sistem memenuhi kondisi waspada, anomali, atau trouble sesuai aturan sistem.
12. Penelitian ini tidak membahas sistem kontrol pendingin otomatis, kontrol kipas, kontrol AC, relay, auto-remediation, optimasi energi, maupun perhitungan PUE.
13. Evaluasi dilakukan melalui pengujian fungsional sistem, pengujian integrasi, pengujian mekanisme *early warning*, serta evaluasi model menggunakan RMSE, MAE, MAPE, dan baseline sederhana.

1.3. Rumusan Permasalahan

Berdasarkan latar belakang dan pembatasan masalah yang telah diuraikan, rumusan permasalahan dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mengembangkan *Environment Monitoring System* untuk memantau suhu dan kelembaban pada lingkungan server?
2. Bagaimana mengelola data sensor EMS sebagai data *time-series* yang dapat digunakan untuk mendukung analisis kondisi termal?
3. Bagaimana menerapkan algoritma *Long Short-Term Memory* untuk mengestimasi suhu sensor S2 berdasarkan data historis EMS?
4. Bagaimana membangun mekanisme *early warning system* berdasarkan hasil estimasi suhu, ambang batas operasional, status termal, dan notifikasi?
5. Bagaimana mengevaluasi kinerja sistem dalam melakukan pemantauan lingkungan, estimasi suhu, klasifikasi status termal, dan pengiriman peringatan dini?

1.4. Tujuan dan Manfaat

1.4.1. Tujuan Penelitian

Tujuan dari penelitian ini adalah sebagai berikut:

1. Mengembangkan *Environment Monitoring System* untuk memantau suhu dan kelembaban pada lingkungan server secara periodik.
2. Mengelola data hasil pembacaan sensor sebagai data *time-series* yang dapat digunakan dalam proses analisis kondisi termal.
3. Menerapkan algoritma *Long Short-Term Memory* untuk mengestimasi suhu sensor S2 berdasarkan data historis EMS.
4. Membangun mekanisme *early warning system* yang dapat mengklasifikasikan kondisi termal menjadi normal, waspada, anomali, atau trouble.

5. Mengimplementasikan notifikasi Telegram sebagai media peringatan dini ketika sistem mendeteksi kondisi yang membutuhkan perhatian.
6. Mengevaluasi kinerja sistem berdasarkan keberhasilan pembacaan sensor, penyimpanan data, visualisasi dashboard, estimasi suhu, klasifikasi status, pengiriman notifikasi, serta metrik evaluasi model.

1.4.2. Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut:

1. Manfaat Praktis

Penelitian ini dapat membantu administrator atau pengguna sistem dalam memantau kondisi lingkungan server secara lebih terstruktur. Dengan adanya dashboard monitoring dan notifikasi Telegram, sistem dapat memberikan informasi awal terhadap potensi gangguan termal sehingga pengguna dapat melakukan pemeriksaan sebelum kondisi menjadi lebih serius.

2. Manfaat Akademis

Penelitian ini dapat menjadi referensi akademik terkait pengembangan *Environment Monitoring System* berbasis sensor, data *time-series*, dan mekanisme *early warning system*. Selain itu, penelitian ini juga dapat menjadi contoh penerapan algoritma *Long Short-Term Memory* sebagai komponen pendukung pada sistem monitoring lingkungan server.

3. Manfaat Pengembangan Sistem

Penelitian ini dapat menjadi dasar pengembangan EMS yang lebih lanjut, misalnya dengan penambahan jenis sensor, peningkatan mekanisme notifikasi, pengembangan dashboard monitoring, peningkatan kualitas model estimasi suhu, atau penerapan metode pembandingan lain pada data *time-series*.

1.5. Sistematika Penulisan

Sistematika penulisan skripsi ini disusun berdasarkan jalur perekayasaan pada Program Studi Informatika. Adapun sistematika penulisan dalam penelitian ini adalah sebagai berikut:

BAB 1 Pendahuluan

Bab ini membahas latar belakang permasalahan, pembatasan masalah, rumusan permasalahan, tujuan dan manfaat penelitian, serta sistematika penulisan skripsi.

BAB 2 Tinjauan Pustaka

Bab ini membahas teori-teori yang mendukung penelitian, meliputi *Environment Monitoring System*, lingkungan termal server, sensor suhu dan kelembaban, data *time-series*, pra-pemrosesan data, *early warning system*, algoritma *Long Short-Term Memory*, mekanisme klasifikasi status termal, evaluasi model, baseline sederhana, serta penelitian terdahulu.

BAB 3 Analisis dan Perancangan

Bab ini menjelaskan analisis kebutuhan sistem, pemilihan metode dan teknologi, perancangan proses, perancangan arsitektur sistem, perancangan basis data, perancangan dashboard, perancangan ML Worker, serta perancangan pengujian yang digunakan dalam penelitian.

BAB 4 Implementasi dan Pengujian

Bab ini membahas implementasi *Environment Monitoring System*, proses pengumpulan data sensor, implementasi gateway Raspberry Pi, backend API, basis data, dashboard monitoring, ML Worker LSTM, klasifikasi status termal, notifikasi Telegram, serta pengujian sistem dalam mendukung mekanisme *early warning system*.

BAB 5 Penutup

Bab ini berisi simpulan dari hasil penelitian serta saran untuk pengembangan sistem dan penelitian selanjutnya.

BAB 2

TINJAUAN PUSTAKA

2.1 Kerangka Teori

Bab ini membahas teori-teori yang menjadi dasar dalam pengembangan **Early Warning System EMS Server Menggunakan Algoritma Long Short-Term Memory**. Pembahasan teori difokuskan pada *Environment Monitoring System*, lingkungan termal server, data *time-series*, pra-pemrosesan data, algoritma *Long Short-Term Memory*, *early warning system*, estimasi suhu S2, klasifikasi status termal berbasis ambang batas, evaluasi model estimasi suhu, *baseline* sederhana, serta notifikasi Telegram sebagai media peringatan dini.

2.1.1 Environment Monitoring System

Environment Monitoring System atau EMS merupakan sistem yang digunakan untuk melakukan pemantauan kondisi lingkungan secara berkala menggunakan perangkat sensor, media komunikasi, sistem penyimpanan data, dan antarmuka pemantauan. Dalam konteks sistem berbasis *Internet of Things*, EMS dapat dipahami sebagai penerapan konsep IoT karena melibatkan perangkat fisik berupa sensor yang mengumpulkan data dari lingkungan, mengirimkan data tersebut melalui jaringan, kemudian mengolahnya menjadi informasi yang dapat dimanfaatkan oleh pengguna sistem (Gubbi et al., 2013).

Pada lingkungan server, EMS berfungsi untuk memantau parameter lingkungan yang berpengaruh terhadap kestabilan perangkat, seperti suhu dan kelembaban. Parameter tersebut penting karena server beroperasi secara terus-menerus dan menghasilkan panas selama proses komputasi berlangsung. Dalam implementasi sederhana pada server *testbed* atau ruang server skala kecil-menengah, EMS dapat digunakan untuk membaca data sensor secara

periodik, menyimpan hasil pembacaan sebagai data historis, serta menampilkannya melalui *dashboard monitoring*.

EMS dalam penelitian ini tidak diarahkan sebagai sistem kontrol pendinginan otomatis, melainkan sebagai sistem pemantauan lingkungan dan peringatan dini. Data yang dikumpulkan oleh EMS menjadi sumber utama untuk proses estimasi suhu S2 dan klasifikasi status termal. Pemantauan suhu dan kelembaban dapat dilakukan menggunakan perangkat *gateway* seperti Raspberry Pi, sehingga konsep *gateway* berbasis perangkat kecil relevan untuk diterapkan pada sistem pemantauan lingkungan berskala *testbed* (Rajkumar, 2025).

Sistem monitoring berbasis sensor juga dapat digunakan untuk membaca kondisi suatu objek dan menghasilkan status sistem berdasarkan data yang diperoleh. Konsep tersebut relevan dengan penelitian ini karena EMS yang dikembangkan tidak hanya menampilkan data sensor, tetapi juga menentukan status kondisi lingkungan berdasarkan hasil estimasi suhu dan ambang batas operasional yang ditentukan (Joshua & Mulyana, 2024).

2.1.2 Lingkungan Termal Server dan Sensor Suhu-Kelembaban

Lingkungan termal server berkaitan dengan kondisi suhu dan kelembaban yang memengaruhi kestabilan perangkat komputasi. Server yang bekerja dalam jangka waktu panjang dapat menghasilkan panas secara terus-menerus. Apabila kondisi panas tersebut tidak dipantau, peningkatan suhu dapat berpotensi menyebabkan penurunan performa, gangguan stabilitas, atau kerusakan perangkat keras.

Suhu menjadi parameter utama dalam penelitian ini karena digunakan sebagai dasar estimasi kondisi termal pada area yang dipantau. Sementara itu, kelembaban digunakan sebagai parameter pendukung karena kondisi kelembaban yang tidak sesuai dapat memengaruhi kestabilan komponen elektronik. Batas operasional suhu dan kelembaban pada lingkungan pemrosesan data dapat mengacu pada pedoman ASHRAE TC 9.9 sebagai dasar dalam memahami kondisi lingkungan server yang aman (ASHRAE Technical Committee 9.9, 2021).

Pada penelitian ini, sensor yang digunakan terdiri dari dua titik pemantauan, yaitu sensor S1 dan sensor S2. Sensor S1 ditempatkan sebagai sensor *ambient* atau *reference* untuk merepresentasikan kondisi lingkungan sekitar. Sensor S2 ditempatkan sebagai sensor *hotspot* atau *exhaust* yang lebih dekat dengan sumber panas. Data suhu S2 digunakan sebagai target utama estimasi LSTM karena sensor tersebut ditempatkan pada titik yang lebih sensitif terhadap perubahan suhu.

Kondisi termal pada penelitian ini diklasifikasikan menjadi tiga status, yaitu normal, waspada, dan anomali. Status normal menunjukkan kondisi termal yang masih berada pada batas aman, status waspada menunjukkan kondisi yang mulai mendekati batas operasional, sedangkan status anomali menunjukkan kondisi yang telah melewati batas operasional yang ditentukan. Selain tiga status termal tersebut, sistem juga mengenal status *trouble* untuk menandai gangguan teknis, seperti sensor tidak terbaca, *gateway* tidak mengirim data, atau data tidak

valid. Pemisahan ini penting karena *trouble* bukan merupakan kondisi termal, melainkan kondisi teknis sistem.

2.1.3 Data Time-Series pada EMS

Data *time-series* merupakan data yang direkam berdasarkan urutan waktu tertentu. Pada EMS, setiap pembacaan sensor suhu dan kelembaban dicatat bersama *timestamp* sehingga membentuk data berurutan. Karakteristik data seperti ini penting karena nilai sensor pada suatu waktu dapat memiliki hubungan dengan nilai sensor pada waktu sebelumnya.

Dalam penelitian ini, data suhu dan kelembaban dari EMS digunakan sebagai data *time-series* untuk mendukung estimasi suhu S2. Data historis yang dikumpulkan digunakan untuk membentuk pola hubungan antara pembacaan sensor sebelumnya dengan suhu pada periode berikutnya. Strategi *machine learning* untuk *forecasting time-series* umumnya dilakukan dengan mengubah data historis menjadi pasangan input dan output agar dapat dipelajari oleh model estimasi (Bontempi et al., 2013).

Dengan memanfaatkan data *time-series*, sistem monitoring dapat dikembangkan dari pendekatan reaktif menjadi pendekatan *early warning*. Sistem reaktif hanya memberikan informasi ketika nilai sensor aktual telah melewati ambang batas, sedangkan sistem *early warning* dapat menggunakan data historis untuk memperkirakan kondisi suhu pada periode mendatang. Hal ini menjadi dasar penggunaan LSTM dalam penelitian ini.

2.1.4 Pra-pemrosesan Data Time-Series

Pra-pemrosesan data merupakan tahapan penting sebelum data digunakan untuk pelatihan dan pengujian model. Data sensor yang diperoleh dari EMS dapat mengandung nilai kosong, nilai tidak wajar, keterlambatan *timestamp*, atau pencilaan akibat kesalahan pembacaan sensor. Apabila data tersebut tidak diproses terlebih dahulu, model estimasi berpotensi menghasilkan performa yang kurang baik.

Tahapan pra-pemrosesan dalam penelitian ini meliputi validasi *timestamp*, penanganan *missing value*, deteksi nilai pencilaan, normalisasi data, *resampling*, dan pembentukan *window* data. Validasi *timestamp* dilakukan untuk memastikan urutan data sesuai dengan waktu pencatatan. Penanganan *missing value* dilakukan apabila terdapat data sensor yang tidak terbaca pada periode tertentu. Normalisasi dilakukan agar fitur yang digunakan berada pada skala yang seragam.

Pada sistem yang dikembangkan, data sensor mentah dapat dikirim oleh *gateway* dalam interval pembacaan pendek, sedangkan data untuk kebutuhan model disusun dalam interval satu menit. Proses *resampling* dilakukan agar data memiliki interval waktu yang konsisten sebelum digunakan oleh model LSTM. Data sensor kemudian disusun menjadi fitur suhu S1, kelembaban S1, suhu S2, dan kelembaban S2.

Pembentukan *window* data dilakukan dengan mengambil sejumlah data historis sebagai input untuk mengestimasi suhu pada periode berikutnya. Dalam penelitian ini, data historis digunakan untuk mengestimasi suhu S2 lima menit ke depan. Pendekatan ini sesuai dengan konsep *forecasting* berbasis *supervised*

learning, yaitu data historis dibentuk menjadi pasangan input-output untuk kebutuhan pelatihan model (Bontempi et al., 2013).

2.1.5 Long Short-Term Memory

Long Short-Term Memory atau LSTM merupakan salah satu pengembangan dari *Recurrent Neural Network* yang dirancang untuk mempelajari pola pada data berurutan. LSTM dikembangkan untuk mengatasi keterbatasan RNN konvensional dalam mempelajari dependensi jangka panjang akibat masalah *vanishing gradient* (Hochreiter & Schmidhuber, 1997).

LSTM memiliki struktur memori yang memungkinkan model menyimpan, memperbarui, dan mengeluarkan informasi melalui mekanisme gerbang. Secara umum, terdapat tiga gerbang utama dalam LSTM, yaitu *forget gate*, *input gate*, dan *output gate*. *Forget gate* menentukan informasi lama yang perlu dipertahankan atau dihapus. *Input gate* menentukan informasi baru yang akan dimasukkan ke dalam memori. *Output gate* menentukan informasi yang akan digunakan sebagai keluaran pada langkah waktu tertentu.

Dalam penelitian ini, LSTM digunakan untuk mengestimasi suhu S2 berdasarkan data historis EMS. Fitur masukan yang digunakan adalah suhu S1, kelembaban S1, suhu S2, dan kelembaban S2 yang telah melalui tahap pra-pemrosesan. Target estimasi model adalah suhu S2 lima menit ke depan. Penggunaan LSTM dinilai sesuai karena data EMS bersifat *time-series* dan memiliki pola temporal yang dapat dipelajari dari data historis.

Penerapan LSTM untuk prediksi suhu pada lingkungan *data center* telah digunakan dalam penelitian sebelumnya, sehingga pendekatan ini relevan sebagai dasar pemodelan estimasi suhu pada lingkungan server atau server *testbed* (Kang et al., 2022).

2.1.6 Early Warning System

Early Warning System atau EWS merupakan mekanisme yang digunakan untuk memberikan informasi peringatan lebih awal terhadap potensi kondisi yang dapat menimbulkan gangguan. Dalam konteks sistem monitoring, EWS tidak hanya menampilkan kondisi aktual, tetapi juga membantu pengguna mengetahui potensi perubahan kondisi berdasarkan data yang tersedia.

Pada penelitian ini, *early warning system* diterapkan pada EMS server untuk memberikan peringatan dini terhadap potensi gangguan termal. Sistem membaca data suhu dan kelembaban dari sensor, menyimpan data sebagai *time-series*, melakukan estimasi suhu S2 pada periode berikutnya, kemudian menentukan status termal berdasarkan ambang batas operasional. Dengan demikian, sistem tidak hanya bersifat reaktif, tetapi juga mendukung proses pemantauan yang lebih antisipatif.

Pemanfaatan LSTM dalam *early warning system* relevan karena LSTM dapat digunakan untuk mempelajari pola historis pada data berurutan. Cassano et al. (2025) menunjukkan bahwa pendekatan EWS berbasis LSTM dapat digunakan untuk memperkirakan potensi kondisi kritis berdasarkan data *time-series*. Dalam penelitian ini, konsep tersebut diterapkan pada konteks EMS server, yaitu dengan

menggunakan LSTM sebagai komponen pendukung untuk mengestimasi suhu S2 lima menit ke depan.

EWS dalam penelitian ini tidak diarahkan sebagai sistem kontrol otomatis. Sistem tidak menghidupkan kipas, mengontrol AC, mematikan perangkat, atau menjalankan relay. Sistem hanya memberikan informasi status dan notifikasi peringatan dini agar administrator atau pengguna sistem dapat melakukan pemeriksaan secara manual.

2.1.7 Estimasi Suhu S2 sebagai Pendukung Early Warning System

Estimasi suhu merupakan proses memperkirakan nilai suhu pada periode mendatang berdasarkan data historis yang telah dikumpulkan. Dalam penelitian ini, estimasi suhu digunakan sebagai komponen pendukung *early warning system*, bukan sebagai tujuan akhir yang berdiri sendiri. Hasil estimasi suhu S2 digunakan sebagai dasar untuk menentukan apakah kondisi termal masih berada pada status normal, mulai memasuki status waspada, atau telah berada pada status anomali.

Perbedaan utama antara monitoring reaktif dan *early warning system* terletak pada waktu pemberian informasi. Monitoring reaktif hanya menampilkan kondisi aktual atau memberi peringatan setelah nilai sensor melewati ambang batas. Sementara itu, *early warning system* berupaya memberikan informasi lebih awal berdasarkan estimasi kondisi pada periode berikutnya. Dengan pendekatan ini, administrator dapat memperoleh gambaran awal mengenai potensi kenaikan suhu sebelum kondisi tersebut berkembang menjadi gangguan yang lebih serius.

Dalam penelitian ini, LSTM digunakan untuk mengestimasi suhu S2 lima menit ke depan berdasarkan data historis EMS. Data masukan yang digunakan mencakup suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Sensor S1 digunakan sebagai referensi kondisi lingkungan, sedangkan sensor S2 digunakan sebagai target estimasi karena ditempatkan pada area *hotspot* atau *exhaust*.

Penelitian Kang et al. (2022) menunjukkan bahwa LSTM dapat digunakan untuk memodelkan prediksi temperatur pada lingkungan *data center*. Selain itu, Khumaidi et al. (2020) juga menggunakan LSTM pada data lingkungan yang memiliki karakteristik *time-series*. Hal tersebut menunjukkan bahwa LSTM relevan untuk digunakan pada data suhu dan kelembaban yang memiliki hubungan temporal. Namun, pada penelitian ini, penggunaan LSTM tetap dibatasi sebagai komponen estimasi suhu S2 untuk mendukung mekanisme peringatan dini, bukan sebagai sistem kontrol pendinginan otomatis.

2.1.8 Klasifikasi Status Termal Berbasis Ambang Batas

Klasifikasi status termal dilakukan dengan membandingkan hasil estimasi suhu S2 terhadap ambang batas operasional yang ditentukan dalam penelitian. Klasifikasi ini digunakan agar hasil estimasi suhu dapat diterjemahkan menjadi informasi status yang lebih mudah dipahami oleh pengguna sistem.

Pada penelitian ini, status termal dibagi menjadi tiga kondisi, yaitu normal, waspada, dan anomali. Status normal menunjukkan bahwa estimasi suhu S2 masih berada pada rentang aman. Status waspada menunjukkan bahwa estimasi suhu S2 mulai mendekati batas operasional yang membutuhkan

perhatian. Status anomali menunjukkan bahwa estimasi suhu S2 telah melewati batas operasional yang ditentukan.

Selain tiga status termal tersebut, sistem juga mengenal status *trouble*. Status *trouble* tidak merepresentasikan kondisi panas, tetapi menunjukkan adanya gangguan teknis pada sistem, seperti sensor tidak terbaca, data sensor tidak valid, *gateway* tidak mengirimkan data, model belum siap, atau hasil estimasi sudah tidak layak digunakan sebagai status aktif. Dengan demikian, status termal dan status teknis dipisahkan agar interpretasi sistem tidak rancu.

Pendekatan berbasis ambang batas dipilih karena sesuai dengan tujuan penelitian yang ingin menghasilkan mekanisme *early warning* yang sederhana, terukur, dan mudah dijelaskan secara teknis. Deteksi anomali pada data *time-series* dapat dilakukan melalui berbagai pendekatan, termasuk analisis terhadap nilai aktual, nilai estimasi, residual, atau pelanggaran terhadap batas tertentu (Blázquez-García et al., 2021). Selain itu, anomaly detection secara umum berkaitan dengan identifikasi pola atau kondisi yang tidak sesuai dengan perilaku normal (Chandola et al., 2009). Dalam penelitian ini, istilah anomali digunakan secara terbatas pada status termal yang melewati ambang batas operasional berdasarkan hasil estimasi suhu S2.

2.1.9 Evaluasi Model Estimasi Suhu

Evaluasi model dilakukan untuk mengukur tingkat kesalahan antara nilai suhu aktual dan nilai suhu hasil estimasi. Karena keluaran model berupa nilai suhu kontinu, maka evaluasi dilakukan menggunakan metrik kesalahan estimasi, bukan hanya akurasi klasifikasi. Dalam penelitian ini, metrik yang digunakan

adalah *Root Mean Square Error*, *Mean Absolute Error*, dan *Mean Absolute Percentage Error*.

Root Mean Square Error atau RMSE digunakan untuk menghitung akar dari rata-rata kuadrat selisih antara nilai aktual dan nilai estimasi. RMSE memberikan penalti lebih besar terhadap kesalahan yang tinggi, sehingga cocok digunakan untuk melihat sensitivitas model terhadap lonjakan *error*.

Mean Absolute Error atau MAE digunakan untuk menghitung rata-rata selisih absolut antara nilai aktual dan nilai estimasi. MAE lebih mudah dipahami karena menunjukkan rata-rata besar kesalahan dalam satuan yang sama dengan data asli, yaitu derajat Celsius.

Mean Absolute Percentage Error atau MAPE digunakan untuk mengukur kesalahan estimasi dalam bentuk persentase. MAPE dapat membantu membandingkan tingkat kesalahan model secara relatif. Namun, penggunaan MAPE perlu diperhatikan apabila nilai aktual mendekati nol, karena dapat menghasilkan persentase kesalahan yang terlalu besar.

Ketiga metrik tersebut umum digunakan dalam evaluasi model *forecasting* untuk mengukur kualitas hasil estimasi atau prediksi (Hyndman & Koehler, 2006). Dalam penelitian ini, hasil estimasi suhu S2 lima menit ke depan dibandingkan dengan suhu aktual S2 pada waktu target. Evaluasi ini digunakan untuk mengetahui seberapa dekat hasil estimasi model LSTM terhadap kondisi aktual.

Selain evaluasi nilai suhu, penelitian ini juga mengevaluasi status termal yang dihasilkan oleh sistem. Evaluasi status dilakukan dengan membandingkan status hasil sistem dengan kondisi aktual berdasarkan ambang batas yang telah ditentukan. Evaluasi ini penting karena keberhasilan penelitian tidak hanya dilihat dari kecilnya nilai *error* estimasi, tetapi juga dari kemampuan sistem dalam mendukung klasifikasi status normal, waspada, anomali, dan *trouble* sebagai bagian dari *early warning system*.

2.1.10 Baseline Sederhana

Baseline sederhana digunakan sebagai pembanding untuk menilai apakah model LSTM memberikan nilai tambah dibandingkan metode estimasi dasar. Dalam penelitian *time-series*, *baseline* seperti *naive forecast*, *persistence model*, atau *moving average* sering digunakan sebagai pembanding awal (Hyndman & Athanasopoulos, 2021).

Persistence model merupakan metode sederhana yang menggunakan nilai terakhir sebagai estimasi untuk periode berikutnya. Dalam konteks penelitian ini, suhu S2 terakhir dapat digunakan sebagai estimasi suhu S2 lima menit ke depan. Metode ini sederhana, tetapi dapat menjadi pembanding yang kuat apabila pola suhu cenderung stabil.

Moving average merupakan metode yang menggunakan rata-rata beberapa data terakhir sebagai nilai estimasi. Metode ini dapat menghasilkan estimasi yang lebih halus karena tidak hanya bergantung pada satu nilai terakhir. Namun, *moving average* dapat terlambat mengikuti perubahan suhu yang terjadi secara cepat karena nilai estimasinya dipengaruhi oleh rata-rata data sebelumnya.

Dalam penelitian ini, performa LSTM dibandingkan dengan *persistence model* dan *moving average* menggunakan metrik RMSE, MAE, dan MAPE. Jika LSTM menghasilkan nilai *error* yang lebih rendah dibandingkan *baseline*, maka LSTM dapat dikatakan memberikan kontribusi yang lebih baik dalam mengestimasi suhu S2. Sebaliknya, apabila LSTM tidak lebih baik dari *baseline*, hasil tersebut tetap menjadi temuan objektif yang dapat digunakan untuk menganalisis keterbatasan dataset, jumlah data historis, konfigurasi model, atau proses pra-pemrosesan.

2.1.11 Notifikasi Telegram sebagai Media Peringatan Dini

Notifikasi merupakan komponen penting dalam *early warning system* karena peringatan yang hanya ditampilkan pada *dashboard* berisiko tidak segera diketahui oleh pengguna. Administrator atau pengguna sistem tidak selalu berada di depan *dashboard*, sehingga diperlukan media notifikasi yang dapat menyampaikan informasi kondisi penting secara lebih cepat.

Telegram Bot API dapat digunakan sebagai media pengiriman pesan otomatis kepada pengguna atau grup tertentu. Dalam konteks monitoring ruang server, Telegram dapat digunakan untuk mengirimkan informasi ketika sistem mendeteksi kondisi yang membutuhkan perhatian. Wicaksono et al. (2022) menunjukkan bahwa Telegram API dapat diterapkan pada sistem monitoring ruang server berbasis IoT untuk mengirimkan informasi kepada pengguna.

Pada penelitian ini, Telegram digunakan sebagai media peringatan dini ketika sistem mendeteksi status waspada, anomali, atau *trouble*. Informasi yang dikirimkan melalui Telegram dapat mencakup status sistem, nilai estimasi suhu,

waktu kejadian, dan keterangan kondisi yang terdeteksi. Dengan adanya notifikasi ini, pengguna dapat memperoleh informasi awal tanpa harus selalu memantau *dashboard* secara langsung.

Agar notifikasi tidak dikirim secara berulang dalam waktu yang terlalu dekat, sistem perlu menerapkan mekanisme jeda atau *cooldown*. Mekanisme ini digunakan untuk mencegah pengiriman pesan yang berlebihan ketika status yang sama terjadi secara berulang. Selain itu, riwayat notifikasi perlu dicatat agar proses pengujian dan penelusuran kejadian dapat dilakukan dengan lebih mudah

2.2 Penelitian Terdahulu

Penelitian terdahulu digunakan untuk mengetahui posisi penelitian ini dibandingkan dengan penelitian yang telah dilakukan sebelumnya. Pembahasan penelitian terdahulu juga digunakan untuk mengidentifikasi persamaan, perbedaan, serta celah penelitian yang menjadi dasar pengembangan sistem dalam skripsi ini.

Tabel 2.1 Penelitian Terdahulu 1

Keterangan	Isi
Judul	Humidity and temperature monitoring using Raspberry Pi via RS232 networking
Tahun Terbit	2025
Nama Penulis	P. Rajkumar
Jurnal	Array
Kesimpulan	Penelitian ini membahas sistem monitoring suhu dan kelembaban menggunakan Raspberry Pi sebagai perangkat penghubung. Sistem tersebut menunjukkan bahwa perangkat kecil seperti Raspberry Pi dapat digunakan untuk membaca dan mengirim data lingkungan secara periodik.

Persamaan	Sama-sama menggunakan parameter suhu dan kelembaban sebagai data utama dalam sistem pemantauan lingkungan.
Perbedaan	Penelitian Rajkumar berfokus pada monitoring suhu dan kelembaban, sedangkan penelitian ini menambahkan mekanisme <i>early warning system</i> melalui estimasi suhu S2 menggunakan LSTM, klasifikasi status termal, dashboard, dan notifikasi Telegram.

Tabel 2.2 Penelitian Terdahulu 2

Keterangan	Isi
Judul	A two-segment LSTM based data center temperature prediction model
Tahun Terbit	2022
Nama Penulis	Y. Kang, C. Ma, S. Wang, W. Wu, dan K. Zhao
Jurnal	IEICE Electronics Express
Kesimpulan	Penelitian ini membahas penggunaan model LSTM untuk memprediksi suhu pada lingkungan data center. Model yang digunakan diarahkan untuk mempelajari pola suhu berdasarkan data historis sehingga dapat menghasilkan prediksi suhu.
Persamaan	Sama-sama menggunakan LSTM untuk memodelkan suhu pada lingkungan yang berkaitan dengan server atau <i>data center</i> .
Perbedaan	Penelitian Kang et al. berfokus pada pengembangan model prediksi suhu <i>data center</i> , sedangkan penelitian ini berfokus pada perekayasaan EMS server <i>testbed</i> yang mencakup pembacaan sensor suhu dan kelembaban, penyimpanan data <i>time-series</i> , dashboard, estimasi suhu S2, klasifikasi status termal, dan notifikasi Telegram.

Tabel 2.3 Penelitian Terdahulu 3

Keterangan	Isi
Judul	Data center temperature prediction and management based on a two-stage self-healing model
Tahun Terbit	2024
Nama Penulis	S. Wang, Y. Kang, Y. Xu, C. Ma, H. Wang, dan W. Wu

Jurnal	Simulation Modelling Practice and Theory
Kesimpulan	Penelitian ini membahas prediksi dan pengelolaan suhu pada data center untuk mendukung respons terhadap kondisi termal. Pendekatan prediksi digunakan agar sistem dapat mengantisipasi perubahan suhu secara lebih baik.
Persamaan	Sama-sama menekankan pentingnya estimasi atau prediksi suhu untuk mengurangi risiko keterlambatan respons terhadap kenaikan suhu.
Perbedaan	Penelitian Wang et al. berada pada konteks manajemen suhu <i>data center</i> dan <i>self-healing model</i> , sedangkan penelitian ini dibatasi pada EMS server <i>testbed</i> sebagai sistem monitoring dan peringatan dini tanpa kontrol pendinginan otomatis.

Tabel 2.4 Penelitian Terdahulu 4

Keterangan	Isi
Judul	Pengujian algoritma Long Short Term Memory untuk prediksi kualitas udara dan suhu Kota Bandung
Tahun Terbit	2020
Nama Penulis	A. Khumaidi, R. Raafi'udin, dan I. P. Solihin
Jurnal	Jurnal Telematika
Kesimpulan	Penelitian ini menerapkan algoritma LSTM untuk memprediksi data kualitas udara dan suhu berbasis time-series. Penelitian tersebut menunjukkan bahwa LSTM dapat digunakan untuk mempelajari pola historis pada data suhu.
Persamaan	Sama-sama menggunakan LSTM untuk memprediksi suhu berdasarkan data historis.
Perbedaan	Objek penelitian Khumaidi et al. adalah suhu dan kualitas udara Kota Bandung, sedangkan penelitian ini berfokus pada suhu lingkungan server <i>testbed</i> dan mekanisme <i>early warning system</i> pada EMS.

Tabel 2.5 Penelitian Terdahulu 5

Keterangan	Isi
Judul	A review on outlier/anomaly detection in time series data
Tahun Terbit	2021
Nama Penulis	A. Blazquez-Garcia, A. Conde, U. Mori, dan J. A. Lozano

Jurnal	ACM Computing Surveys
Kesimpulan	Penelitian ini membahas berbagai pendekatan deteksi outlier dan anomali pada data time-series. Anomali pada data berurutan dapat dianalisis melalui nilai aktual, hasil prediksi, residual, atau pelanggaran terhadap ambang batas tertentu.
Persamaan	Sama-sama membahas anomali pada data time-series.
Perbedaan	Penelitian Blázquez-García et al. merupakan kajian literatur mengenai anomali <i>time-series</i> , sedangkan penelitian ini menerapkan klasifikasi status termal secara langsung pada EMS server berdasarkan hasil estimasi suhu S2 dan ambang batas operasional.

Tabel 2.6 Penelitian Terdahulu 6

Keterangan	Isi
Judul	Engine health monitoring pada sepeda motor berbasis Arduino menggunakan algoritma Fuzzy Inference System Tsukamoto
Tahun Terbit	2024p
Nama Penulis	L. Joshua dan T. M. S. Mulyana
Jurnal	Jurnal Algoritma, Logika dan Komputasi
Kesimpulan	Penelitian ini membahas sistem monitoring berbasis sensor untuk membaca kondisi objek dan menghasilkan status sistem menggunakan pendekatan inferensi fuzzy.
Persamaan	Sama-sama menggunakan data sensor untuk melakukan pemantauan kondisi dan menghasilkan status sistem.
Perbedaan	Penelitian Joshua dan Mulyana berfokus pada <i>engine health monitoring</i> sepeda motor menggunakan metode <i>Fuzzy Inference System Tsukamoto</i> , sedangkan penelitian ini berfokus pada EMS server dengan estimasi suhu S2 menggunakan LSTM dan mekanisme peringatan dini melalui dashboard serta Telegram.

Tabel 2.7 Penelitian Terdahulu 7

Keterangan	Isi
Judul	An EWS-LSTM-based deep learning early warning system for industrial machine fault prediction

Tahun Terbit	2025
Nama Penulis	F. Cassano, A. M. Crespino, M. Lazoi, G. Specchia, dan A. Spennato
Jurnal	Applied Sciences
Kesimpulan	Penelitian ini membahas early warning system berbasis LSTM untuk memperkirakan potensi kondisi kritis atau fault berdasarkan data historis.
Persamaan	Sama-sama menggunakan LSTM sebagai komponen pendukung early warning system pada data time-series.
Perbedaan	Penelitian Cassano et al. diterapkan pada konteks industrial machine fault prediction, sedangkan penelitian ini diterapkan pada EMS server untuk estimasi suhu S2, klasifikasi status termal, dan notifikasi Telegram.

Tabel 2.8 Penelitian Terdahulu 8

Keterangan	Isi
Judul	IoT implementation for server room security monitoring using Telegram API
Tahun Terbit	2022
Nama Penulis	M. F. Wicaksono, M. D. Rahmatya, dan Ilham
Jurnal	International Journal on Advanced Science, Engineering and Information Technology
Kesimpulan	Penelitian ini membahas implementasi IoT untuk monitoring ruang server dengan memanfaatkan Raspberry Pi dan Telegram API sebagai media pengiriman informasi kepada pengguna.
Persamaan	Sama-sama berada pada konteks monitoring ruang server, menggunakan pendekatan IoT, dan memanfaatkan Telegram sebagai media notifikasi.
Perbedaan	Penelitian Wicaksono et al. berfokus pada monitoring keamanan ruang server, sedangkan penelitian ini berfokus pada EMS termal yang menggunakan data suhu dan kelembaban, estimasi suhu S2 menggunakan LSTM, klasifikasi status termal, dan pengiriman notifikasi peringatan dini.

Berdasarkan penelitian terdahulu tersebut, dapat disimpulkan bahwa penelitian mengenai monitoring suhu, data *time-series*, LSTM, anomali, dan notifikasi telah dilakukan pada berbagai konteks. Penelitian Rajkumar menunjukkan penggunaan Raspberry Pi dalam monitoring suhu dan kelembaban. Penelitian Kang et al., Wang et al., dan Khumaidi et al. menunjukkan bahwa LSTM dapat digunakan untuk memodelkan data suhu atau data lingkungan berbasis *time-series*. Penelitian Blázquez-García et al. dan Chandola et al. memberikan dasar konseptual mengenai anomali pada data. Penelitian Cassano et al. menunjukkan penggunaan LSTM dalam *early warning system*, sedangkan Wicaksono et al. menunjukkan penggunaan Telegram API pada monitoring ruang server.

Celah penelitian yang diisi oleh penelitian ini adalah pengembangan EMS server *testbed* yang menggabungkan pembacaan sensor suhu dan kelembaban, pengiriman data melalui Raspberry Pi *gateway*, penyimpanan data *time-series*, estimasi suhu S2 lima menit ke depan menggunakan LSTM, klasifikasi status termal berbasis ambang batas, visualisasi dashboard, dan notifikasi Telegram sebagai satu kesatuan *early warning system*. Dengan demikian, penelitian ini tidak hanya berfokus pada model estimasi suhu, tetapi juga pada integrasi sistem perekayasa agar data sensor dapat dipantau, dianalisis, diklasifikasikan, dan dikirimkan sebagai peringatan dini.

2.3 Kerangka / Skema Berpikir

Kerangka berpikir dalam penelitian ini disusun untuk menggambarkan hubungan antara permasalahan, teori, metode, dan solusi yang dikembangkan. Permasalahan utama yang diangkat adalah pemantauan lingkungan server yang masih berpotensi bersifat reaktif. Pada kondisi tersebut, administrator baru mengetahui gangguan termal ketika suhu aktual telah berubah atau melewati batas tertentu. Kondisi ini menimbulkan kebutuhan terhadap sistem yang mampu melakukan pemantauan lingkungan secara periodik dan memberikan peringatan dini.

Penelitian ini menggunakan *Environment Monitoring System* untuk membaca suhu dan kelembaban dari sensor S1 dan S2. Sensor S1 digunakan sebagai sensor *ambient* atau *reference*, sedangkan sensor S2 digunakan sebagai sensor *hotspot* atau *exhaust*. Data dari kedua sensor dikirimkan oleh Raspberry Pi *gateway* ke backend, kemudian disimpan ke dalam basis data sebagai data *time-series*.

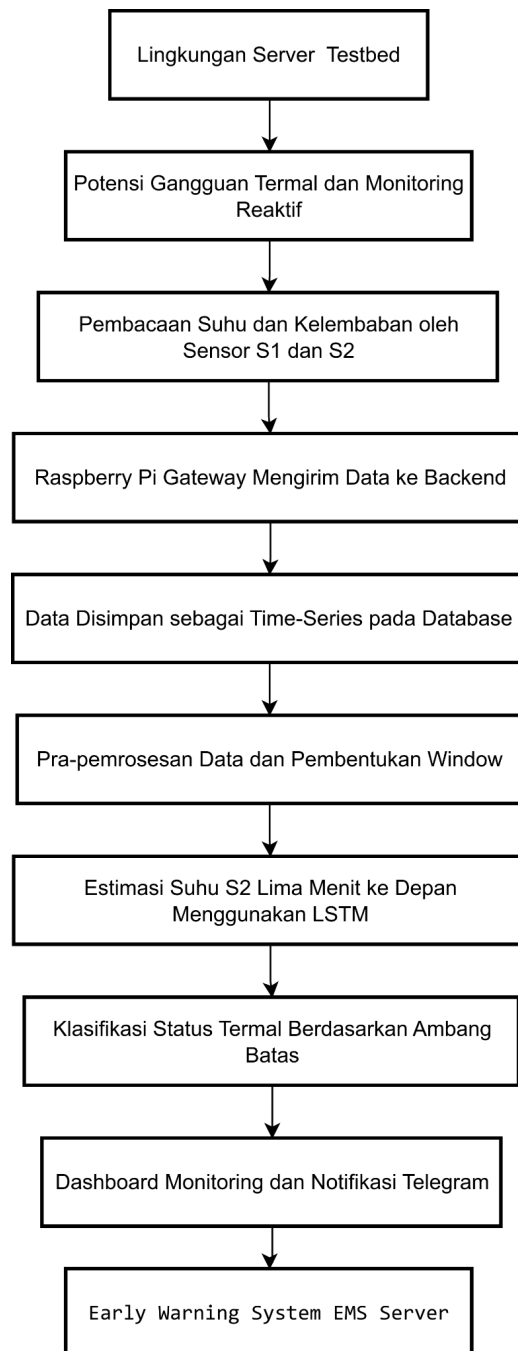
Data historis yang tersimpan digunakan oleh ML Worker untuk melakukan pra-pemrosesan, *resampling*, pembentukan *window*, pelatihan model, evaluasi, dan inferensi menggunakan algoritma *Long Short-Term Memory*. Hasil inferensi berupa estimasi suhu S2 lima menit ke depan. Estimasi tersebut kemudian dibandingkan dengan ambang batas operasional untuk menghasilkan status termal normal, waspada, atau anomali.

Selain status termal, sistem juga memperhatikan status teknis berupa *trouble*. Status *trouble* digunakan ketika terjadi gangguan pada sensor, *gateway*, data, atau proses sistem pendukung. Status yang dihasilkan kemudian ditampilkan

melalui dashboard monitoring dan dapat dikirimkan melalui Telegram sebagai media peringatan dini.

Dengan demikian, kerangka berpikir penelitian ini menghubungkan permasalahan monitoring reaktif dengan solusi EMS berbasis sensor, pengolahan data *time-series*, estimasi suhu menggunakan LSTM, klasifikasi status termal, dan pengiriman notifikasi sebagai bentuk *early warning system*.

Gambar 2.1 Diagram Kerangka Berpikir



BAB 3

ANALISIS DAN PERANCANGAN

Bab ini membahas analisis kebutuhan dan perancangan sistem yang dikembangkan dalam penelitian. Sistem yang dirancang merupakan *Early Warning System* pada *Environment Monitoring System* (EMS) server yang berfungsi untuk memantau suhu dan kelembaban, menyimpan data sensor sebagai data *time-series*, mengestimasi suhu S2 menggunakan algoritma *Long Short-Term Memory* (LSTM), mengklasifikasikan status termal, serta mengirimkan notifikasi peringatan dini melalui Telegram.

Perancangan sistem dilakukan dengan pendekatan perekayasa karena penelitian tidak hanya berfokus pada pemodelan data, tetapi juga mencakup perancangan perangkat pemantauan, gateway sensor, backend API, basis data, dashboard monitoring, ML Worker, visualisasi layout sensor, klasifikasi status, dan sistem notifikasi. Dengan demikian, Bab ini disusun untuk menjelaskan kebutuhan sistem, pemilihan metode dan teknologi, perancangan proses, perancangan sistem atau aplikasi, serta perancangan pengujian.

3.1 Rekayasa Kebutuhan

Rekayasa kebutuhan merupakan tahap awal dalam perancangan sistem yang bertujuan untuk mengidentifikasi fitur, komponen, dan karakteristik sistem

yang akan dikembangkan. Pada penelitian ini, rekayasa kebutuhan dilakukan agar sistem EMS yang dibangun mampu menjalankan fungsi utama, yaitu membaca data suhu dan kelembaban dari lingkungan server *testbed*, mengirimkan data ke backend, menyimpan data sebagai *time-series*, mengolah data sensor, mengestimasi suhu S2 menggunakan LSTM, menentukan status termal, menampilkan informasi melalui dashboard, serta mengirimkan notifikasi peringatan dini.

Sistem yang dikembangkan terdiri dari beberapa komponen utama, yaitu sensor suhu dan kelembaban XY-MD02, Raspberry Pi sebagai gateway, backend API, basis data PostgreSQL, dashboard monitoring, ML Worker LSTM, dan Telegram Bot API. Setiap komponen memiliki peran tertentu dalam alur kerja sistem agar proses pemantauan, estimasi suhu, klasifikasi status, dan pengiriman notifikasi dapat berjalan secara terintegrasi.

3.1.1 Kebutuhan Fungsional

Kebutuhan fungsional merupakan kebutuhan yang menjelaskan fungsi utama yang harus dapat dilakukan oleh sistem. Kebutuhan fungsional pada penelitian ini disusun berdasarkan alur kerja sistem, mulai dari pembacaan sensor, pengiriman data, validasi data, penyimpanan data, visualisasi dashboard, pra-pemrosesan data, estimasi suhu S2, klasifikasi status, notifikasi, hingga evaluasi model.

3.1.1.1 Fungsi Akuisisi Data Sensor

Sistem harus mampu membaca data suhu dan kelembaban dari dua sensor yang ditempatkan pada dua titik berbeda di sekitar server *testbed*. Penggunaan dua sensor bertujuan agar sistem dapat membandingkan kondisi suhu antara area referensi lingkungan dan area yang lebih dekat dengan sumber panas perangkat.

Sensor pertama, yaitu S1, ditempatkan pada area *ambient* atau referensi lingkungan. Sensor ini berfungsi untuk membaca suhu dan kelembaban lingkungan sekitar yang tidak terkena sumber panas secara langsung. Data dari S1 digunakan sebagai fitur referensi terhadap kondisi lingkungan sekitar server *testbed*.

Sensor kedua, yaitu S2, ditempatkan pada area *hotspot* atau *exhaust* yang lebih dekat dengan sumber panas. Sensor ini menjadi sensor utama karena posisinya lebih sensitif terhadap perubahan suhu. Oleh karena itu, suhu S2 digunakan sebagai target estimasi model LSTM. Dengan rancangan ini, S1 berperan sebagai sensor referensi, sedangkan S2 berperan sebagai sensor utama untuk estimasi suhu dan penentuan status termal.

3.1.1.2 Fungsi Pengiriman Data Gateway

Sistem harus mampu mengirimkan data hasil pembacaan sensor dari Raspberry Pi gateway menuju backend. Raspberry Pi berperan sebagai perangkat penghubung antara sensor dan sistem backend.

Sensor membaca suhu dan kelembaban menggunakan komunikasi Modbus RTU melalui RS485. Agar data dari sensor dapat dibaca oleh Raspberry Pi, digunakan USB RS485 Converter sebagai penghubung antara jalur RS485 sensor dan port USB pada Raspberry Pi. Setelah data sensor berhasil dibaca, Raspberry Pi membentuk data dalam format JSON yang berisi identitas sensor, waktu pembacaan, suhu, kelembaban, sumber data, dan status kualitas data.

Data tersebut kemudian dikirimkan ke backend menggunakan HTTP REST API. Backend menerima data dari gateway, melakukan validasi, lalu menyimpan data ke dalam basis data. Dengan adanya gateway, proses pembacaan sensor dan pengelolaan data pada backend dapat dipisahkan sehingga sistem lebih mudah diuji dan dikembangkan.

3.1.1.3 Fungsi Validasi Data Sensor

Backend harus mampu melakukan validasi terhadap data yang diterima dari gateway sebelum data disimpan ke basis data. Validasi diperlukan untuk mencegah data kosong, data tidak lengkap, data tidak sesuai format, atau data tidak logis masuk ke dalam dataset utama.

Validasi data meliputi pengecekan identitas gateway, identitas sensor, waktu pembacaan, nilai suhu, nilai kelembaban, sumber data, dan status kualitas data. Apabila data yang diterima tidak valid, sistem mencatat kondisi tersebut sebagai log kesalahan. Apabila data valid, backend melanjutkan proses penyimpanan ke basis data.

Hasil validasi juga digunakan untuk mendukung penentuan status teknis sistem. Data yang tidak valid, sensor yang tidak terbaca, atau keterlambatan pengiriman data dapat menjadi dasar bagi sistem untuk menampilkan status *trouble*.

3.1.1.4 Fungsi Penyimpanan Data Time-Series

Sistem harus mampu menyimpan data hasil pembacaan sensor berdasarkan urutan waktu. Data EMS memiliki karakteristik *time-series* karena setiap pembacaan suhu dan kelembaban dicatat dengan *timestamp* tertentu. Data historis tersebut diperlukan untuk kebutuhan monitoring, visualisasi grafik, pra-pemrosesan, pelatihan model, pengujian model, evaluasi, dan inferensi.

Basis data yang digunakan dalam penelitian ini adalah PostgreSQL. PostgreSQL digunakan sebagai basis data utama untuk menyimpan data sensor, data gateway, hasil estimasi suhu, status termal, versi model, metrik evaluasi, hasil baseline, layout sensor, event, log sistem, dan riwayat notifikasi. TimescaleDB dapat digunakan sebagai dukungan tambahan apabila tersedia, tetapi sistem tetap dirancang agar dapat berjalan menggunakan PostgreSQL biasa.

Tabel utama yang digunakan dalam sistem mencakup data sensor readings, predictions, model metrics, baseline results, anomaly events, notification logs, settings, dan system logs. Dengan rancangan penyimpanan tersebut, proses monitoring dan early warning dapat ditelusuri berdasarkan data yang tersimpan.

3.1.1.5 Fungsi Monitoring Dashboard Real-Time

Sistem harus mampu menampilkan data suhu dan kelembaban pada dashboard monitoring. Dashboard digunakan oleh administrator atau pengguna sistem untuk melihat kondisi lingkungan server *testbed* secara lebih mudah.

Informasi yang ditampilkan pada dashboard meliputi suhu dan kelembaban dari S1 dan S2, grafik perubahan suhu, grafik perubahan kelembaban, hasil estimasi suhu S2, status termal, status gateway, status sensor, metrik evaluasi model, serta riwayat event. Dashboard juga menampilkan indikator status agar pengguna dapat mengetahui apakah kondisi sistem berada pada status normal, waspada, anomali, atau *trouble*.

Pengiriman data dari backend ke dashboard dapat dilakukan menggunakan API dan Server-Sent Events (SSE). SSE digunakan untuk mengirimkan pembaruan data satu arah dari backend ke browser secara *real-time* atau mendekati *real-time*. Dengan mekanisme ini, dashboard dapat menampilkan perubahan data tanpa harus selalu melakukan refresh manual.

3.1.1.6 Fungsi Visualisasi Layout Sensor

Sistem harus mampu menampilkan posisi sensor pada gambar layout server *testbed* atau ruangan. Gambar layout digunakan sebagai representasi visual dari posisi perangkat, seperti server *testbed*, Raspberry Pi gateway, dan sensor suhu-kelembaban.

Pada dashboard, setiap sensor direpresentasikan menggunakan icon atau marker yang menunjukkan status sensor. Status normal, waspada, dan anomali

digunakan untuk menunjukkan kondisi termal berdasarkan hasil estimasi suhu dan ambang batas. Sementara itu, status *trouble* digunakan untuk menunjukkan masalah teknis seperti sensor tidak terbaca, data tidak valid, atau gateway tidak aktif.

Fitur visualisasi layout sensor memungkinkan pengguna mengetahui posisi S1 dan S2 secara lebih mudah. Apabila sensor berada pada kondisi waspada, anomali, atau *trouble*, dashboard dapat menyorot posisi sensor tersebut agar pengguna mengetahui lokasi yang perlu diperhatikan. Fitur ini berfungsi sebagai pendukung visualisasi monitoring dan tidak mengubah proses utama estimasi LSTM.

3.1.1.7 Fungsi Pra-pemrosesan Data

Sistem harus memiliki proses pra-pemrosesan data sebelum data digunakan oleh model LSTM. Pra-pemrosesan dilakukan untuk memastikan data sensor yang digunakan dalam proses pelatihan, pengujian, dan inferensi berada dalam kondisi layak.

Tahapan pra-pemrosesan meliputi validasi *timestamp*, pengecekan *missing value*, pengecekan nilai tidak wajar, penggabungan data S1 dan S2 berdasarkan waktu, resampling, normalisasi data, dan pembentukan *window* data. Data sensor yang awalnya tersimpan berdasarkan pembacaan periodik dari gateway diolah menjadi data dengan interval yang lebih seragam untuk kebutuhan model.

Pada penelitian ini, data untuk kebutuhan ML disusun dalam interval satu menit melalui proses resampling. Data yang digunakan sebagai fitur terdiri dari suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Pembentukan *window* dilakukan dengan mengambil sejumlah data historis sebagai input model untuk mengestimasi suhu S2 lima menit ke depan.

3.1.1.8 Fungsi Estimasi Suhu S2 Menggunakan LSTM

Sistem harus mampu melakukan estimasi suhu S2 menggunakan algoritma *Long Short-Term Memory*. LSTM digunakan karena data suhu dan kelembaban yang dikumpulkan oleh EMS bersifat berurutan dan memiliki hubungan temporal antarwaktu.

Pada penelitian ini, model LSTM difokuskan untuk mengestimasi suhu S2 karena S2 ditempatkan pada area *hotspot* atau titik utama pemantauan termal pada server *testbed*. Data suhu dan kelembaban dari S1 dan S2 digunakan sebagai fitur input, sedangkan target estimasi adalah suhu S2 lima menit ke depan.

Proses estimasi dilakukan oleh ML Worker. ML Worker bertugas mengambil data historis dari basis data, melakukan pra-pemrosesan, membentuk dataset, melatih model, mengevaluasi model, dan melakukan inferensi. Hasil estimasi suhu S2 kemudian dikirimkan ke backend untuk disimpan, diklasifikasikan, dan ditampilkan pada dashboard.

3.1.1.9 Fungsi Klasifikasi Status Termal

Sistem harus mampu mengklasifikasikan kondisi termal berdasarkan hasil estimasi suhu S2. Status yang digunakan dalam penelitian ini adalah **normal**, **waspada**, dan **anomali**.

Klasifikasi status dilakukan dengan membandingkan suhu hasil estimasi terhadap threshold operasional penelitian. Threshold ini digunakan untuk menentukan apakah suhu yang diprediksi masih berada dalam kondisi aman, mendekati batas peringatan, atau telah melewati batas anomali.

Tabel 3.1 Threshold Status Termal

Status	Kriteria Suhu Estimasi S2
Normal	Suhu Estimasi S2 < 30°C
Waspada	Suhu Estimasi S2 berada pada rentang 30°C sampai 32°C
Anomali	Suhu Estimasi S2 > 32°C

Nilai threshold pada Tabel 3.1 digunakan sebagai ambang operasional pada lingkungan server *testbed*. Nilai tersebut dapat disesuaikan apabila hasil observasi awal menunjukkan bahwa karakter suhu perangkat dan suhu ruangan memiliki rentang yang berbeda. Dengan demikian, threshold tidak diposisikan sebagai standar universal untuk seluruh lingkungan server, tetapi sebagai batas operasional yang digunakan dalam penelitian.

Selain status termal, sistem juga memiliki status teknis berupa *trouble*. Status *trouble* digunakan ketika terdapat gangguan pada sensor, gateway, data, model, atau komponen sistem pendukung. Contoh kondisi *trouble* adalah sensor

tidak terbaca, gateway tidak mengirimkan data, data tidak valid, model belum siap, atau hasil estimasi sudah tidak layak digunakan sebagai status aktif. Dengan pemisahan ini, sistem dapat membedakan antara kondisi termal dan gangguan teknis.

3.1.1.10 Fungsi Notifikasi Telegram

Sistem harus mampu mengirimkan notifikasi melalui Telegram apabila status sistem membutuhkan perhatian. Notifikasi digunakan sebagai bentuk peringatan dini agar administrator atau pengguna sistem dapat mengetahui potensi gangguan tanpa harus terus-menerus membuka dashboard.

Notifikasi Telegram dikirim ketika sistem mendeteksi status waspada, anomali, atau *trouble* sesuai aturan yang ditentukan. Notifikasi berisi informasi waktu kejadian, status sistem, sensor acuan, nilai estimasi suhu, dan keterangan kondisi. Untuk menghindari pengiriman pesan secara berulang dalam waktu singkat, sistem menerapkan jeda atau *cooldown* notifikasi.

Riwayat pengiriman notifikasi disimpan agar dapat ditelusuri kembali. Riwayat tersebut mencakup notifikasi yang berhasil dikirim, gagal dikirim, maupun dilewati karena aturan *cooldown*. Pencatatan ini diperlukan untuk kebutuhan pengujian dan pembuktian implementasi pada Bab 4.

3.1.1.11 Fungsi Evaluasi Model dan Baseline

Sistem harus mampu mengevaluasi kinerja model LSTM menggunakan metrik RMSE, MAE, dan MAPE. Evaluasi dilakukan dengan membandingkan nilai suhu aktual S2 dengan nilai suhu hasil estimasi. Karena keluaran model berupa nilai suhu kontinu, evaluasi model menggunakan metrik kesalahan estimasi, bukan hanya akurasi klasifikasi.

RMSE digunakan untuk melihat besar kesalahan model dengan penalti lebih tinggi pada error yang besar. MAE digunakan untuk mengetahui rata-rata selisih absolut antara nilai aktual dan nilai estimasi. MAPE digunakan untuk mengetahui besar kesalahan dalam bentuk persentase. Ketiga metrik tersebut digunakan agar performa model dapat dianalisis dari beberapa sudut pandang.

Selain mengevaluasi LSTM, sistem juga membandingkan hasil LSTM dengan baseline sederhana, yaitu *persistence model* dan *moving average*. *Persistence model* menggunakan nilai suhu terakhir sebagai estimasi suhu berikutnya, sedangkan *moving average* menggunakan rata-rata beberapa data terakhir sebagai nilai estimasi. Perbandingan ini diperlukan agar performa LSTM dapat dievaluasi secara objektif.

3.1.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menjelaskan karakteristik kualitas sistem yang perlu dipenuhi agar sistem dapat berjalan dengan baik. Kebutuhan non-fungsional pada penelitian ini mencakup kinerja sistem, keandalan pembacaan sensor, integritas data *time-series*, kualitas estimasi model, keamanan dasar API, keterlacakan data, dan kemudahan penggunaan dashboard.

3.1.2.1 Kinerja Waktu-Nyata

Sistem dirancang untuk melakukan pembacaan data sensor secara periodik dan menampilkan data terbaru pada dashboard. Data yang dikirimkan oleh gateway perlu dapat diterima oleh backend, disimpan ke basis data, dan ditampilkan pada dashboard dalam waktu yang wajar.

Dashboard diharapkan mampu menampilkan pembaruan data secara *real-time* atau mendekati *real-time* melalui mekanisme API dan Server-Sent Events (SSE). Dengan demikian, pengguna dapat memantau perubahan suhu, kelembaban, status sensor, dan status termal tanpa harus selalu melakukan refresh halaman secara manual.

3.1.2.2 Keandalan Pembacaan Sensor

Sistem harus mampu menangani kondisi ketika sensor gagal terbaca, mengalami *timeout*, atau menghasilkan nilai yang tidak valid. Apabila kondisi tersebut terjadi, data tidak langsung digunakan sebagai data valid untuk proses estimasi. Sistem perlu mencatat kondisi tersebut sebagai log agar dapat ditelusuri pada tahap pengujian.

Keandalan pembacaan sensor penting karena kualitas data sensor akan memengaruhi data historis, proses pra-pemrosesan, hasil pelatihan model, hasil estimasi suhu, dan status yang ditampilkan pada dashboard.

3.1.2.3 Integritas Data Time-Series

Setiap data sensor yang disimpan harus memiliki informasi waktu pembacaan, identitas sensor, nilai suhu, nilai kelembaban, sumber data, dan status

kualitas data. Informasi waktu digunakan untuk menjaga urutan data, sedangkan identitas sensor digunakan untuk membedakan data yang berasal dari S1 dan S2.

Integritas data *time-series* perlu dijaga karena proses pelatihan dan evaluasi LSTM membutuhkan urutan data yang benar. Oleh karena itu, pembagian data untuk pelatihan, validasi, dan pengujian dilakukan secara kronologis agar tidak terjadi kebocoran informasi dari data masa depan ke data masa lalu.

3.1.2.4 Kualitas Estimasi Model

Model LSTM perlu dievaluasi menggunakan metrik kesalahan estimasi. Metrik yang digunakan dalam penelitian ini adalah RMSE, MAE, dan MAPE. RMSE digunakan untuk melihat besar kesalahan dengan penalti lebih tinggi terhadap error besar. MAE digunakan untuk melihat rata-rata kesalahan absolut dalam satuan derajat Celsius. MAPE digunakan untuk melihat persentase kesalahan estimasi terhadap nilai aktual.

Kualitas model tidak hanya dilihat dari kecilnya nilai error, tetapi juga dari kemampuan hasil estimasi dalam mendukung mekanisme *early warning system*. Oleh karena itu, hasil estimasi suhu S2 juga digunakan sebagai dasar klasifikasi status termal normal, waspada, dan anomali.

3.1.2.5 Keamanan Dasar API

Endpoint API yang menerima data dari Raspberry Pi gateway dan ML Worker perlu memiliki mekanisme keamanan dasar. Mekanisme keamanan dapat

berupa token pada header request, validasi format payload, dan pembatasan akses pada endpoint tertentu.

Keamanan dasar API diperlukan agar backend tidak menerima data dari sumber yang tidak dikenal. Hal ini penting karena data yang masuk ke backend akan disimpan sebagai data historis dan dapat digunakan dalam proses monitoring, estimasi suhu, dan penentuan status sistem.

3.1.2.6 Keterlacakan Data dan Log

Sistem harus mampu mencatat data dan kejadian penting agar proses monitoring dan pengujian dapat ditelusuri kembali. Data yang perlu dicatat meliputi data pembacaan sensor, hasil estimasi suhu, metrik evaluasi model, hasil baseline, event status, riwayat notifikasi, dan log sistem.

Keterlacakan data diperlukan untuk mendukung pembuktian implementasi pada Bab 4. Dengan adanya data dan log yang tersimpan, hasil pembacaan sensor, proses estimasi, status sistem, dan pengiriman notifikasi dapat diverifikasi melalui basis data, API, dashboard, maupun output terminal.

3.1.2.7 Kemudahan Penggunaan Dashboard

Dashboard harus dirancang agar pengguna dapat memahami kondisi sistem dengan cepat. Informasi utama seperti suhu S1, kelembaban S1, suhu S2, kelembaban S2, hasil estimasi suhu S2, status termal, status gateway, dan riwayat event perlu ditampilkan secara jelas.

Status normal, waspada, anomali, dan *trouble* perlu dibuat mudah dibedakan agar pengguna dapat segera mengetahui kondisi sistem. Selain itu, grafik, tabel, dan layout sensor perlu disusun secara ringkas agar dashboard dapat digunakan sebagai media pemantauan utama.

3.1.3 Kebutuhan Perangkat Keras

Kebutuhan perangkat keras merupakan komponen fisik yang digunakan untuk membangun EMS server *testbed*. Perangkat keras yang digunakan meliputi laptop sebagai server *testbed* dan pusat pengolahan sistem, Raspberry Pi sebagai gateway, sensor suhu dan kelembaban, USB RS485 Converter, serta perangkat pendukung komunikasi dan daya.

3.1.3.1 Laptop sebagai Server Testbed dan Pusat Sistem

Laptop digunakan sebagai objek uji yang berperan sebagai server *testbed*. Laptop dipilih karena dapat merepresentasikan perangkat komputasi yang menghasilkan panas selama menjalankan beban kerja tertentu. Beban kerja dapat berupa aktivitas komputasi ringan atau sedang yang aman untuk perangkat.

Selain sebagai objek uji, laptop juga digunakan untuk menjalankan komponen pusat sistem, yaitu backend API, basis data PostgreSQL, dashboard monitoring, dan ML Worker. Dengan rancangan ini, Raspberry Pi difokuskan sebagai gateway sensor, sedangkan proses pengolahan data dan estimasi suhu dilakukan pada laptop.

3.1.3.2 Raspberry Pi Gateway

Raspberry Pi digunakan sebagai gateway yang menghubungkan sensor dengan backend. Perangkat ini bertugas membaca data dari sensor XY-MD02 melalui USB RS485 Converter, memberikan waktu pembacaan, membentuk payload data, dan mengirimkan data ke backend melalui jaringan.

Pada penelitian ini, Raspberry Pi difokuskan sebagai perangkat akuisisi data sensor. Pemisahan peran ini dilakukan agar gateway tetap ringan dan stabil dalam menjalankan tugas pembacaan sensor serta pengiriman data.

3.1.3.3 Sensor XY-MD02 S1 Ambient

Sensor XY-MD02 S1 digunakan untuk membaca suhu dan kelembaban lingkungan sekitar. Sensor ini ditempatkan pada area *ambient* atau referensi lingkungan yang tidak terlalu dekat dengan sumber panas langsung.

Data dari S1 digunakan sebagai fitur referensi untuk menggambarkan kondisi lingkungan sekitar server *testbed*. Data ini juga digunakan sebagai pembandingan terhadap perubahan suhu yang terjadi pada area *hotspot*.

3.1.3.4 Sensor XY-MD02 S2 Hotspot atau Exhaust

Sensor XY-MD02 S2 ditempatkan pada area *hotspot* atau *exhaust* yang lebih dekat dengan sumber panas server *testbed*. Sensor ini digunakan untuk membaca perubahan suhu pada area yang lebih sensitif terhadap kenaikan panas.

Suhu S2 digunakan sebagai target utama estimasi LSTM karena posisi sensor tersebut lebih mewakili area yang perlu dipantau dalam mekanisme *early warning system*. Data kelembaban S2 tetap digunakan sebagai fitur pendukung dalam proses pemodelan.

3.1.3.5 USB RS485 Converter

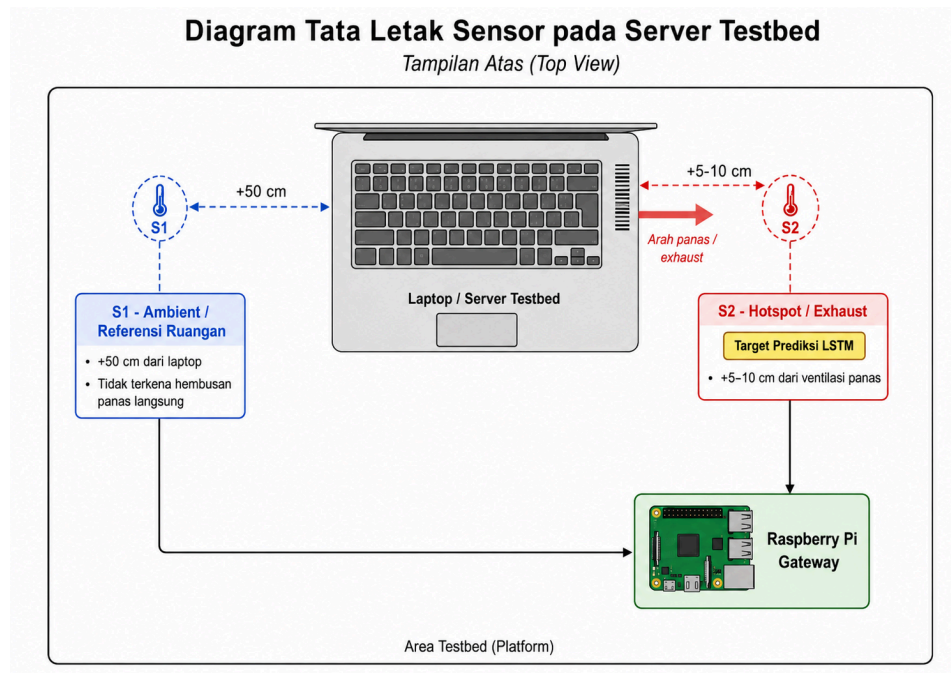
USB RS485 Converter digunakan sebagai penghubung antara sensor XY-MD02 dan Raspberry Pi. Sensor XY-MD02 menggunakan komunikasi Modbus RTU melalui RS485, sehingga diperlukan converter agar data sensor dapat dibaca oleh Raspberry Pi melalui antarmuka USB.

Komponen ini berperan penting dalam proses akuisisi data karena menjadi jembatan komunikasi antara sensor dan gateway.

3.1.3.6 Perangkat Pendukung Komunikasi dan Daya

Perangkat pendukung komunikasi dan daya digunakan untuk memastikan seluruh komponen dapat terhubung dan berjalan stabil selama proses pengujian. Perangkat ini meliputi kabel RS485, kabel USB, adaptor Raspberry Pi, jaringan lokal, serta perangkat pendukung lain yang diperlukan untuk menghubungkan sensor, gateway, backend, dan dashboard.

Gambar 3.1 Tata Letak Sensor pada Server Testbed



3.1.4 Kebutuhan Perangkat Lunak

Kebutuhan perangkat lunak merupakan komponen *software* yang digunakan untuk membangun sistem EMS, mulai dari gateway, backend, basis data, ML Worker, dashboard, hingga notifikasi Telegram.

3.1.4.1 Sistem Operasi Raspberry Pi

Raspberry Pi menjalankan sistem operasi yang mendukung program pembacaan sensor dan komunikasi jaringan. Program pada Raspberry Pi digunakan untuk membaca sensor XY-MD02 melalui USB RS485 Converter, membentuk payload data, dan mengirimkan data ke backend.

3.1.4.2 Go/Golang Backend

Go/Golang digunakan untuk membangun backend sistem. Backend bertugas menerima data sensor dari Raspberry Pi, melakukan validasi data, menyimpan data ke basis data, menyediakan API, mengirim pembaruan data ke dashboard melalui SSE, menerima hasil estimasi dari ML Worker, mengklasifikasikan status, dan memicu notifikasi Telegram.

Go dipilih karena memiliki performa yang baik untuk membangun service backend, API, dan proses monitoring yang ringan.

3.1.4.3 PostgreSQL

PostgreSQL digunakan sebagai basis data utama untuk menyimpan data sistem. Data yang disimpan meliputi data gateway, sensor, pembacaan sensor, hasil estimasi suhu, status termal, versi model, metrik evaluasi, hasil baseline, layout sensor, event, log sistem, dan riwayat notifikasi.

Data sensor yang dikumpulkan secara periodik membutuhkan penyimpanan yang mampu mengelola *timestamp*, query historis, dan data berurutan. Oleh karena itu, PostgreSQL digunakan untuk mendukung penyimpanan data *time-series* pada sistem EMS.

3.1.4.4 Python untuk ML Worker LSTM

Python digunakan untuk membangun ML Worker. Modul ini mencakup pengambilan data dari basis data, pra-pemrosesan data, pembentukan *window*,

pelatihan model LSTM, evaluasi model, perbandingan baseline, dan inferensi estimasi suhu S2.

Python dipilih karena memiliki ekosistem *machine learning* yang lengkap dan mendukung library seperti TensorFlow/Keras, Pandas, NumPy, dan Scikit-learn.

3.1.4.5 React, Vite, Tailscript dan Tailwind CSS

React, Vite, TypeScript, dan Tailwind CSS digunakan untuk membangun dashboard monitoring. Dashboard berfungsi untuk menampilkan data sensor, grafik suhu, grafik kelembaban, hasil estimasi suhu S2, status sistem, metrik evaluasi model, visualisasi layout sensor, event, dan riwayat notifikasi.

Penggunaan dashboard berbasis web memudahkan pengguna dalam mengakses informasi sistem melalui browser.

3.1.4.6 Chart.js

Chart.js digunakan untuk menampilkan grafik pada dashboard. Grafik yang dibutuhkan meliputi grafik suhu aktual per sensor, grafik kelembaban, grafik suhu aktual dibandingkan estimasi, grafik riwayat estimasi, dan grafik evaluasi model.

Grafik digunakan agar perubahan suhu, kelembaban, dan hasil estimasi dapat dipahami secara visual.

3.1.4.7 Telegram Bot API

Telegram Bot API digunakan untuk mengirimkan notifikasi kepada pengguna ketika sistem mendeteksi status waspada, anomali, atau *trouble*. Fitur ini berfungsi sebagai media peringatan dini agar pengguna dapat mengetahui kondisi penting tanpa harus selalu membuka dashboard.

3.1.5 Kebutuhan Data

Data yang digunakan dalam penelitian ini adalah data suhu dan kelembaban yang diperoleh dari dua sensor XY-MD02 pada lingkungan server *testbed*. Data dikumpulkan secara periodik oleh Raspberry Pi gateway dan dikirimkan ke backend untuk disimpan ke basis data.

Setiap data memiliki atribut minimal berupa waktu pembacaan, identitas sensor, nilai suhu, nilai kelembaban, sumber data, dan status kualitas data. Data tersebut digunakan untuk monitoring, penyimpanan historis, visualisasi dashboard, pra-pemrosesan, pelatihan model LSTM, pengujian model, evaluasi, inferensi, dan klasifikasi status.

Untuk kebutuhan ML Worker, data sensor diolah menjadi interval satu menit melalui proses *resampling*. Data historis tersebut kemudian disusun menjadi *window* input. Target estimasi model adalah suhu S2 lima menit ke depan. Dengan demikian, model tidak menghasilkan estimasi untuk seluruh sensor secara terpisah, tetapi difokuskan pada suhu S2 sebagai sensor utama pada area *hotspot*.

Tabel 3.2 Kebutuhan Data Model Estimasi

Komponen	Keterangan
Fitur input	Suhu S1, kelembaban S1, suhu S2, kelembaban S2
Target prediksi	Suhu S2 Lima Menit Kedepan
Data raw	Pembacaan Sensor dari Raspberry PI Gateway
Interval data ML	1 menit setelah proses <i>resampling</i>
Window input	30 data hasil <i>resampling</i>
Durasi window	30 menit data Historis
Horizon prediksi	5 menit ke depan
Output model	Nilai estimasi suhu S2
Output sistem	Status normal, waspada, anomali atau trouble

3.1.6 Asumsi dan Batasan Implementasi

Agar implementasi sistem tetap sesuai dengan ruang lingkup penelitian, asumsi dan batasan implementasi yang digunakan adalah sebagai berikut:

1. Objek uji berupa laptop atau perangkat komputasi yang berperan sebagai *server testbed*.
2. Lingkungan pengujian berupa meja terbuka atau mini *testbed platform* yang memungkinkan penempatan sensor S1 dan S2 secara terpisah.
3. Sistem menggunakan dua sensor suhu dan kelembaban XY-MD02, yaitu S1 dan S2.
4. Sensor S1 digunakan sebagai sensor *ambient* atau referensi lingkungan.
5. Sensor S2 digunakan sebagai sensor *hotspot* atau *exhaust* dan menjadi target utama estimasi suhu.

6. Sensor XY-MD02 dibaca oleh Raspberry Pi menggunakan komunikasi Modbus RTU melalui RS485 dengan bantuan USB RS485 Converter.
7. Data mentah dari gateway dikirimkan secara periodik ke backend dan disimpan sebagai data historis.
8. Data untuk kebutuhan ML Worker diolah menjadi interval satu menit melalui proses *resampling*.
9. Window input menggunakan 30 data hasil *resampling* yang merepresentasikan 30 menit data historis.
10. Model LSTM digunakan untuk mengestimasi suhu S2 lima menit ke depan.
11. Status termal ditentukan berdasarkan ambang batas operasional penelitian, yaitu normal, waspada, dan anomali.
12. Status *trouble* digunakan untuk menunjukkan gangguan teknis pada sensor, gateway, data, model, atau komponen pendukung sistem.
13. Evaluasi model dilakukan menggunakan RMSE, MAE, MAPE, serta perbandingan dengan baseline sederhana.
14. Dashboard menyediakan visualisasi data sensor, grafik historis, hasil estimasi suhu, status sistem, layout posisi sensor, event, dan riwayat notifikasi sebagai fitur pendukung monitoring.

3.2 Pemilihan Metode dan Teknologi

Pemilihan metode dan teknologi dilakukan untuk menentukan pendekatan serta perangkat yang digunakan dalam pengembangan sistem. Pada penelitian ini, sistem dikembangkan menggunakan pendekatan perekayasa dengan beberapa

komponen utama, yaitu sensor suhu dan kelembaban, Raspberry Pi gateway, backend API, basis data, dashboard monitoring, ML Worker, dan notifikasi Telegram.

Pemilihan teknologi dilakukan berdasarkan kebutuhan sistem yang telah dijelaskan pada tahap rekayasa kebutuhan. Setiap teknologi dipilih agar dapat mendukung proses pemantauan lingkungan server, pengelolaan data *time-series*, estimasi suhu S2 menggunakan LSTM, klasifikasi status termal, serta pengiriman notifikasi peringatan dini.

3.2.1 Pemilihan Metode Prototyping

Metode *prototyping* digunakan karena sistem yang dikembangkan melibatkan integrasi perangkat keras dan perangkat lunak. Melalui metode ini, sistem dapat dibangun secara bertahap mulai dari pembacaan sensor, pengiriman data ke backend, penyimpanan data, visualisasi dashboard, pengolahan data *time-series*, hingga penerapan model LSTM.

Pendekatan *prototyping* sesuai digunakan pada penelitian ini karena setiap komponen dapat diuji secara bertahap sebelum digabungkan menjadi sistem yang utuh. Misalnya, sensor dan gateway dapat diuji terlebih dahulu untuk memastikan data suhu dan kelembaban terbaca. Setelah itu, backend dan basis data diuji untuk memastikan data dapat diterima dan disimpan. Tahap berikutnya adalah pengujian dashboard, ML Worker, klasifikasi status, dan notifikasi Telegram.

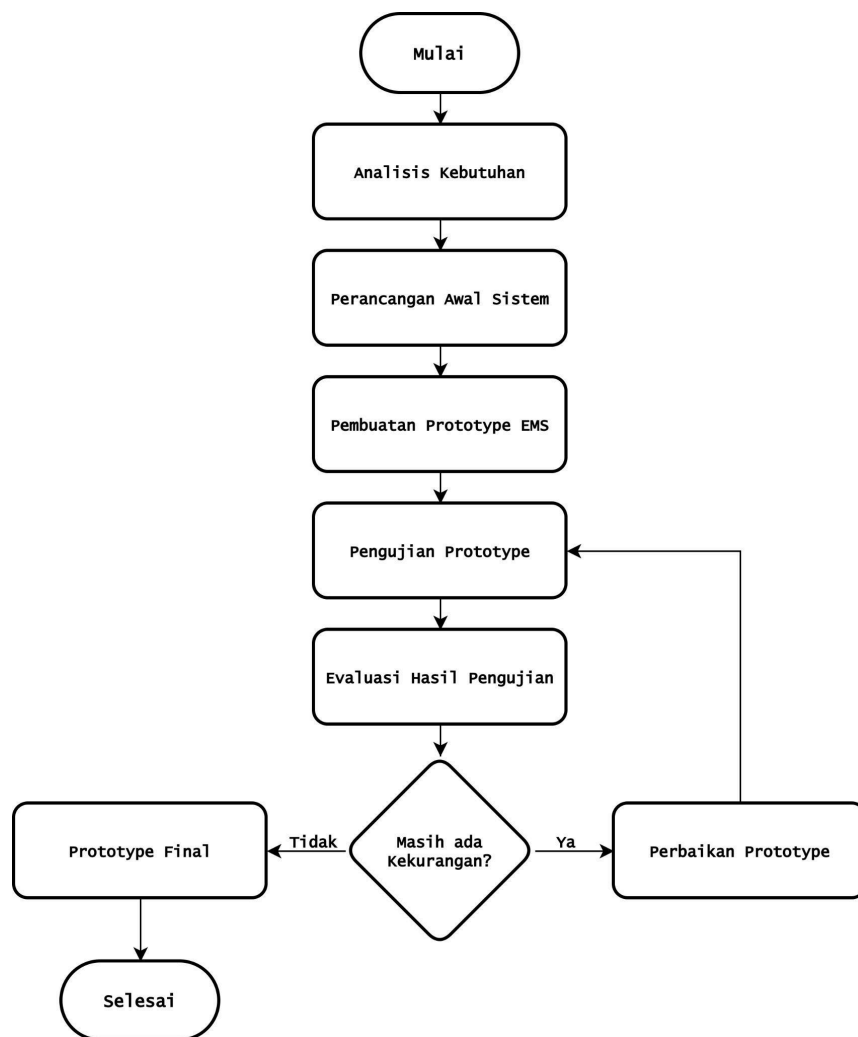
Dengan menggunakan pendekatan ini, kesalahan pada salah satu komponen dapat ditemukan lebih awal. Hal tersebut penting karena sistem EMS yang dikembangkan terdiri dari beberapa bagian yang saling terhubung.

Tahapan prototyping pada penelitian ini terdiri dari:

1. **Pengumpulan kebutuhan**, yaitu menentukan kebutuhan perangkat keras, perangkat lunak, data, dan fungsi sistem.
2. **Perancangan awal**, yaitu membuat rancangan arsitektur, flowchart, database, API, dashboard, dan integrasi LSTM.
3. **Pembuatan prototype**, yaitu membangun sistem awal berdasarkan rancangan.
4. **Pengujian prototype**, yaitu menguji pembacaan sensor, backend, database, dashboard, prediksi, dan notifikasi.
5. **Evaluasi dan perbaikan**, yaitu memperbaiki sistem berdasarkan hasil pengujian.
6. **Prototype final**, yaitu sistem yang telah diperbaiki dan siap diuji pada

Bab 4.

Gambar 3.2 Tahapan Metode Prototyping



3.2.2 Pemilihan Raspberry Pi sebagai Gateway

Raspberry Pi dipilih sebagai gateway karena perangkat ini mampu menjalankan program pembacaan sensor, mendukung komunikasi serial melalui USB RS485 Converter, dan memiliki konektivitas jaringan untuk mengirimkan data ke backend. Raspberry Pi juga cukup ringan untuk digunakan sebagai perangkat penghubung antara sensor dan sistem pusat.

Pada penelitian ini, Raspberry Pi bertugas membaca data dari sensor XY-MD02, membentuk payload data, dan mengirimkan data tersebut ke backend melalui HTTP REST API. Peran Raspberry Pi difokuskan pada akuisisi data sensor agar proses pengolahan data, penyimpanan, dashboard, dan ML Worker dapat dijalankan pada laptop sebagai pusat sistem.

Pemilihan Raspberry Pi sebagai gateway juga membuat sistem lebih modular. Backend tidak membaca sensor secara langsung, melainkan menerima data dari gateway. Dengan pemisahan tersebut, proses pengujian sensor, gateway, backend, dan dashboard dapat dilakukan secara lebih terstruktur.

3.2.3 Pemilihan Sensor XY-MD02 dan Modbus RS485

Sensor XY-MD02 dipilih karena mampu membaca dua parameter lingkungan, yaitu suhu dan kelembaban. Kedua parameter tersebut sesuai dengan kebutuhan EMS server yang berfokus pada pemantauan kondisi termal dan kelembaban lingkungan server *testbed*.

Sensor XY-MD02 menggunakan komunikasi Modbus RTU melalui RS485. Komunikasi RS485 dipilih karena umum digunakan pada perangkat sensor industri dan mendukung komunikasi antarperangkat melalui jalur kabel. Pada sistem ini, dua sensor digunakan dengan peran yang berbeda, yaitu S1 sebagai sensor *ambient* atau referensi lingkungan dan S2 sebagai sensor *hotspot* atau *exhaust*.

Data suhu S2 digunakan sebagai target estimasi LSTM, sedangkan suhu dan kelembaban S1 digunakan sebagai fitur referensi lingkungan. Dengan pembagian tersebut, sistem dapat memanfaatkan informasi dari dua titik pengukuran untuk mendukung estimasi suhu S2 lima menit ke depan..

3.2.4 Pemilihan Go/Golang sebagai Backend

Go/Golang dipilih sebagai bahasa pemrograman backend karena memiliki performa yang baik, struktur yang sederhana, serta cocok digunakan untuk membangun REST API. Backend berperan sebagai pusat komunikasi antara gateway, basis data, dashboard, ML Worker, dan Telegram.

Backend memiliki beberapa fungsi utama, yaitu menerima data sensor dari gateway, melakukan validasi payload, menyimpan data ke basis data, menyediakan API untuk dashboard, menerima hasil estimasi dari ML Worker, mengklasifikasikan status, mencatat event, dan memproses notifikasi Telegram.

Backend juga mendukung komunikasi *real-time* atau mendekati *real-time* melalui Server-Sent Events. Dengan demikian, perubahan data sensor, hasil estimasi, atau status sistem dapat dikirimkan ke dashboard tanpa harus selalu menunggu proses refresh manual.

3.2.5 Pemilihan PostgreSQL sebagai Basis Data Time-Series

PostgreSQL dipilih sebagai basis data utama karena mampu menyimpan data relasional secara terstruktur dan mendukung pengelolaan data historis. Pada penelitian ini, data sensor disimpan berdasarkan waktu pembacaan sehingga memiliki karakteristik *time-series*.

PostgreSQL digunakan untuk menyimpan data gateway, sensor, pembacaan sensor, hasil estimasi suhu, versi model, metrik evaluasi, hasil baseline, event, layout sensor, log sistem, dan riwayat notifikasi. Dengan struktur tersebut, setiap proses dalam sistem dapat ditelusuri melalui data yang tersimpan.

3.2.6 Pemilihan Python untuk ML Worker LSTM

Python dipilih untuk membangun ML Worker karena memiliki dukungan library yang kuat untuk pengolahan data dan *machine learning*. Pada penelitian ini, Python digunakan bersama library seperti TensorFlow/Keras, Pandas, NumPy, Scikit-learn, dan Joblib.

ML Worker bertugas mengambil data sensor dari basis data, melakukan pra-pemrosesan, melakukan *resampling*, membentuk *window* data, melatih model LSTM, mengevaluasi model, membandingkan hasil dengan baseline, dan melakukan inferensi suhu S2 lima menit ke depan.

Pemilihan Python untuk ML Worker juga memisahkan proses *machine learning* dari backend. Backend tetap berfokus pada API, validasi, penyimpanan, status, dan notifikasi, sedangkan ML Worker berfokus pada proses estimasi suhu. Pemisahan ini membuat sistem lebih mudah dikembangkan dan diuji.

3.2.7 Pemilihan Long Short-Term Memory

Long Short-Term Memory dipilih karena data suhu dan kelembaban yang digunakan dalam penelitian ini memiliki karakteristik *time-series*. LSTM mampu mempelajari hubungan temporal pada data berurutan sehingga sesuai digunakan untuk mengestimasi suhu berdasarkan data historis.

Pada penelitian ini, LSTM digunakan untuk mengestimasi suhu S2 lima menit ke depan. Fitur input yang digunakan adalah suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Data tersebut diolah menjadi *window* historis sebelum digunakan oleh model.

Hasil estimasi LSTM tidak langsung menjadi tujuan akhir penelitian, tetapi digunakan sebagai komponen pendukung *early warning system*. Nilai estimasi suhu S2 kemudian dibandingkan dengan ambang batas operasional untuk menghasilkan status normal, waspada, atau anomali.

3.2.8 Pemilihan Server-Sent Events

Server-Sent Events atau SSE dipilih untuk mendukung pembaruan data dari backend ke dashboard. SSE digunakan karena kebutuhan sistem adalah mengirimkan pembaruan satu arah dari server ke browser ketika terdapat data baru, status baru, atau event baru.

SSE lebih sederhana dibandingkan WebSocket untuk kebutuhan sistem ini karena dashboard hanya perlu menerima pembaruan dari backend. Data yang

dapat diperbarui melalui SSE mencakup data sensor terbaru, status gateway, status sensor, hasil estimasi suhu, event, dan notifikasi.

Dengan menggunakan SSE, dashboard dapat menampilkan kondisi sistem secara lebih responsif tanpa harus selalu melakukan refresh manual.

3.2.9 Pemilihan Telegram Bot API

Telegram Bot API dipilih sebagai media notifikasi karena dapat mengirim pesan otomatis kepada pengguna atau grup tertentu. Notifikasi diperlukan agar pengguna dapat mengetahui kondisi penting tanpa harus selalu membuka dashboard.

Pada penelitian ini, Telegram digunakan untuk mengirimkan peringatan ketika sistem mendeteksi status waspada, anomali, atau *trouble*. Pesan notifikasi dapat berisi waktu kejadian, status sistem, sensor acuan, nilai estimasi suhu, dan keterangan kondisi.

Telegram juga dipilih karena implementasinya relatif sederhana dan sesuai untuk kebutuhan sistem *early warning* pada skala penelitian. Selain itu, riwayat notifikasi tetap disimpan pada basis data agar proses pengiriman pesan dapat ditelusuri pada tahap pengujian.

3.2.10 Pemilihan Baseline Sederhana

Baseline sederhana dipilih sebagai pembanding performa model LSTM. Pada penelitian ini, baseline yang digunakan adalah *persistence model* dan

moving average. Kedua baseline tersebut digunakan untuk mengetahui apakah LSTM memberikan hasil estimasi yang lebih baik dibandingkan metode sederhana.

Persistence model menggunakan nilai suhu terakhir sebagai estimasi suhu pada periode berikutnya. Sementara itu, *moving average* menggunakan rata-rata beberapa data terakhir sebagai estimasi. Kedua metode ini sederhana, tetapi penting sebagai pembanding karena data suhu pada lingkungan server dapat memiliki pola yang relatif stabil.

Perbandingan antara LSTM dan baseline dilakukan menggunakan metrik RMSE, MAE, dan MAPE. Hasil perbandingan tersebut digunakan untuk menilai kualitas model secara lebih objektif.

3.3 Perancangan Proses

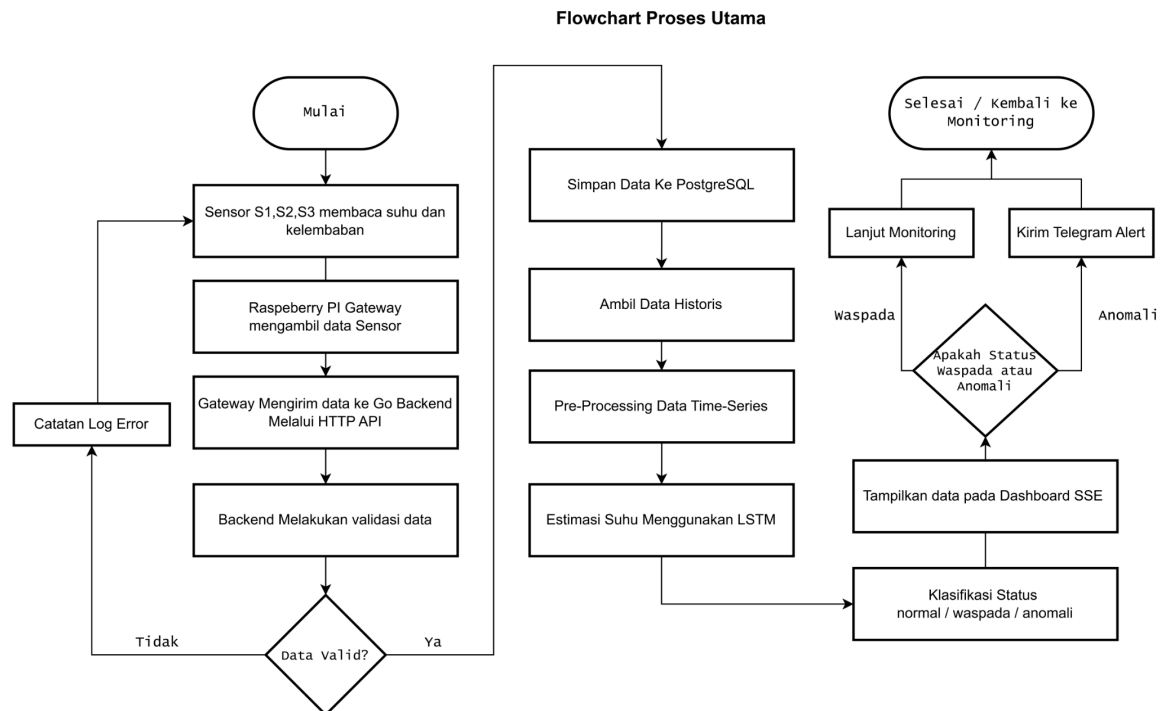
Perancangan proses menjelaskan alur kerja sistem dari pembacaan sensor hingga keluaran berupa dashboard monitoring, estimasi suhu S2, status termal, visualisasi layout sensor, dan notifikasi Telegram. Perancangan proses dibuat agar setiap tahapan dalam sistem dapat dipahami secara terstruktur sebelum masuk ke tahap implementasi.

Secara umum, proses sistem dimulai dari pembacaan sensor oleh Raspberry Pi gateway. Data sensor kemudian dikirim ke backend untuk divalidasi dan disimpan ke basis data. Data historis yang terkumpul digunakan oleh ML

Worker untuk melakukan pra-pemrosesan, pelatihan model, evaluasi, dan inferensi menggunakan LSTM. Hasil inferensi berupa estimasi suhu S2 lima menit ke depan dibandingkan dengan ambang batas operasional untuk menentukan status normal, waspada, atau anomali. Selain itu, sistem juga dapat menghasilkan status *trouble* apabila terdapat gangguan teknis pada sensor, gateway, data, model, atau komponen pendukung.

Status yang dihasilkan kemudian ditampilkan pada dashboard, divisualisasikan melalui marker sensor pada layout, dan dikirimkan melalui Telegram apabila memenuhi kondisi peringatan. Dengan alur tersebut, sistem tidak hanya menampilkan kondisi aktual, tetapi juga mendukung mekanisme *early warning system* berdasarkan hasil estimasi suhu S2.

Gambar 3.3 Flowchart Proses Utama EMS



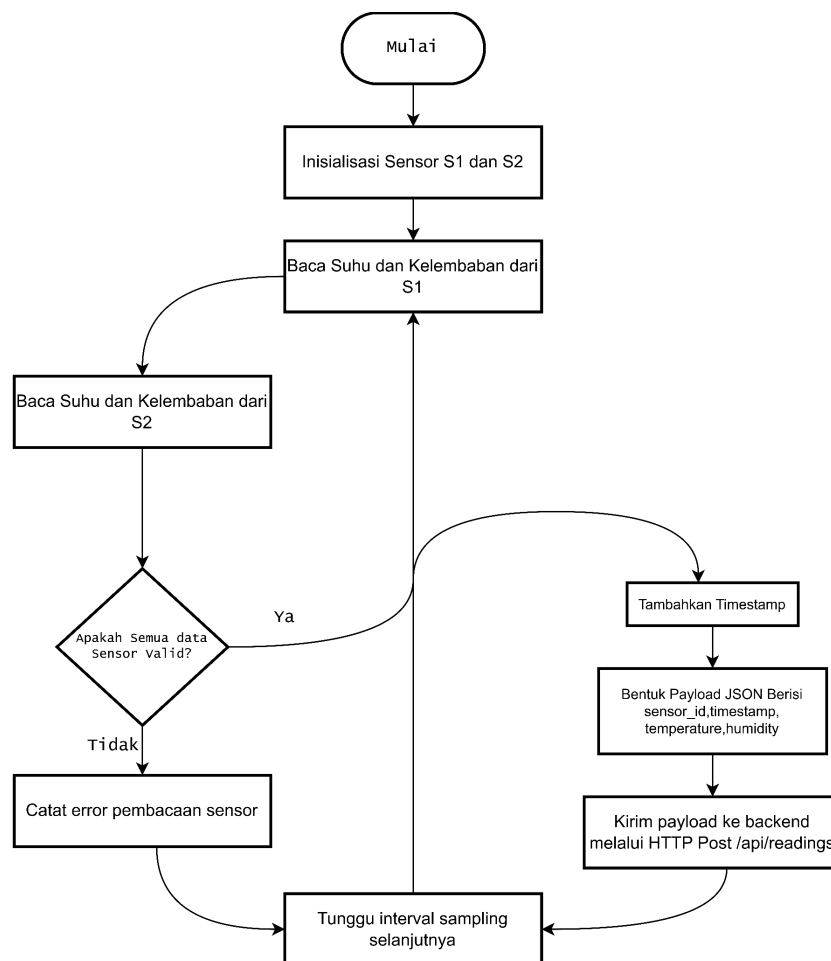
3.3.1 Proses Akuisisi Data Sensor

Proses akuisisi data dimulai ketika Raspberry Pi membaca data dari sensor XY-MD02 S1 dan S2 melalui komunikasi Modbus RTU RS485. Sensor S1 membaca suhu dan kelembaban pada area *ambient* atau referensi lingkungan, sedangkan sensor S2 membaca suhu dan kelembaban pada area *hotspot* atau *exhaust server testbed*.

Data dari sensor dibaca melalui USB RS485 Converter yang terhubung ke Raspberry Pi. Setiap data yang berhasil dibaca diberi waktu pembacaan agar dapat disimpan sebagai data *time-series*. Data yang dibaca meliputi kode sensor, nilai suhu, nilai kelembaban, waktu pembacaan, sumber data, dan status kualitas data.

Apabila sensor gagal terbaca, mengalami *timeout*, atau menghasilkan nilai yang tidak sesuai, gateway menandai data tersebut sebagai kondisi yang perlu diperiksa. Kondisi ini tidak langsung digunakan sebagai data valid untuk kebutuhan estimasi model, tetapi dicatat agar dapat ditelusuri melalui log sistem

Gambar 3.4 Flowchart Akuisisi Data Sensor



3.3.2 Proses Pengiriman Data Gateway ke Backend

Setelah data sensor berhasil dibaca, Raspberry Pi gateway membentuk payload dalam format JSON. Payload tersebut berisi identitas gateway, waktu

pembacaan, sumber data, serta daftar pembacaan sensor S1 dan S2. Setiap data pembacaan sensor memuat kode sensor, peran sensor, nilai suhu, nilai kelembaban, dan status kualitas data.

Data dikirimkan ke backend menggunakan HTTP POST melalui endpoint `POST /api/v1/readings`. Backend menerima payload tersebut dan melakukan validasi awal terhadap token gateway, struktur payload, identitas gateway, identitas sensor, waktu pembacaan, nilai suhu, nilai kelembaban, sumber data, dan status kualitas data.

Apabila data tidak sesuai format, backend mengembalikan respons error dan mencatat log kesalahan. Apabila data sesuai format, backend melanjutkan proses penyimpanan data ke basis data.

Contoh payload data sensor adalah sebagai berikut:

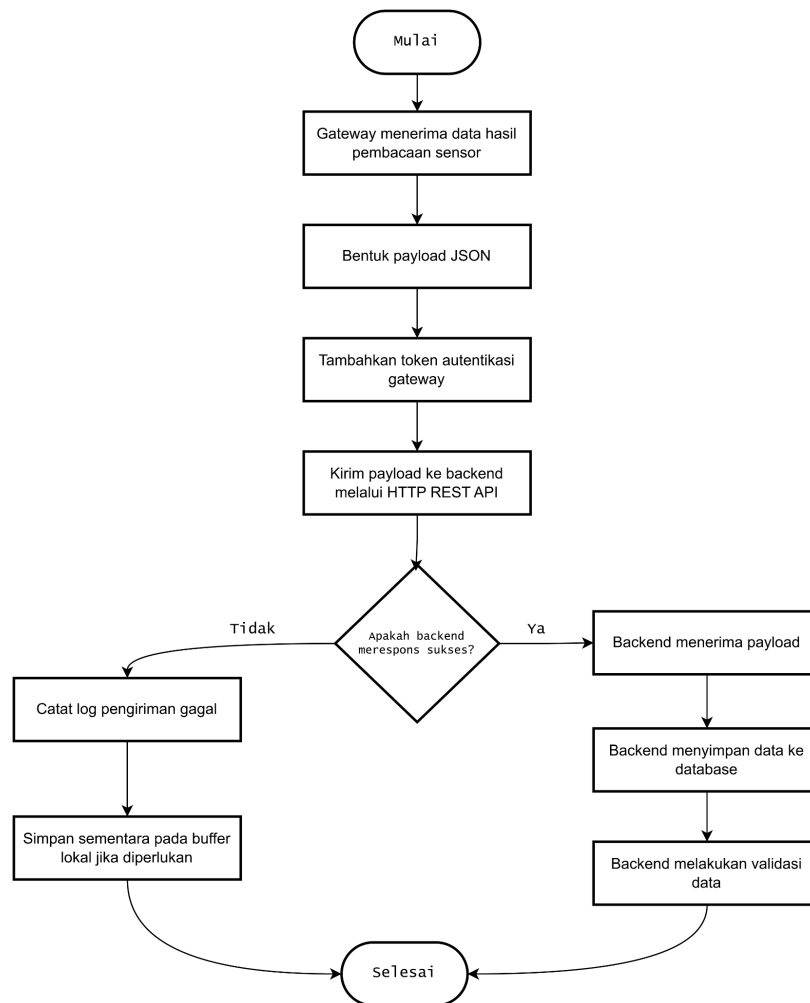
```
{
  "gateway_code": "raspi-gateway-01",
  "recorded_at": "2026-05-16T10:15:00+07:00",
  "source": "hardware",
  "readings": [
    {
      "sensor_code": "S1",
      "sensor_role": "ambient",
      "temperature": 27.4,
      "humidity": 63.2,
      "quality_status": "valid"
    },
    {
      "sensor_code": "S2",
      "sensor_role": "hotspot",
      "temperature": 31.2,
      "humidity": 58.4,
    }
  ]
}
```

```
        "quality_status": "valid"  
    }  
]  
}
```

Field `gateway_code` digunakan untuk mengidentifikasi Raspberry Pi gateway yang mengirimkan data. Field `recorded_at` digunakan sebagai penanda waktu pembacaan data. Field `source` menunjukkan sumber data, misalnya `hardware` untuk data sensor asli. Field `readings` berisi daftar pembacaan sensor S1 dan S2.

Di dalam `readings`, field `sensor_code` digunakan untuk membedakan sensor S1 dan S2. Field `sensor_role` digunakan untuk menunjukkan peran sensor, yaitu `ambient` untuk S1 dan `hotspot` untuk S2. Field `temperature` dan `humidity` digunakan sebagai nilai hasil pembacaan sensor, sedangkan `quality_status` digunakan untuk menunjukkan kualitas data yang diterima.

Gambar 3.5 Flowchart Proses Pengiriman Data Gateway ke Backend



3.3.3 Proses Penyimpanan Data Time-Series

Data sensor yang telah divalidasi disimpan ke tabel *sensor_readings*.

Setiap data disimpan berdasarkan waktu pembacaan, identitas gateway, identitas sensor, nilai suhu, nilai kelembaban, sumber data, dan status kualitas data.

Struktur penyimpanan ini memungkinkan sistem mengambil data historis berdasarkan rentang waktu tertentu serta membedakan data yang berasal dari sensor S1 dan S2.

Data historis yang tersimpan digunakan untuk dua kebutuhan utama.

Pertama, data digunakan untuk visualisasi dashboard, seperti kartu nilai terbaru, grafik suhu, grafik kelembaban, dan riwayat pembacaan sensor. Kedua, data digunakan sebagai dataset untuk pra-pemrosesan, pelatihan model LSTM, pengujian model, evaluasi model, dan inferensi estimasi suhu S2.

Tabel penyimpanan utama pada proses ini adalah sebagai berikut:

Tabel	Fungsi
gateways	Menyimpan identitas Raspberry Pi gateway pengirim data
sensors	Menyimpan identitas sensor S1 dan S2 beserta perannya
sensor_readings	Menyimpan data suhu dan kelembaban berdasarkan waktu pembacaan
gateway_status_logs	Menyimpan riwayat status gateway dan sensor
system_logs	Menyimpan log sistem apabila terjadi error atau kondisi penting

Struktur field utama pada tabel sensor_readings dirancang sebagai berikut:

Field	Keterangan
id	Identitas unik data pembacaan
gateway_id	Relasi ke gateway pengirim data
sensor_id	Relasi ke sensor S1 atau S2
recorded_at	Waktu pembacaan sensor
temperature	Nilai suhu hasil pembacaan sensor
humidity	Nilai kelembaban hasil pembacaan sensor
source	Sumber data, misalnya hardware atau simulator
quality_status	Status kualitas data, misalnya valid, invalid, timeout, atau simulated
created_at	Waktu data disimpan ke sistem

Dengan rancangan tersebut, data pembacaan sensor dapat digunakan sebagai data historis yang terstruktur. Data ini menjadi dasar untuk monitoring

dashboard, proses pra-pemrosesan *time-series*, pelatihan dan evaluasi model LSTM, serta inferensi suhu S2 lima menit ke depan.

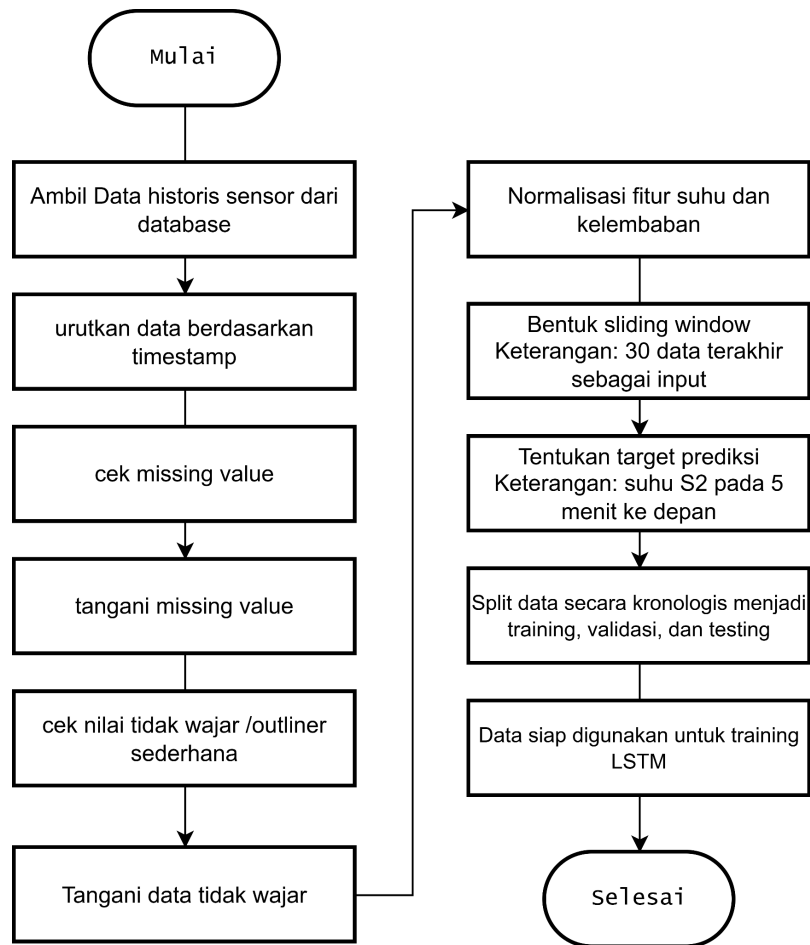
3.3.4 Proses Pra-pemrosesan Data

Pra-pemrosesan dilakukan sebelum data digunakan dalam model LSTM. Proses ini dimulai dengan mengambil data historis dari database. Data kemudian diurutkan berdasarkan waktu pembacaan untuk memastikan urutan data sesuai dengan karakteristik *time-series*.

Selanjutnya, sistem memeriksa *missing value*, nilai tidak wajar, dan konsistensi interval data. Data dari sensor S1 dan S2 digabungkan berdasarkan waktu agar setiap baris data dapat memuat suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Setelah itu, data diolah melalui proses *resampling* menjadi interval satu menit agar data yang digunakan oleh model memiliki jarak waktu yang seragam.

Data yang telah melalui proses *resampling* kemudian dinormalisasi agar skala fitur menjadi lebih seragam. Normalisasi diperlukan karena fitur suhu dan kelembaban memiliki rentang nilai yang berbeda. Setelah data dinormalisasi, data dibentuk menjadi *window input*. Pada rancangan ini, *window input* menggunakan 30 data hasil *resampling* untuk mengestimasi suhu S2 lima menit ke depan.

Gambar 3.6 Flowchart Pra-pemrosesan Data dan Pembentukan Window



3.3.5 Proses Training dan Evaluasi LSTM

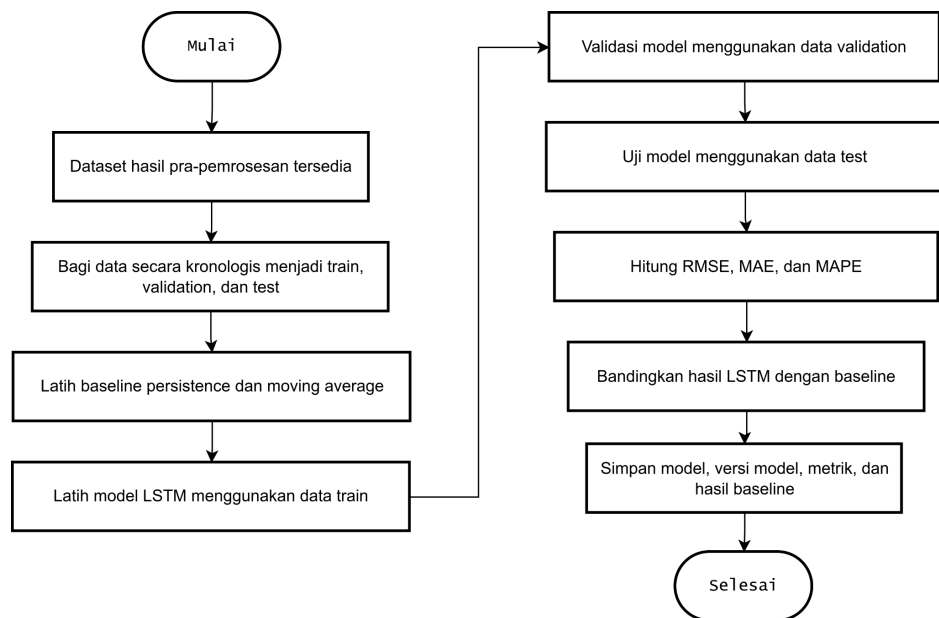
Proses *training* dilakukan menggunakan data historis yang telah melalui tahap pra-pemrosesan. Data dibagi menjadi data *training*, validasi, dan *testing* secara kronologis. Pembagian data secara kronologis digunakan karena data *time-series* memiliki urutan waktu yang tidak boleh diacak.

Model LSTM mempelajari pola historis suhu dan kelembaban dari S1 dan S2. Fitur input yang digunakan adalah suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Target estimasi adalah suhu S2 lima menit ke depan. Setelah

model dilatih, model diuji menggunakan data *testing*. Hasil estimasi dibandingkan dengan nilai aktual untuk menghitung RMSE, MAE, dan MAPE.

Selain itu, hasil LSTM dibandingkan dengan *baseline* sederhana. *Baseline* yang digunakan adalah *persistence model* dan *moving average*. Perbandingan ini digunakan untuk mengetahui apakah LSTM memiliki kinerja estimasi yang lebih baik dibandingkan metode dasar.

Gambar 3.7 Flowchart Training dan Evaluasi LSTM



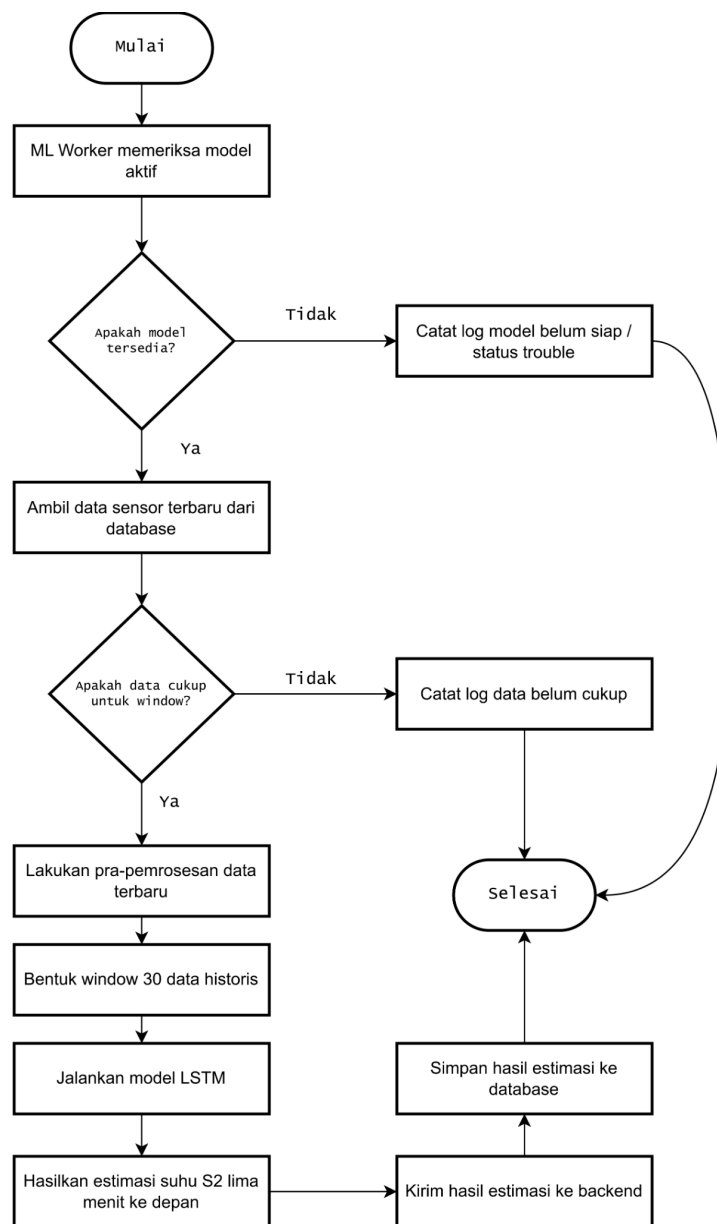
3.3.6 Proses Inferensi Estimasi Suhu S2

Proses inferensi dilakukan dengan mengambil data terbaru dari database sesuai jumlah *window* yang dibutuhkan. Data terbaru tersebut diproses menggunakan tahapan pra-pemrosesan yang sama, seperti pengurutan waktu, penggabungan data S1 dan S2, *resampling*, normalisasi, dan pembentukan *window input*.

Input inferensi menggunakan 30 data historis terakhir hasil *resampling*. Data tersebut digunakan oleh model LSTM yang telah dilatih untuk menghasilkan satu nilai estimasi suhu S2 pada lima menit ke depan. Dengan demikian, keluaran dari proses inferensi adalah estimasi suhu S2 pada waktu target, bukan deret estimasi untuk beberapa waktu sekaligus.

Hasil estimasi kemudian dikirimkan ke backend dan disimpan ke tabel *predictions*. Data hasil estimasi ini digunakan untuk ditampilkan pada dashboard dan diproses lebih lanjut dalam klasifikasi status termal.

Gambar 3.8 Flowchart Inferensi Estimasi Suhu S2



3.3.7 Proses Klasifikasi Status Termal

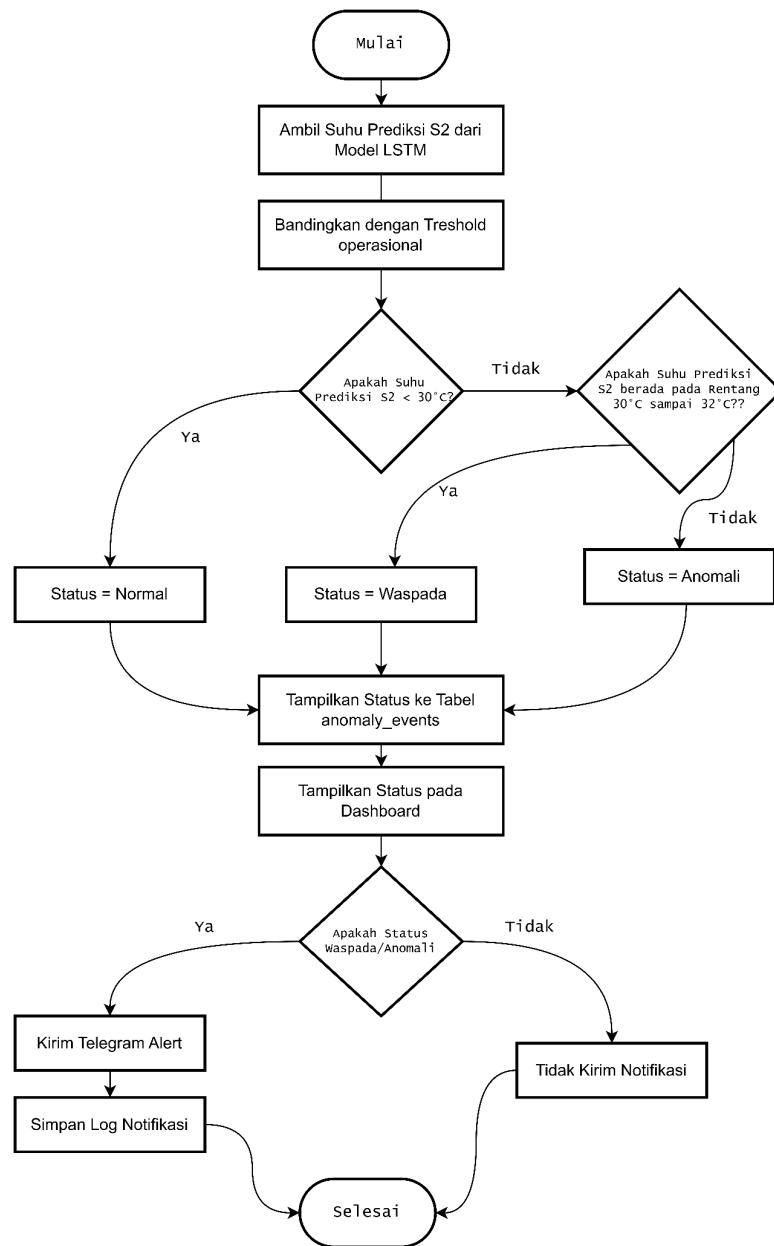
Setelah nilai estimasi suhu S2 diperoleh, sistem membandingkan nilai tersebut dengan ambang batas operasional yang telah ditentukan. Apabila estimasi suhu S2 berada di bawah 30°C, sistem memberikan status normal. Apabila estimasi suhu S2 berada pada rentang 30°C sampai 32°C, sistem memberikan

status waspada. Apabila estimasi suhu S2 berada di atas 32°C, sistem memberikan status anomali.

Status yang dihasilkan disimpan sebagai riwayat kejadian pada sistem. Status tersebut juga ditampilkan pada dashboard dan layout sensor agar pengguna dapat memahami kondisi lingkungan server *testbed* secara lebih mudah. Status termal normal, waspada, dan anomali digunakan untuk menunjukkan kondisi berdasarkan hasil estimasi suhu S2.

Selain status termal, sistem juga memeriksa status teknis. Apabila terdapat masalah seperti sensor tidak terbaca, gateway tidak aktif, data tidak valid, model belum siap, atau hasil estimasi sudah melewati masa berlaku, sistem dapat memberikan status *trouble*. Status *trouble* digunakan untuk menunjukkan gangguan teknis pada sistem dan dipisahkan dari status termal.

Gambar 3.9 Flowchart Klasifikasi Status dan Telegram Alert



3.3.8 Proses Monitoring Dashboard dan Layout Sensor

Dashboard digunakan untuk menampilkan hasil pemantauan sistem secara visual. Data yang ditampilkan pada dashboard berasal dari backend dan mencakup suhu serta kelembaban sensor S1 dan S2, grafik historis, hasil estimasi suhu S2, status sistem, metrik evaluasi model, riwayat event, dan riwayat notifikasi.

Selain dashboard utama, sistem juga memiliki visualisasi layout sensor. Layout sensor digunakan untuk menampilkan posisi S1 dan S2 pada gambar server *testbed* atau ruangan. Marker sensor pada layout dapat menunjukkan status normal, waspada, anomali, atau *trouble* sesuai kondisi terbaru yang diterima dari backend.

Dashboard menerima data melalui API dan pembaruan *real-time* atau mendekati *real-time* melalui Server-Sent Events. Dengan mekanisme ini, pengguna dapat memantau perubahan kondisi sensor, estimasi suhu, dan status sistem tanpa harus melakukan refresh halaman secara manual.

3.3.9 Proses Notifikasi Telegram

Proses notifikasi Telegram dilakukan ketika sistem mendeteksi status yang membutuhkan perhatian. Status yang dapat memicu notifikasi adalah waspada, anomali, atau *trouble*. Notifikasi digunakan sebagai media peringatan dini agar pengguna dapat mengetahui kondisi penting tanpa harus selalu membuka dashboard.

Sebelum mengirimkan notifikasi, sistem memeriksa konfigurasi Telegram dan aturan pengiriman pesan. Sistem juga menerapkan mekanisme *cooldown* agar notifikasi yang sama tidak dikirimkan berulang dalam waktu yang terlalu dekat.

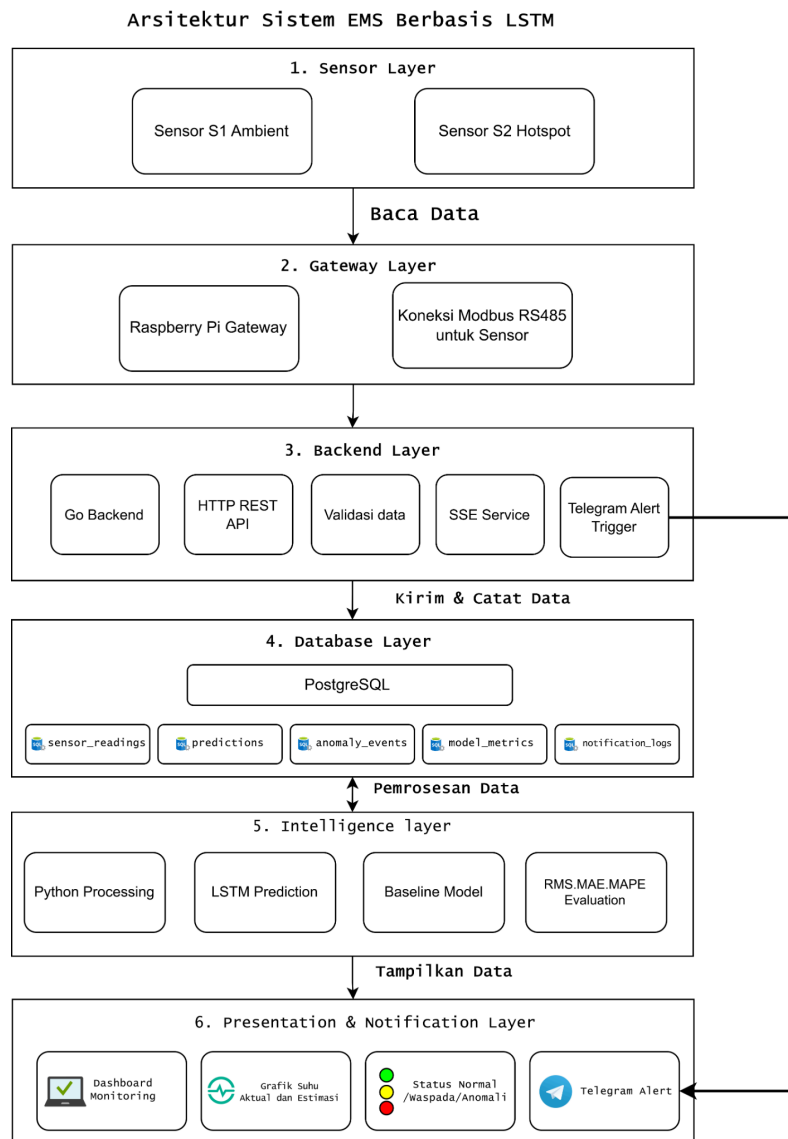
Pesan notifikasi berisi informasi utama seperti status sistem, sensor acuan, nilai estimasi suhu S2, waktu kejadian, dan keterangan kondisi. Riwayat pengiriman notifikasi disimpan ke tabel *notification_logs*, baik notifikasi yang berhasil dikirim, gagal dikirim, maupun dilewati karena aturan *cooldown*.

3.4 Perancangan Sistem/Aplikasi

Perancangan sistem atau aplikasi menjelaskan struktur komponen yang digunakan dalam pembangunan *Early Warning System* pada EMS server. Sistem dirancang secara modular menggunakan pendekatan berlapis agar setiap komponen memiliki peran yang jelas dan dapat diuji secara terpisah maupun terintegrasi.

Secara umum, sistem terdiri dari sensor XY-MD02, Raspberry Pi gateway, backend API, database PostgreSQL, ML Worker LSTM, dashboard monitoring, layout sensor, dan Telegram Bot API. Sensor digunakan untuk membaca suhu dan kelembaban. Raspberry Pi gateway digunakan untuk mengambil data sensor dan mengirimkannya ke backend. Backend berfungsi sebagai pusat pengelolaan data, validasi, status, API, dan notifikasi. Database digunakan untuk menyimpan data historis. ML Worker digunakan untuk melakukan pra-pemrosesan, training, evaluasi, dan inferensi. Dashboard digunakan sebagai antarmuka monitoring, sedangkan Telegram digunakan sebagai media peringatan dini.

Gambar 3.10 Arsitektur Sistem EMS Berbasis LSTM



3.4.1 Sensor Layer

Sensor layer merupakan lapisan yang bertugas mengambil data lingkungan dari server *testbed*. Lapisan ini terdiri dari dua sensor XY-MD02,

yaitu S1 dan S2. Sensor S1 membaca suhu dan kelembaban pada area *ambient* atau referensi lingkungan, sedangkan sensor S2 membaca suhu dan kelembaban pada area *hotspot* atau dekat *exhaust*.

Sensor S1 digunakan untuk merepresentasikan kondisi lingkungan sekitar server *testbed*. Sensor S2 digunakan sebagai titik utama pemantauan karena posisinya lebih dekat dengan sumber panas. Oleh karena itu, suhu S2 digunakan sebagai target estimasi model LSTM.

Sensor layer menghasilkan data suhu dan kelembaban yang menjadi input utama sistem EMS. Data tersebut kemudian diteruskan ke gateway melalui komunikasi Modbus RTU RS485.

3.4.2 Gateway Layer

Gateway layer dijalankan oleh Raspberry Pi. Lapisan ini bertugas membaca data dari sensor, memberikan waktu pembacaan, membentuk payload data, dan mengirimkan data ke backend.

Raspberry Pi membaca data sensor XY-MD02 melalui USB RS485 Converter. Data yang berhasil dibaca kemudian dikirim ke backend menggunakan HTTP REST API melalui endpoint [/api/v1/readings](#). Dengan pembagian ini, Raspberry Pi berfungsi sebagai jembatan antara perangkat sensor dan sistem backend.

Pada rancangan ini, Raspberry Pi difokuskan sebagai perangkat akuisisi dan pengiriman data sensor. Proses penyimpanan data, pengolahan status, dashboard, dan machine learning dijalankan pada komponen lain agar beban kerja gateway tetap ringan.

3.4.3 Backend Layer

Backend layer dikembangkan menggunakan Go. Backend berfungsi sebagai pusat pengelolaan data masuk dari gateway. Backend menerima payload sensor, melakukan validasi, menyimpan data ke database, menyediakan API untuk dashboard, mengirim data melalui Server-Sent Events, mengelola data layout sensor, menerima hasil estimasi dari ML Worker, menentukan status sistem, dan mengirimkan notifikasi Telegram ketika status memenuhi kondisi tertentu.

Backend juga menjadi penghubung antara data sensor yang masuk dengan lapisan database, dashboard, modul LSTM, dan layanan notifikasi. Dengan demikian, backend berperan sebagai pusat integrasi sistem.

Status yang dikelola backend terdiri dari status termal dan status teknis. Status termal meliputi normal, waspada, dan anomali berdasarkan hasil estimasi suhu S2. Status teknis direpresentasikan dengan status *trouble* apabila terdapat gangguan seperti sensor tidak terbaca, gateway tidak aktif, data tidak valid, model belum siap, atau hasil estimasi tidak tersedia.

3.4.4 Database Layer

Database layer digunakan untuk menyimpan data utama yang dihasilkan oleh sistem EMS. Database yang digunakan adalah PostgreSQL. Data yang disimpan meliputi data gateway, data sensor, pembacaan sensor, hasil estimasi suhu S2, status sistem, informasi versi model, metrik evaluasi, hasil baseline, layout sensor, dan riwayat notifikasi.

Tabel *gateways* digunakan untuk menyimpan identitas Raspberry Pi gateway. Tabel *sensors* digunakan untuk menyimpan identitas sensor yang digunakan pada sistem. Tabel *sensor_readings* digunakan untuk menyimpan data suhu dan kelembaban berdasarkan waktu pembacaan. Tabel *model_versions* digunakan untuk menyimpan informasi versi model LSTM yang digunakan, seperti nama model, versi model, algoritma, window input, horizon estimasi, feature set, dan waktu pelatihan model.

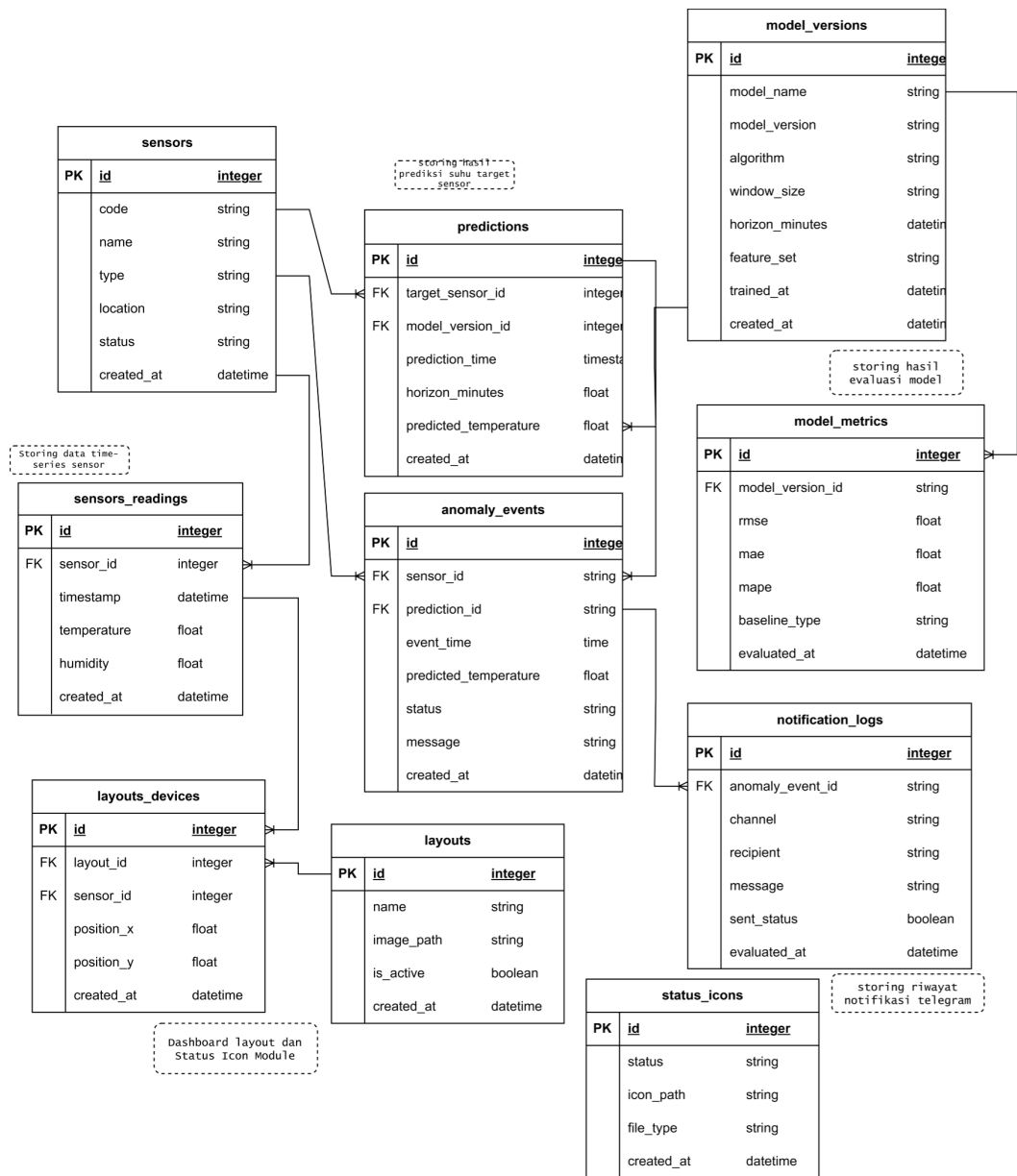
Tabel *predictions* digunakan untuk menyimpan hasil estimasi suhu S2 yang dihasilkan oleh model tertentu. Tabel *anomaly_events* digunakan untuk menyimpan riwayat status sistem, baik status normal, waspada, anomali, maupun *trouble*. Tabel *model_metrics* digunakan untuk menyimpan hasil evaluasi model menggunakan RMSE, MAE, dan MAPE. Tabel *baseline_results* digunakan untuk menyimpan hasil evaluasi baseline seperti *persistence model* dan *moving average*.

Tabel notification_logs digunakan untuk menyimpan riwayat pengiriman notifikasi Telegram. Selain itu, tabel layouts dan layout_devices digunakan untuk mendukung fitur visualisasi layout sensor pada dashboard. Tabel settings digunakan untuk menyimpan konfigurasi sistem, sedangkan tabel system_logs digunakan untuk menyimpan log error atau kejadian penting pada sistem.

Tabel 3.3 Rancangan Tabel Database

Tabel	Fungsi
gateways	Menyimpan identitas Raspberry Pi gateway
sensors	Menyimpan identitas sensor S1 dan S2 beserta perannya
sensor_readings	Menyimpan data suhu dan kelembaban berdasarkan waktu pembacaan
gateway_status_logs	Menyimpan riwayat status gateway dan sensor
model_versions	Menyimpan informasi versi model LSTM
prediction_runs	Menyimpan riwayat proses inferensi model
predictions	Menyimpan hasil estimasi suhu S2
model_metrics	Menyimpan hasil evaluasi RMSE, MAE, dan MAPE
baseline_results	Menyimpan hasil evaluasi baseline
anomaly_events	Menyimpan riwayat status normal, waspada, anomali, dan trouble
notification_logs	Menyimpan riwayat pengiriman notifikasi Telegram
layouts	Menyimpan gambar layout server testbed atau ruangan
layout_devices	Menyimpan posisi sensor pada gambar layout
settings	Menyimpan konfigurasi sistem
system_logs	Menyimpan log error dan kejadian penting

Gambar 3.11 Entity Relationship Diagram Database EMS



3.4.5 Intelligence Layer

Intelligence layer merupakan lapisan yang menjalankan proses machine learning. Lapisan ini dikembangkan menggunakan Python dan berfungsi untuk melakukan pra-pemrosesan data, training model LSTM, evaluasi model, perbandingan baseline, dan inferensi estimasi suhu.

Training model dilakukan menggunakan data historis dari database. Data sensor terlebih dahulu diolah melalui proses pra-pemrosesan, seperti pengurutan data berdasarkan waktu, penggabungan data S1 dan S2, penanganan data kosong, resampling menjadi interval satu menit, normalisasi, dan pembentukan window input.

Model LSTM menggunakan fitur suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Target model adalah suhu S2 lima menit ke depan. Hasil estimasi dari model dikirimkan ke backend dan disimpan ke database agar dapat digunakan oleh dashboard dan modul klasifikasi status.

Informasi versi model disimpan pada tabel model_versions, sedangkan hasil estimasi disimpan pada tabel predictions. Hasil evaluasi model disimpan pada tabel model_metrics, dan hasil pembandingan baseline disimpan pada tabel baseline_results. Dengan rancangan ini, hasil estimasi dan hasil evaluasi dapat ditelusuri berdasarkan model yang digunakan.

3.4.6 Presentation and Notification Layer

Presentation and notification layer terdiri dari dashboard monitoring dan Telegram Bot. Dashboard digunakan untuk menampilkan data sensor, grafik suhu aktual, grafik kelembaban, grafik hasil estimasi, status termal, status teknis, metrik evaluasi model, visualisasi layout sensor, riwayat event, dan riwayat notifikasi.

Dashboard monitoring juga menyediakan fitur visualisasi layout sensor. Fitur ini memungkinkan pengguna menampilkan gambar layout server *testbed* atau ruangan sebagai latar visual, kemudian menempatkan marker sensor pada posisi yang sesuai. Marker sensor berubah berdasarkan status sistem, yaitu normal, waspada, anomali, atau *trouble*. Fitur ini digunakan sebagai pendukung monitoring visual agar pengguna dapat mengetahui posisi sensor yang mengalami kondisi tidak normal secara lebih cepat.

Telegram Bot digunakan untuk mengirimkan peringatan ketika status sistem berada pada kondisi waspada, anomali, atau *trouble*. Notifikasi ini berfungsi sebagai media peringatan dini agar pengguna dapat mengetahui kondisi penting tanpa harus selalu membuka dashboard.

3.4.7 Rancangan API

Backend menyediakan beberapa endpoint untuk mendukung komunikasi antara gateway, dashboard, database, ML Worker, visualisasi layout sensor, dan notifikasi. Endpoint dirancang menggunakan prefix */api/v1*.

Tabel 3.4 Rancangan API

Method	Endpoint	Fungsi
POST	/api/v1/readings	Menerima data sensor dari Raspberry Pi gateway
GET	/api/v1/readings/latest	Mengambil data sensor terbaru

GET	/api/v1/readings/history	Mengambil data historis sensor berdasarkan rentang waktu
POST	/api/v1/gateway/status	Menerima status gateway dari Raspberry Pi
POST	/api/v1/ml/predictions	Menerima hasil estimasi suhu S2 dari ML Worker
GET	/api/v1/predictions/latest	Mengambil hasil estimasi terbaru
GET	/api/v1/model/metrics	Mengambil hasil evaluasi model
GET	/api/v1/events	Mengambil riwayat event sistem
GET	/api/v1/events/stream	Menyediakan koneksi SSE untuk dashboard
GET	/api/v1/layouts/active	Mengambil layout aktif yang ditampilkan pada dashboard
POST	/api/v1/layouts	Menyimpan layout baru
GET	/api/v1/layout-devices	Mengambil posisi sensor pada layout
POST	/api/v1/layout-devices	Menyimpan atau mengubah posisi sensor pada layout
GET	/api/v1/settings	Mengambil konfigurasi sistem
PUT	/api/v1/settings	Mengubah konfigurasi sistem

3.4.8 Rancangan Integrasi LSTM

Integrasi LSTM dilakukan dengan memisahkan proses machine learning dari backend utama. Backend Go bertugas menerima dan menyimpan data sensor, menyediakan API, mengelola status, serta mengirimkan notifikasi. Sementara itu, Python ML Worker bertugas memproses data historis dan menjalankan model LSTM.

Proses integrasi dilakukan dengan cara ML Worker mengambil data historis dari database, menjalankan pra-pemrosesan, melakukan training dan evaluasi model, menjalankan inferensi, kemudian mengirimkan hasil estimasi

suhu S2 ke backend. Backend menyimpan hasil estimasi tersebut ke database dan menggunakannya sebagai dasar klasifikasi status termal.

Dengan pemisahan tersebut, backend tetap fokus pada pengelolaan API, database, dashboard, visualisasi layout, status, dan notifikasi, sedangkan proses machine learning dikelola secara terpisah menggunakan Python.

3.4.9 Rancangan Notifikasi Telegram

Notifikasi Telegram dikirim ketika status sistem berada pada kondisi waspada, anomali, atau *trouble*. Format pesan notifikasi dirancang agar mudah dipahami oleh pengguna. Informasi yang ditampilkan dalam notifikasi meliputi status sistem, sensor acuan, estimasi suhu S2, horizon estimasi, waktu kejadian, dan keterangan kondisi.

Contoh format pesan notifikasi:

```
Peringatan EMS Server Testbed
Status: WASPADA
Sensor Acuan: S2 - Hotspot/Exhaust
Estimasi Suhu: 31.4°C
Horizon Estimasi: 5 menit ke depan
Waktu: 16-05-2026 10:15
Keterangan: Suhu estimasi berada pada rentang waspada
```

Untuk kondisi anomali, status pesan berubah menjadi ANOMALI. Untuk kondisi gangguan teknis, status pesan berubah menjadi TROUBLE. Sistem juga menyimpan riwayat pengiriman pesan ke tabel *notification_logs*.

3.5 Perancangan Pengujian

Perancangan pengujian dilakukan untuk memastikan setiap komponen pada sistem dapat berjalan sesuai kebutuhan. Pengujian dilakukan secara bertahap mulai dari pembacaan sensor, gateway, backend, database, dashboard, visualisasi layout sensor, model LSTM, baseline, klasifikasi status termal, notifikasi Telegram, hingga demo sistem secara terintegrasi.

Pengujian pada penelitian ini tidak hanya berfokus pada model LSTM, tetapi juga mencakup pengujian sistem EMS secara keseluruhan. Hal ini dilakukan karena sistem yang dikembangkan merupakan *Early Warning System* pada EMS server yang melibatkan integrasi perangkat keras, perangkat lunak, basis data, dashboard, model machine learning, dan notifikasi.

3.5.1 Pengujian Pembacaan Sensor

Pengujian pembacaan sensor dilakukan untuk memastikan sensor XY-MD02 S1 dan S2 dapat membaca suhu dan kelembaban melalui komunikasi Modbus RTU RS485. Pengujian dilakukan dengan mencatat jumlah data yang berhasil dibaca, jumlah data yang gagal dibaca, serta kesesuaian waktu pembacaan dan identitas sensor.

Sensor S1 digunakan sebagai sensor *ambient* atau referensi lingkungan, sedangkan sensor S2 digunakan sebagai sensor *hotspot* atau *exhaust*. Kedua sensor harus dapat terbaca oleh Raspberry Pi melalui USB RS485 Converter.

Tabel 3.5 Pengujian Pembacaan Sensor

Komponen yang Diuji	Tujuan Pengujian	Indikator Keberhasilan
Sensor XY-MD02 S1	Memastikan sensor ambient membaca suhu dan kelembaban	Data suhu dan kelembaban S1 terbaca sesuai interval
Sensor XY-MD02 S2	Memastikan sensor hotspot membaca suhu dan kelembaban	Data suhu dan kelembaban S2 terbaca sesuai interval
Modbus RS485	Memastikan komunikasi sensor berjalan	Data dapat dibaca melalui USB RS485 Converter
Timestamp	Memastikan setiap data memiliki waktu pencatatan	Setiap data memiliki timestamp
Sensor ID	Memastikan identitas sensor terbaca benar	Data memiliki sensor_id S1 atau S2

3.5.2 Pengujian Gateway

Pengujian gateway dilakukan untuk memastikan Raspberry Pi dapat membaca data sensor dan mengirimkan data ke backend. Pengujian dilakukan dengan menjalankan program gateway, memeriksa data sensor yang terbaca, serta melihat respons backend terhadap data yang dikirimkan gateway.

Data dikirim dalam format JSON melalui HTTP POST ke endpoint [/api/v1/readings](#). Payload yang dikirim memuat identitas gateway, waktu pembacaan, sumber data, serta daftar pembacaan sensor S1 dan S2.

Tabel 3.6 Pengujian Gateway

Komponen yang Diuji	Tujuan Pengujian	Indikator Keberhasilan
Raspberry Pi Gateway	Mengirim data sensor ke backend	Backend menerima data dengan respons sukses
Payload JSON	Memastikan format data sesuai	Data memiliki gateway, waktu pembacaan, source, dan readings
HTTP POST	Menguji endpoint /api/v1/readings	Data berhasil diterima backend
Token gateway	Memastikan data berasal dari gateway yang valid	Backend menerima data dengan token yang sesuai
Error handling	Memastikan gateway mencatat kegagalan pembacaan atau pengiriman	Error tercatat ketika sensor gagal dibaca atau data gagal dikirim

3.5.3 Pengujian Backend dan Database

Pengujian backend dan database dilakukan untuk memastikan data sensor yang diterima backend dapat divalidasi dan disimpan dengan benar ke database. Backend harus mampu menerima data valid, menolak data tidak valid, dan mencatat kejadian penting pada log sistem.

Pengujian juga dilakukan untuk memastikan data historis dapat diambil kembali berdasarkan rentang waktu tertentu. Data historis tersebut digunakan untuk kebutuhan dashboard, pra-pemrosesan data, training model, evaluasi model, dan inferensi estimasi suhu S2.

Tabel 3.7 Pengujian Backend dan Database

Komponen yang Diuji	Tujuan Pengujian	Indikator Keberhasilan
Backend Go	Melakukan validasi data sensor	Data valid diterima dan data tidak valid ditolak
Database PostgreSQL	Menyimpan data sensor	Data tersimpan pada tabel sensor_readings
Query historis	Mengambil data berdasarkan waktu	Data dapat diambil sesuai rentang timestamp
Model version	Menyimpan informasi versi model	Data versi model tersimpan pada model_versions
Prediction data	Menyimpan hasil estimasi suhu S2	Hasil estimasi tersimpan pada predictions
Layout sensor	Menyimpan data layout dan posisi sensor	Data layout tersimpan pada layouts dan layout_devices
System log	Mencatat error atau kejadian penting	Error dan kejadian penting tersimpan pada system_logs

3.5.4 Pengujian Dashboard Real-Time

Pengujian dashboard dilakukan untuk memastikan data sensor dan status sistem dapat ditampilkan pada dashboard. Dashboard harus mampu menampilkan data suhu dan kelembaban S1 dan S2, grafik historis, hasil estimasi suhu S2, status sistem, dan riwayat event.

Pengujian juga dilakukan untuk memastikan dashboard dapat menerima pembaruan data melalui Server-Sent Events sehingga pengguna dapat melihat perubahan data tanpa melakukan refresh halaman secara manual.

Tabel 3.8 Pengujian Dashboard Real-Time

Komponen yang Diuji	Tujuan Pengujian	Indikator Keberhasilan
Dashboard	Menampilkan data sensor	Suhu dan kelembaban S1 dan S2 tampil pada dashboard
Grafik	Menampilkan data historis	Grafik berubah sesuai data yang masuk
Estimasi suhu	Menampilkan estimasi suhu S2	Grafik aktual dan estimasi suhu S2 tampil
Status sistem	Menampilkan status normal, waspada, anomali, atau trouble	Status tampil sesuai hasil klasifikasi
SSE	Mengirim update real-time	Dashboard menerima update tanpa refresh manual

3.5.5 Pengujian Visualisasi Layout Sensor

Pengujian visualisasi layout sensor dilakukan untuk memastikan dashboard dapat menampilkan gambar layout server *testbed* atau ruangan, menampilkan marker sensor pada posisi yang sesuai, serta mengubah tampilan marker berdasarkan status sistem. Pengujian ini diperlukan karena fitur layout digunakan untuk membantu pengguna mengetahui posisi sensor yang mengalami kondisi tidak normal.

Sensor S1 dan S2 ditampilkan pada layout sesuai posisi pemasangan. Marker sensor berubah berdasarkan status normal, waspada, anomali, atau *trouble*.

Tabel 3.9 Pengujian Visualisasi Layout Sensor

Skenario	Indikator Keberhasilan
Menampilkan layout server testbed	Gambar layout tampil pada dashboard
Menampilkan marker sensor	Marker S1 dan S2 tampil pada posisi yang sesuai
Mengubah status sensor	Marker berubah sesuai status normal, waspada, anomali, atau trouble
Menyorot sensor bermasalah	Dashboard menandai sensor yang berstatus waspada, anomali, atau trouble
Menyimpan posisi sensor	Posisi sensor tersimpan dan dapat ditampilkan kembali

3.5.6 Pengujian Model LSTM

Pengujian model LSTM dilakukan untuk mengetahui kemampuan model dalam mengestimasi suhu S2 lima menit ke depan. Evaluasi dilakukan menggunakan data testing yang tidak digunakan dalam proses training. Data dibagi secara kronologis agar urutan waktu tetap terjaga.

Model dievaluasi menggunakan RMSE, MAE, dan MAPE. Ketiga metrik tersebut digunakan untuk mengetahui besar kesalahan estimasi dari sudut pandang yang berbeda.

Tabel 3.10 Metrik Evaluasi Model LSTM

Metrik	Fungsi
---------------	---------------

RMSE	Mengukur akar rata-rata kuadrat error estimasi
MAE	Mengukur rata-rata error absolut dalam derajat Celsius
MAPE	Mengukur persentase error estimasi

Indikator keberhasilan pengujian ini adalah model LSTM dapat dilatih, menghasilkan estimasi suhu S2, dan memiliki nilai RMSE, MAE, serta MAPE yang dapat dihitung dan dianalisis.

3.5.7 Pengujian Baseline

Pengujian baseline dilakukan untuk membandingkan hasil LSTM dengan metode sederhana. Baseline yang digunakan adalah *persistence model* dan *moving average*. Pengujian ini diperlukan agar performa LSTM tidak dinilai secara tunggal, tetapi dibandingkan dengan metode dasar yang relevan untuk data *time-series*.

Tabel 3.11 Pengujian Baseline

Model	Tujuan
Persistence Model	Menggunakan nilai suhu terakhir sebagai estimasi berikutnya
Moving Average	Menggunakan rata-rata beberapa data terakhir sebagai estimasi
LSTM	Menggunakan pola historis time-series untuk estimasi suhu

Indikator keberhasilan pengujian ini adalah LSTM memiliki nilai error yang lebih rendah dibandingkan baseline, atau minimal menunjukkan performa yang kompetitif dengan penjelasan teknis yang dapat dipertanggungjawabkan.

3.5.8 Pengujian Klasifikasi Status Termal

Pengujian status termal dilakukan untuk memastikan sistem dapat menentukan status normal, waspada, atau anomali berdasarkan hasil estimasi suhu S2. Selain itu, sistem juga diuji untuk memastikan status *trouble* dapat muncul ketika terdapat gangguan teknis pada sensor, gateway, data, atau model.

Tabel 3.12 Pengujian Klasifikasi Status Termal

Skenario	Estimasi Suhu S2	Status yang Diharapkan
Kondisi normal	< 30°C	Normal
Kondisi mendekati batas	30°C–32°C	Waspada
Kondisi melewati batas	> 32°C	Anomali
Gangguan teknis	Data sensor/gateway/model bermasalah	Trouble

Pengujian ini dapat dilakukan menggunakan data live maupun data historis/replay agar skenario waspada dan anomali dapat diuji secara aman.

3.5.9 Pengujian Telegram Alert

Pengujian Telegram alert dilakukan untuk memastikan sistem mengirimkan notifikasi ketika status berubah menjadi waspada, anomali, atau *trouble*. Pengujian dilakukan dengan memicu kondisi status tertentu dan memeriksa apakah pesan Telegram berhasil diterima.

Selain itu, pengujian juga dilakukan terhadap mekanisme *cooldown* agar sistem tidak mengirimkan notifikasi berulang secara berlebihan.

Tabel 3.13 Pengujian Telegram Alert

Skenario	Kondisi	Indikator Keberhasilan
Status normal	Suhu Estimasi < 30°C	Tidak ada alert Telegram
Status waspada	Suhu Estimasi 30°C–32°C	Telegram mengirim pesan waspada
Status anomali	Suhu Estimasi > 32°C	Telegram mengirim pesan anomali
Cooldown alert	Status sama berulang	Sistem tidak mengirim spam notifikasi

3.5.10 Pengujian Demo Sistem

Pengujian demo sistem dilakukan untuk memastikan sistem dapat ditampilkan secara utuh. Demo mencakup live monitoring, visualisasi layout sensor, estimasi suhu S2 menggunakan LSTM, evaluasi model, simulasi kondisi waspada atau anomali, Telegram alert, dan recovery status.

Pengujian demo sistem bertujuan untuk membuktikan bahwa setiap komponen dapat berjalan sebagai satu kesatuan sistem *early warning*.

Tabel 3.14 Pengujian Demo Sistem

Mode Demo	Yang Ditampilkan	Tujuan
Live monitoring	Dashboard suhu dan kelembaban S1 dan S2	Membuktikan EMS berjalan
Visualisasi layout	Layout testbed, marker	Membuktikan posisi sensor

	sensor, dan status sensor	dapat dimonitor secara visual
Estimasi LSTM	Grafik aktual dan estimasi suhu S2	Membuktikan modul LSTM bekerja
Evaluasi model	RMSE, MAE, MAPE, dan baseline	Membuktikan model dievaluasi secara objektif
Simulasi waspada/anomali	CPU stress ringan, sensor S2 didekatkan ke area panas, atau replay data	Membuktikan status waspada/anomali muncul
Telegram alert	Pesan peringatan Telegram	Membuktikan fitur notifikasi berjalan
Recovery	Status kembali normal	Membuktikan sistem mencatat perubahan kondisi

BAB 4

IMPLEMENTASI DAN PENGUJIAN

Bab ini membahas implementasi dan pengujian sistem *Early Warning System* pada Environment Monitoring System (EMS) server menggunakan algoritma Long Short-Term Memory (LSTM). Implementasi dilakukan berdasarkan analisis dan perancangan yang telah dijelaskan pada Bab 3, meliputi implementasi sensor suhu dan kelembaban, Raspberry Pi gateway, backend API, basis data PostgreSQL, dashboard monitoring, ML Worker LSTM, klasifikasi status termal, visualisasi layout sensor, serta notifikasi Telegram.

Sistem yang diimplementasikan bertujuan untuk memantau suhu dan kelembaban pada lingkungan server *testbed*, menyimpan data sensor sebagai data *time-series*, mengestimasi suhu S2 lima menit ke depan menggunakan LSTM, serta menentukan status sistem berdasarkan hasil estimasi dan kondisi teknis komponen. Status yang digunakan meliputi normal, waspada, anomali, dan *trouble*. Status normal, waspada, dan anomali digunakan untuk menunjukkan

kondisi termal, sedangkan status *trouble* digunakan untuk menunjukkan adanya gangguan teknis pada sensor, gateway, data, model, atau komponen pendukung lainnya.

Pengujian dilakukan untuk memastikan setiap komponen sistem dapat berjalan sesuai kebutuhan yang telah dirancang. Pengujian mencakup pembacaan sensor XY-MD02, pengiriman data dari Raspberry Pi gateway ke backend, penyimpanan data ke basis data, tampilan dashboard, visualisasi layout sensor, proses pra-pemrosesan data, training dan evaluasi model LSTM, pengujian baseline, inferensi estimasi suhu S2, klasifikasi status, notifikasi Telegram, serta pengujian integrasi sistem. Hasil pengujian digunakan untuk menilai apakah sistem yang dikembangkan mampu menjalankan fungsi monitoring dan peringatan dini secara terintegrasi.

4.1 Implementasi

Implementasi merupakan tahap penerapan hasil rancangan sistem ke dalam bentuk perangkat keras dan perangkat lunak yang dapat dijalankan. Sistem dikembangkan secara modular agar setiap komponen memiliki tanggung jawab yang jelas dan dapat diuji secara terpisah maupun terintegrasi.

Komponen utama yang diimplementasikan meliputi Raspberry Pi gateway, sensor XY-MD02, backend API, basis data PostgreSQL, dashboard monitoring, ML Worker LSTM, serta Telegram Bot API. Raspberry Pi gateway digunakan untuk membaca data sensor dan mengirimkannya ke backend. Backend API digunakan untuk menerima, memvalidasi, menyimpan, dan

menyediakan data bagi dashboard serta komponen lain. Basis data digunakan untuk menyimpan data sensor, hasil estimasi, metrik evaluasi, event, konfigurasi, dan log. ML Worker digunakan untuk melakukan pra-pemrosesan, training, evaluasi, dan inferensi estimasi suhu S2. Dashboard digunakan sebagai antarmuka monitoring, sedangkan Telegram digunakan sebagai media notifikasi peringatan dini

4.1.1 Lingkungan Implementasi

Implementasi sistem dilakukan pada lingkungan lokal dengan pemisahan peran antara Raspberry Pi dan laptop pengembangan. Raspberry Pi digunakan sebagai gateway sensor yang membaca data suhu dan kelembaban dari sensor XY-MD02 melalui komunikasi Modbus RTU RS485. Laptop pengembangan digunakan untuk menjalankan backend API, basis data PostgreSQL, dashboard monitoring, dan ML Worker LSTM.

Pembagian ini dilakukan agar Raspberry Pi tetap berperan sebagai perangkat akuisisi data yang ringan, sedangkan proses penyimpanan data, visualisasi dashboard, training model, evaluasi, dan inferensi dijalankan pada laptop. Dengan demikian, beban kerja gateway tidak terlalu besar dan proses pengembangan sistem dapat dilakukan secara lebih fleksibel.

Tabel 4.1 Lingkungan Implementasi Sistem

Komponen	Teknologi/Perangkat	Fungsi
Server testbed	Laptop/komputer uji	Objek pengujian yang dipantau kondisi suhu dan kelembabannya
Sensor suhu dan kelembaban	XY-MD02 S1 dan S2	Membaca suhu dan kelembaban pada titik ambient dan hotspot/exhaust
Gateway fisik	Raspberry Pi 3	Membaca sensor melalui Modbus RTU RS485 dan mengirimkan data ke backend
USB RS485 Converter	USB to RS485 Adapter	Menghubungkan sensor XY-MD02 dengan Raspberry Pi
Backend API	Go/Golang	Menerima data sensor, melakukan validasi, menyediakan REST API, SSE, status, dan notifikasi
Database	PostgreSQL	Menyimpan data sensor, hasil estimasi, metrik evaluasi, baseline, event, konfigurasi, dan log
ML Worker	Python, TensorFlow/Keras	Melakukan pra-pemrosesan, training LSTM, evaluasi, baseline, dan inferensi
Dashboard	React, Vite, TypeScript, Tailwind CSS	Menampilkan data sensor, grafik, estimasi suhu S2, status, layout, event, dan evaluasi
Notifikasi	Telegram Bot API	Mengirimkan peringatan ketika status waspada, anomali, atau trouble

Struktur program dibuat secara modular agar setiap komponen dapat dikembangkan dan diuji secara terpisah. Folder *backend-go* digunakan untuk backend API, folder *frontend-dashboard* digunakan untuk dashboard monitoring, folder *gateway-rpi* digunakan untuk gateway fisik pada Raspberry Pi, folder *ml-worker* digunakan untuk proses machine learning berbasis LSTM, sedangkan folder database atau migration digunakan untuk pengelolaan struktur basis data.

Tempat gambar:

Gambar 4.1 Struktur Direktori Sistem EMS

Gambar 4.2 Lingkungan Sistem Saat Dijalankan

4.1.2 Implementasi Arsitektur Sistem

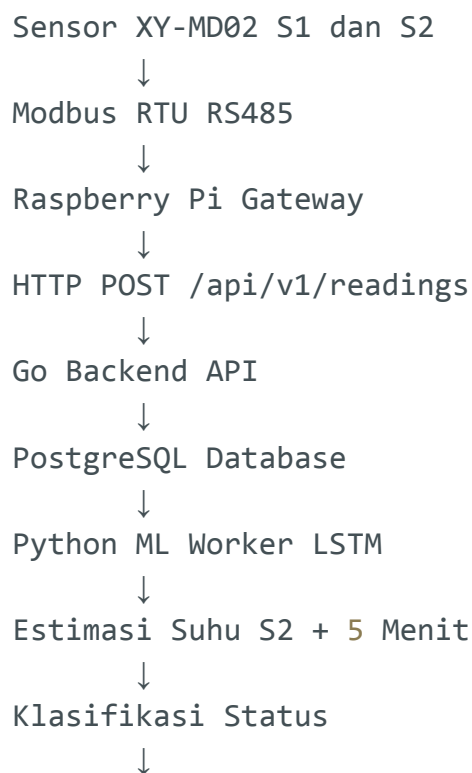
Arsitektur sistem diimplementasikan menggunakan pendekatan berlapis sesuai rancangan pada Bab 3. Lapisan pertama adalah *sensor layer* yang terdiri dari sensor suhu dan kelembaban XY-MD02 S1 dan S2. Lapisan kedua adalah *gateway layer* yang dijalankan oleh Raspberry Pi untuk membaca data sensor dan mengirimkannya ke backend. Lapisan ketiga adalah *backend layer* yang berfungsi sebagai pusat penerimaan data, validasi, penyimpanan, penyedia API, pengelolaan status, dan notifikasi.

Lapisan berikutnya adalah *database layer* yang digunakan untuk menyimpan data sensor, hasil estimasi, metrik evaluasi, baseline, event, pengaturan sistem, dan log. Selanjutnya, *intelligence layer* dijalankan oleh ML Worker yang berfungsi untuk melakukan pra-pemrosesan data, training, evaluasi, dan inferensi estimasi suhu S2. Lapisan terakhir adalah *presentation and notification layer* yang terdiri dari dashboard monitoring dan Telegram Bot API.

Alur kerja sistem dimulai dari sensor XY-MD02 yang membaca suhu dan kelembaban pada lingkungan server *testbed*. Sensor S1 ditempatkan pada area *ambient* atau referensi lingkungan, sedangkan sensor S2 ditempatkan pada area

hotspot atau *exhaust* yang lebih dekat dengan sumber panas. Data sensor dibaca oleh Raspberry Pi gateway melalui komunikasi Modbus RTU RS485. Setelah data berhasil terbaca, gateway membentuk payload data dan mengirimkannya ke backend menggunakan HTTP POST.

Backend menerima payload dari gateway, melakukan validasi, lalu menyimpan data ke basis data PostgreSQL. Data historis yang tersimpan kemudian digunakan oleh ML Worker untuk melakukan pra-pemrosesan, training, evaluasi, dan inferensi model LSTM. Hasil inferensi berupa estimasi suhu S2 lima menit ke depan dikirimkan kembali ke backend untuk disimpan, diklasifikasikan, dan ditampilkan pada dashboard. Apabila status sistem berada pada kondisi waspada, anomali, atau *trouble*, sistem dapat mengirimkan notifikasi melalui Telegram.



Dashboard Monitoring dan Telegram Alert

Dengan pembagian arsitektur tersebut, backend tidak membaca sensor secara langsung. Pembacaan sensor dilakukan oleh Raspberry Pi gateway, sedangkan backend berperan sebagai pusat pengelolaan data dan komunikasi antarkomponen. Pemisahan ini membuat sistem lebih modular karena gateway, backend, database, dashboard, ML Worker, dan Telegram dapat diuji secara bertahap.

Tempat gambar:

Gambar 4.3 Implementasi Fisik Server Testbed

Gambar 4.4 Arsitektur Implementasi Sistem EMS

4.1.3 Implementasi Sensor dan Gateway Raspberry Pi

Implementasi sensor dan gateway dilakukan menggunakan dua sensor XY-MD02 yang terhubung ke Raspberry Pi melalui USB RS485 Converter. Sensor XY-MD02 digunakan untuk membaca suhu dan kelembaban pada dua titik pemantauan. Sensor S1 ditempatkan pada area *ambient* atau referensi lingkungan, sedangkan sensor S2 ditempatkan pada area *hotspot* atau *exhaust server testbed*.

Raspberry Pi berperan sebagai gateway yang membaca data dari sensor, membentuk payload, dan mengirimkan data ke backend. Komunikasi antara sensor dan Raspberry Pi menggunakan Modbus RTU melalui RS485. Setiap sensor dikonfigurasi dengan identitas berbeda agar data S1 dan S2 dapat dibedakan pada sistem.

Implementasi gateway dibuat menggunakan Python. Program gateway memiliki fungsi untuk membaca konfigurasi sensor, membuka koneksi serial, membaca register suhu dan kelembaban, melakukan validasi hasil pembacaan, membentuk payload JSON, mengirim data ke backend, dan mencatat log apabila terjadi kesalahan pembacaan atau pengiriman.

Konfigurasi sensor yang digunakan pada gateway secara umum mencakup kode sensor, peran sensor, alamat slave ID, register suhu, register kelembaban, dan skala pembacaan. Contoh konfigurasi sensor adalah sebagai berikut:

```
sensors:
- code: "S1"
  role: "ambient"
  slave_id: 1
  registers:
    temperature:
      address: 1
      scale: 0.1
    humidity:
      address: 2
      scale: 0.1

- code: "S2"
  role: "hotspot"
  slave_id: 2
  registers:
    temperature:
      address: 1
      scale: 0.1
    humidity:
      address: 2
      scale: 0.1
```

Data yang berhasil dibaca oleh gateway dikirimkan ke backend melalui endpoint [/api/v1/readings](#). Payload yang dikirim berisi identitas gateway, waktu pembacaan, sumber data, dan daftar data sensor. Contoh bentuk payload yang dikirimkan gateway adalah sebagai berikut:

```
{
  "gateway_code": "raspi-gateway-01",
  "recorded_at": "2026-06-03T10:15:00+07:00",
  "source": "hardware",
  "readings": [
    {
      "sensor_code": "S1",
      "sensor_role": "ambient",
      "temperature": 27.4,
      "humidity": 63.2,
      "quality_status": "valid"
    },
    {
      "sensor_code": "S2",
      "sensor_role": "hotspot",
      "temperature": 31.2,
      "humidity": 58.4,
      "quality_status": "valid"
    }
  ]
}
```

Selain mengirimkan data pembacaan sensor, gateway juga mengirimkan status gateway ke backend. Status ini digunakan untuk mengetahui apakah gateway masih aktif dan apakah sensor berhasil terbaca. Apabila sensor tidak terbaca atau gateway tidak mengirimkan data dalam waktu tertentu, sistem dapat menampilkan status *trouble*.

Tabel 4.2 Implementasi Gateway Raspberry Pi

Komponen	Implementasi	Keterangan
Bahasa program	Python	Digunakan untuk program gateway
Komunikasi sensor	Modbus RTU RS485	Digunakan untuk membaca XY-MD02
Adapter	USB RS485 Converter	Menghubungkan sensor dengan Raspberry Pi
Sensor S1	XY-MD02 ambient	Membaca suhu dan kelembaban area referensi
Sensor S2	XY-MD02 hotspot/exhaust	Membaca suhu dan kelembaban area utama pemantauan
Pengiriman data	HTTP POST	Mengirim data ke backend
Endpoint data	/api/v1/readings	Endpoint penerimaan data sensor
Endpoint status	/api/v1/gateway/status	Endpoint penerimaan status gateway
Sumber data	hardware	Menandai data berasal dari sensor fisik
Kualitas data	valid, invalid, atau timeout	Menandai kualitas hasil pembacaan

Tempat gambar:

Gambar 4.5 Implementasi Sensor S1 dan S2 pada Server Testbed

Gambar 4.6 Implementasi Raspberry Pi dan USB RS485 Converter

Gambar 4.7 Output Gateway Saat Membaca dan Mengirim Data Sensor

4.1.4 Implementasi Basis Data

Basis data diimplementasikan menggunakan PostgreSQL. Basis data digunakan untuk menyimpan data gateway, sensor, pembacaan sensor, hasil

estimasi suhu S2, versi model, metrik evaluasi, hasil baseline, event status, riwayat notifikasi, layout sensor, pengaturan sistem, dan log.

Data pembacaan sensor disimpan sebagai data *time-series* karena setiap data memiliki waktu pembacaan. Data tersebut digunakan untuk kebutuhan monitoring dashboard, grafik historis, pra-pemrosesan data, training model LSTM, evaluasi model, inferensi, dan pembuktian hasil pengujian.

Struktur basis data dibuat agar setiap data dapat ditelusuri. Data pembacaan sensor dapat ditelusuri ke sensor dan gateway sumber. Hasil estimasi dapat ditelusuri ke versi model yang digunakan. Event status dan notifikasi juga disimpan agar proses *early warning* dapat diperiksa kembali pada tahap pengujian.

Tabel 4.3 Implementasi Tabel Database

Tabel	Fungsi
gateways	Menyimpan identitas Raspberry Pi gateway
api_tokens	Menyimpan token autentikasi gateway dan internal service
sensors	Menyimpan identitas sensor S1 dan S2
sensor_readings	Menyimpan data suhu dan kelembaban berdasarkan waktu pembacaan
gateway_status_logs	Menyimpan riwayat status gateway dan sensor
model_versions	Menyimpan informasi versi model LSTM
prediction_runs	Menyimpan riwayat proses training atau inferensi
predictions	Menyimpan hasil estimasi suhu S2
model_metrics	Menyimpan hasil evaluasi model menggunakan RMSE, MAE, dan MAPE
baseline_results	Menyimpan hasil evaluasi baseline
anomaly_events	Menyimpan riwayat status normal, waspada, anomali, dan trouble

notification_logs	Menyimpan riwayat pengiriman notifikasi Telegram
layouts	Menyimpan data layout server testbed atau ruangan
layout_devices	Menyimpan posisi sensor pada layout
settings	Menyimpan konfigurasi threshold, Telegram, gateway, dan ML
system_logs	Menyimpan log error dan kejadian penting pada sistem

Tabel *sensor_readings* menjadi tabel utama untuk menyimpan data hasil pembacaan sensor. Setiap data pada tabel ini memiliki relasi terhadap gateway dan sensor. Field penting pada tabel ini meliputi *gateway_id*, *sensor_id*, *recorded_at*, *temperature*, *humidity*, *source*, dan *quality_status*.

Tabel *predictions* digunakan untuk menyimpan hasil estimasi suhu S2 yang dihasilkan oleh model LSTM. Data pada tabel ini mencakup nilai estimasi suhu, waktu target estimasi, status termal, status akhir, dan informasi model yang digunakan. Dengan adanya tabel ini, hasil estimasi dapat ditampilkan pada dashboard dan dibandingkan dengan data aktual untuk kebutuhan evaluasi.

Tabel *model_metrics* digunakan untuk menyimpan nilai RMSE, MAE, dan MAPE dari model LSTM. Tabel *baseline_results* digunakan untuk menyimpan hasil evaluasi baseline seperti *persistence model* dan *moving average*. Kedua tabel ini digunakan untuk membandingkan performa LSTM dengan metode sederhana.

Tabel *anomaly_events* digunakan untuk menyimpan riwayat kejadian status sistem. Walaupun nama tabel menggunakan istilah anomaly, tabel ini tidak hanya menyimpan status anomali, tetapi juga status normal, waspada, dan *trouble*. Tabel *notification_logs* digunakan untuk menyimpan riwayat pengiriman notifikasi Telegram, baik yang berhasil dikirim, gagal dikirim, maupun dilewati karena aturan *cooldown*.

Tempat gambar:

Gambar 4.8 Hasil Migration Database

Gambar 4.9 Isi Tabel *sensor_readings* dari Data Hardware

Gambar 4.10 Isi Tabel *predictions*, *model_metrics*, dan *baseline_results*

Tempat gambar:

Gambar 4.5 Migration dan Seed Database Berhasil Dijalankan

Gambar 4.6 Data Sensor Tersimpan pada Tabel *sensor_readings*

4.1.5 Implementasi Backend Go API

Backend API diimplementasikan menggunakan bahasa pemrograman Go. Backend berfungsi sebagai pusat integrasi sistem yang menghubungkan Raspberry Pi gateway, basis data PostgreSQL, dashboard monitoring, ML Worker, Server-Sent Events, dan Telegram Bot API.

Fungsi utama backend adalah menerima data sensor dari gateway, melakukan validasi data, menyimpan data ke basis data, menyediakan endpoint untuk dashboard, menerima hasil estimasi suhu S2 dari ML Worker, menentukan status sistem, mengirim pembaruan data secara *real-time* melalui Server-Sent Events, serta mencatat event dan notifikasi.

Pada proses penerimaan data sensor, Raspberry Pi gateway mengirimkan payload ke endpoint [/api/v1/readings](#). Backend kemudian melakukan pengecekan token, validasi format payload, validasi identitas gateway, validasi sensor, validasi nilai suhu, validasi nilai kelembaban, validasi waktu pembacaan, sumber data, dan status kualitas data. Data yang valid disimpan ke basis data, sedangkan data yang tidak valid ditolak atau dicatat sebagai log sistem.

Backend juga menerima hasil estimasi dari ML Worker melalui endpoint internal. Hasil estimasi tersebut tidak langsung digunakan oleh dashboard tanpa proses tambahan, tetapi terlebih dahulu disimpan dan diklasifikasikan. Backend menentukan status termal berdasarkan nilai estimasi suhu S2 dan ambang batas operasional. Selain itu, backend juga memeriksa status teknis seperti kondisi sensor, gateway, data, dan model untuk menentukan status akhir sistem.

Tabel 4.3 Implementasi Endpoint Backend

Endpoint	Method	Fungsi
/api/v1/health	GET	Mengecek status backend dan koneksi

		database
/api/v1/readings	POST	Menerima data suhu dan kelembaban dari Raspberry Pi gateway
/api/v1/readings/latest	GET	Mengambil data sensor terbaru
/api/v1/readings/history	GET	Mengambil data historis sensor berdasarkan rentang waktu
/api/v1/gateway/status	POST	Menerima status gateway dan sensor
/api/v1/dashboard/summary	GET	Mengambil ringkasan data untuk dashboard utama
/api/v1/ml/predictions	POST	Menerima hasil estimasi suhu S2 dari ML Worker
/api/v1/predictions/latest	GET	Mengambil hasil estimasi suhu S2 terbaru
/api/v1/predictions/history	GET	Mengambil riwayat hasil estimasi
/api/v1/model-metrics/latest	GET	Mengambil metrik evaluasi model terbaru
/api/v1/model-comparison/latest	GET	Mengambil perbandingan LSTM dengan baseline
/api/v1/anomaly-events	GET	Mengambil riwayat event status sistem
/api/v1/notification-logs	GET	Mengambil riwayat notifikasi Telegram
/api/v1/events	GET	Mengirim pembaruan data melalui Server-Sent Events
/api/v1/layout	GET	Mengambil layout aktif dan posisi marker sensor
/api/v1/settings	GET	Mengambil konfigurasi sistem

Backend menerapkan keamanan dasar menggunakan *bearer token* pada endpoint yang menerima data dari gateway dan ML Worker. Mekanisme ini digunakan agar backend hanya menerima data dari sumber yang valid. Endpoint gateway dilindungi menggunakan token gateway, sedangkan endpoint ML Worker dilindungi menggunakan token internal.

Backend juga menerapkan format respons yang konsisten untuk respons sukses maupun respons error. Hal ini memudahkan dashboard dan ML Worker dalam membaca hasil komunikasi dengan backend. Jika terjadi kegagalan seperti data tidak valid, token salah, database tidak tersedia, atau Telegram gagal

mengirim pesan, backend mencatat kondisi tersebut pada log tanpa menghentikan keseluruhan sistem.

Tempat gambar:

Gambar 4.11 Backend Go API Berjalan

Gambar 4.12 Pengujian Endpoint Health Check

Gambar 4.13 Pengujian Endpoint POST Readings

Gambar 4.14 Data Sensor Berhasil Diterima Backend

4.1.6 Implementasi Dashboard Monitoring

Dashboard monitoring diimplementasikan menggunakan React, Vite, TypeScript, Tailwind CSS, shadcn/ui, dan Chart.js. Dashboard digunakan sebagai antarmuka utama bagi pengguna untuk memantau kondisi lingkungan server *testbed*.

Dashboard mengambil data dari backend API dan menampilkan informasi dalam bentuk kartu, grafik, tabel, status, dan visualisasi layout sensor. Data yang ditampilkan mencakup suhu dan kelembaban sensor S1, suhu dan kelembaban sensor S2, grafik historis suhu dan kelembaban, hasil estimasi suhu S2, status sistem, metrik evaluasi model, riwayat event, riwayat notifikasi, serta posisi sensor pada layout.

Dashboard juga menerima pembaruan data melalui Server-Sent Events. Dengan mekanisme ini, dashboard dapat memperbarui data ketika terdapat pembacaan sensor baru, hasil estimasi baru, perubahan status, atau event baru tanpa perlu melakukan refresh halaman secara manual.

Menu utama pada dashboard terdiri dari enam bagian, yaitu Dashboard, Sensors & Readings, Prediction & LSTM, Layout, Events & Logs, dan Settings. Struktur ini digunakan agar fitur sistem tetap ringkas dan tidak terlalu banyak memecah menu.

Tabel 4.5 Implementasi Halaman Dashboard

Menu	Fungsi
Dashboard	Menampilkan ringkasan kondisi sistem, status sensor, status gateway, status model, estimasi suhu S2, dan recent event
Sensors & Readings	Menampilkan data sensor terbaru, riwayat pembacaan, tabel data, dan grafik suhu serta kelembaban
Prediction & LSTM	Menampilkan hasil estimasi suhu S2, grafik aktual dan estimasi, metrik model, serta perbandingan baseline
Layout	Menampilkan gambar layout server testbed dan posisi marker sensor S1 dan S2
Events & Logs	Menampilkan riwayat status sistem, notifikasi Telegram, dan log sistem
Settings	Menampilkan dan mengatur konfigurasi sistem seperti threshold, interval, Telegram, dan status fitur

Pada halaman Dashboard, sistem menampilkan ringkasan kondisi terkini. Ringkasan tersebut mencakup nilai suhu dan kelembaban S1, nilai suhu dan kelembaban S2, estimasi suhu S2 lima menit ke depan, status termal, status gateway, status sensor, status model, dan status Telegram. Status ditampilkan agar pengguna dapat mengetahui kondisi sistem secara cepat.

Pada halaman Sensors & Readings, dashboard menampilkan data pembacaan sensor secara historis. Data ini ditampilkan dalam bentuk grafik dan tabel. Grafik suhu dan kelembaban digunakan untuk melihat pola perubahan kondisi lingkungan dari waktu ke waktu.

Pada halaman Prediction & LSTM, dashboard menampilkan informasi terkait model LSTM. Informasi tersebut mencakup model aktif, hasil estimasi suhu S2, grafik aktual dibandingkan estimasi, metrik RMSE, MAE, MAPE, serta hasil perbandingan dengan baseline. Halaman ini digunakan untuk menunjukkan bahwa estimasi suhu tidak hanya ditampilkan, tetapi juga dievaluasi.

Pada halaman Layout, dashboard menampilkan gambar layout server *testbed* atau ruangan. Sensor S1 dan S2 ditampilkan dalam bentuk marker pada posisi yang sesuai. Marker dapat menunjukkan status normal, waspada, anomali, atau *trouble* sesuai data terbaru dari backend.

Pada halaman Events & Logs, dashboard menampilkan riwayat event status, notifikasi Telegram, dan log sistem. Halaman ini digunakan untuk menelusuri kejadian penting seperti status waspada, anomali, *trouble*, notifikasi berhasil dikirim, notifikasi gagal dikirim, atau error pada komponen sistem.

Tempat gambar:

Gambar 4.15 Halaman Dashboard Utama

Gambar 4.16 Halaman Sensors & Readings

Gambar 4.17 Halaman Prediction & LSTM

Gambar 4.18 Halaman Layout Sensor

Gambar 4.19 Halaman Events & Logs

Gambar 4.20 Halaman Settings

4.1.7 Implementasi ML Worker LSTM

ML Worker diimplementasikan menggunakan Python. Modul ini bertugas menjalankan proses yang berkaitan dengan pengolahan data *time-series* dan model LSTM. ML Worker dibuat terpisah dari backend agar proses training, evaluasi, dan inferensi tidak membebani service backend.

Fungsi utama ML Worker meliputi pengambilan data sensor dari PostgreSQL, penggabungan data S1 dan S2 berdasarkan waktu, pra-pemrosesan data, *resampling* menjadi interval satu menit, pembentukan *window* data, training model LSTM, evaluasi model, perbandingan baseline, penyimpanan model, dan inferensi estimasi suhu S2 lima menit ke depan.

Data input model berasal dari suhu dan kelembaban sensor S1 dan S2. Sensor S1 berperan sebagai fitur referensi lingkungan, sedangkan sensor S2 berperan sebagai sensor utama. Target estimasi model adalah suhu S2 lima menit ke depan. Dengan demikian, model tidak diarahkan untuk mengestimasi seluruh sensor secara terpisah, tetapi difokuskan pada suhu S2 sebagai titik utama pemantauan termal.

Tabel 4.6 Implementasi Model LSTM

Komponen	Implementasi
Bahasa program	Python
Library utama	TensorFlow/Keras, Pandas, NumPy, Scikit-learn
Sumber data	Tabel <code>sensor_readings</code> pada PostgreSQL
Fitur input	Suhu S1, kelembaban S1, suhu S2, kelembaban S2
Target estimasi	Suhu S2 lima menit ke depan

Interval data ML	1 menit setelah proses resampling
Window historis	30 data hasil resampling
Durasi window	30 menit data historis
Model utama	Long Short-Term Memory
Baseline	Persistence model dan moving average
Evaluasi	RMSE, MAE, MAPE
Output	Estimasi suhu S2 dan metadata inferensi

Tahapan kerja ML Worker dimulai dari pengambilan data historis dari basis data. Data yang diambil adalah data dengan status kualitas valid dan sumber data yang sesuai. Setelah itu, data S1 dan S2 digabungkan berdasarkan waktu agar setiap baris data memiliki fitur suhu S1, kelembaban S1, suhu S2, dan kelembaban S2.

Data kemudian di-*resampling* menjadi interval satu menit. Proses ini dilakukan karena data mentah dari gateway dapat masuk dalam interval yang lebih pendek, sedangkan model LSTM menggunakan data dengan interval yang lebih seragam. Setelah itu, data dinormalisasi agar fitur berada pada skala yang sesuai sebelum digunakan oleh model.

Pembentukan *window* dilakukan dengan menggunakan 30 data historis sebagai input. Target model adalah suhu S2 lima menit setelah waktu input terakhir. Dataset kemudian dibagi secara kronologis menjadi data training, validasi, dan testing. Pembagian kronologis digunakan agar urutan waktu tetap terjaga dan tidak terjadi kebocoran data dari masa depan ke masa lalu.

Setelah model dilatih, ML Worker menghitung metrik evaluasi berupa RMSE, MAE, dan MAPE. Selain itu, ML Worker juga menghitung performa

baseline menggunakan *persistence model* dan *moving average*. Perbandingan ini digunakan agar performa LSTM dapat dianalisis secara lebih objektif.

Hasil training menghasilkan model artifact, scaler, metadata, dan metrik evaluasi. Pada proses inferensi, ML Worker mengambil data terbaru dari basis data, melakukan pra-pemrosesan dengan tahapan yang sama, membentuk *window* 30 data terakhir, lalu menghasilkan estimasi suhu S2 lima menit ke depan. Hasil inferensi kemudian dikirimkan ke backend melalui endpoint [/api/v1/ml/predictions](#) untuk disimpan dan diklasifikasikan.

Tempat gambar:

Gambar 4.21 Output Training ML Worker

Gambar 4.22 Artifact Model LSTM

Gambar 4.23 Output Evaluasi RMSE, MAE, dan MAPE

Gambar 4.24 Output Inferensi Estimasi Suhu S2

4.1.8 Implementasi Klasifikasi Status dan Notifikasi Telegram

Klasifikasi status diimplementasikan pada backend setelah backend menerima hasil estimasi suhu S2 dari ML Worker. Backend membandingkan hasil estimasi suhu S2 dengan ambang batas operasional yang telah ditentukan.

Ambang batas yang digunakan adalah normal untuk estimasi suhu S2 di bawah 30°C, waspada untuk estimasi suhu S2 pada rentang 30°C sampai 32°C, dan anomali untuk estimasi suhu S2 di atas 32°C.

Selain status termal, backend juga memeriksa status teknis sistem. Status teknis digunakan untuk mendeteksi kondisi seperti sensor tidak terbaca, gateway tidak mengirim data, data tidak valid, model belum siap, atau hasil estimasi sudah tidak layak digunakan sebagai status aktif. Jika terdapat gangguan teknis, sistem memberikan status akhir *trouble*. Dengan demikian, sistem dapat membedakan kondisi termal dan kondisi teknis.

Tabel 4.7 Implementasi Status Sistem

Status	Jenis Status	Kondisi
Normal	Termal	Estimasi suhu S2 < 30°C
Waspada	Termal	Estimasi suhu S2 berada pada rentang 30°C sampai 32°C
Anomali	Termal	Estimasi suhu S2 > 32°C
Trouble	Teknis	Sensor, gateway, data, model, atau komponen pendukung mengalami gangguan

Status akhir sistem ditentukan berdasarkan prioritas. Apabila terdapat status *trouble*, maka status akhir menjadi *trouble* meskipun estimasi suhu terakhir masih berada pada rentang normal. Apabila tidak terdapat gangguan teknis, status akhir mengikuti status termal berdasarkan hasil estimasi suhu S2. Model status final ini memang dipisahkan menjadi status kesehatan sensor, status termal, dan status akhir dengan prioritas *trouble* > *anomali* > *waspada* > *normal*.

Notifikasi Telegram diimplementasikan menggunakan Telegram Bot API. Notifikasi dikirimkan ketika sistem mendeteksi status waspada, anomali, atau

trouble. Pesan notifikasi berisi informasi status, sensor acuan, estimasi suhu S2, horizon estimasi, waktu kejadian, dan keterangan kondisi.

Contoh isi pesan notifikasi Telegram adalah sebagai berikut:

Peringatan EMS Server Testbed

Status: WASPADA

Sensor Acuan: S2 - Hotspot/Exhaust

Estimasi Suhu S2: 31.4°C

Horizon Estimasi: 5 menit ke depan

Waktu: 16-05-2026 10:15

Keterangan: Suhu estimasi berada pada rentang waspada.

Sistem juga menerapkan mekanisme *cooldown* agar notifikasi yang sama tidak dikirimkan berulang dalam waktu singkat. Setiap pengiriman notifikasi disimpan pada tabel *notification_logs*. Riwayat tersebut mencakup notifikasi yang berhasil dikirim, gagal dikirim, maupun dilewati karena aturan *cooldown*. Dengan pencatatan ini, proses pengiriman notifikasi dapat ditelusuri kembali pada tahap pengujian.

Tempat gambar:

Gambar 4.25 Event Status Waspada atau Anomali pada Dashboard

Gambar 4.26 Riwayat Notification Logs

Gambar 4.27 Pesan Telegram Berhasil Diterima

Gambar 4.28 Status Trouble pada Dashboard

4.1.8 Implementasi Integrasi Sistem

Integrasi sistem dilakukan dengan menghubungkan seluruh komponen yang telah diimplementasikan, yaitu sensor XY-MD02, Raspberry Pi gateway, backend API, basis data PostgreSQL, dashboard monitoring, ML Worker LSTM, dan Telegram Bot API. Integrasi ini bertujuan agar sistem dapat menjalankan alur kerja secara utuh mulai dari pembacaan data sensor hingga pengiriman peringatan dini.

Pada tahap integrasi, Raspberry Pi gateway membaca data suhu dan kelembaban dari sensor S1 dan S2. Data tersebut kemudian dikirimkan ke backend melalui endpoint [/api/v1/readings](#). Backend melakukan validasi terhadap data yang diterima, menyimpan data valid ke basis data, serta mengirim pembaruan data ke dashboard melalui API dan Server-Sent Events.

Data historis yang tersimpan pada basis data digunakan oleh ML Worker untuk proses pra-pemrosesan, training, evaluasi, dan inferensi. Hasil inferensi berupa estimasi suhu S2 lima menit ke depan dikirimkan kembali ke backend melalui endpoint [/api/v1/ml/predictions](#). Backend kemudian menyimpan hasil estimasi, menentukan status termal, membentuk status akhir sistem, mencatat event, dan memicu notifikasi Telegram apabila status memenuhi kondisi peringatan.

Dengan integrasi tersebut, sistem dapat menampilkan kondisi aktual sensor, hasil estimasi suhu S2, status sistem, grafik historis, posisi sensor pada layout, riwayat event, log sistem, dan riwayat notifikasi. Implementasi integrasi ini menunjukkan bahwa setiap komponen tidak hanya berjalan secara terpisah, tetapi juga dapat bekerja sebagai satu kesatuan sistem *early warning*.

Tabel 4.8 Implementasi Integrasi Komponen Sistem

Komponen	Terhubung Dengan	Fungsi Integrasi
Sensor XY-MD02	Raspberry Pi gateway	Mengirim data suhu dan kelembaban melalui Modbus RTU RS485
Raspberry Pi gateway	Backend API	Mengirim data sensor dan status gateway
Backend API	PostgreSQL	Menyimpan data sensor, estimasi, event, konfigurasi, dan log
Backend API	Dashboard	Menyediakan data monitoring melalui REST API dan SSE
PostgreSQL	ML Worker	Menyediakan data historis untuk pra-pemrosesan dan training
ML Worker	Backend API	Mengirim hasil estimasi suhu S2
Backend API	Telegram Bot API	Mengirim notifikasi status waspada, anomali, atau trouble
Dashboard	Pengguna	Menampilkan hasil monitoring dan early warning

Tempat gambar:

Gambar 4.29 Integrasi Sistem Saat Berjalan

Gambar 4.30 Alur Data Sensor sampai Notifikasi Telegram

4.2 Pengujian

Pengujian dilakukan untuk memastikan sistem yang telah diimplementasikan dapat berjalan sesuai dengan rancangan pada Bab 3. Pengujian dilakukan secara bertahap mulai dari pembacaan sensor, gateway, backend, database, dashboard, visualisasi layout, model LSTM, baseline, klasifikasi status, Telegram alert, hingga pengujian demo sistem secara terintegrasi.

Pengujian pada penelitian ini tidak hanya menilai model LSTM, tetapi juga menilai keberhasilan integrasi sistem EMS sebagai *early warning system*. Oleh karena itu, hasil pengujian mencakup bukti pembacaan data sensor fisik, pengiriman data ke backend, penyimpanan data ke database, tampilan dashboard, hasil estimasi suhu S2, status sistem, dan notifikasi Telegram.

4.2.1 Skenario Pengujian

Skenario pengujian disusun berdasarkan kebutuhan fungsional dan rancangan pengujian yang telah dijelaskan pada Bab 3. Setiap skenario digunakan untuk memastikan bahwa komponen sistem dapat berjalan sesuai fungsi yang dirancang. Pengujian dilakukan secara bertahap agar setiap komponen dapat diuji secara terpisah sebelum diuji sebagai satu kesatuan sistem.

Tabel 4.9 Skenario Pengujian Sistem

No	Skenario Pengujian	Tujuan Pengujian	Bukti Pengujian
1	Pengujian pembacaan sensor	Memastikan sensor XY-MD02 S1 dan S2 dapat membaca suhu dan kelembaban	Output terminal gateway dan data sensor
2	Pengujian gateway	Memastikan Raspberry Pi dapat mengirim data sensor ke backend	Log gateway dan respons API
3	Pengujian backend dan database	Memastikan backend menerima, memvalidasi, dan menyimpan data sensor	Respons API dan isi tabel database
4	Pengujian dashboard real-time	Memastikan dashboard menampilkan data sensor, grafik, estimasi, dan status	Screenshot dashboard
5	Pengujian visualisasi layout sensor	Memastikan posisi sensor S1 dan S2 tampil pada layout	Screenshot halaman layout
6	Pengujian model LSTM	Memastikan model dapat dilatih dan menghasilkan metrik evaluasi	Output training dan metrik RMSE, MAE, MAPE
7	Pengujian baseline	Membandingkan hasil LSTM dengan persistence model dan moving average	Tabel perbandingan metrik
8	Pengujian klasifikasi status termal	Memastikan status normal, waspada, anomali, dan trouble dapat terbentuk	Data event dan tampilan status
9	Pengujian Telegram alert	Memastikan notifikasi dikirim saat status waspada, anomali, atau trouble	Screenshot Telegram dan notification log
10	Pengujian demo sistem	Memastikan seluruh komponen berjalan sebagai sistem terintegrasi	Screenshot, log, API response, dan database evidence

Skenario tersebut digunakan sebagai dasar dalam penyajian hasil pengujian pada subbab berikutnya. Setiap hasil pengujian disajikan dalam bentuk tabel yang memuat komponen yang diuji, hasil yang diharapkan, hasil pengujian, dan status pengujian.

4.2.2 Pengujian Pembacaan Sensor

Pengujian pembacaan sensor dilakukan untuk memastikan sensor XY-MD02 S1 dan S2 dapat membaca suhu dan kelembaban melalui komunikasi Modbus RTU RS485. Pengujian dilakukan dengan menjalankan program gateway pada Raspberry Pi, kemudian memeriksa apakah nilai suhu dan kelembaban dari masing-masing sensor berhasil terbaca.

Sensor S1 digunakan sebagai sensor *ambient* atau referensi lingkungan, sedangkan sensor S2 digunakan sebagai sensor *hotspot* atau *exhaust*. Kedua sensor harus dapat terbaca dengan identitas yang benar agar data dapat disimpan dan digunakan pada tahap berikutnya.

Tabel 4.10 Hasil Pengujian Pembacaan Sensor

Komponen yang Diuji	Hasil yang Diharapkan	Hasil Pengujian	Status
Sensor XY-MD02 S1	S1 membaca suhu dan kelembaban	Data suhu dan kelembaban S1 berhasil terbaca	Berhasil
Sensor XY-MD02 S2	S2 membaca suhu dan kelembaban	Data suhu dan kelembaban S2 berhasil terbaca	Berhasil
Modbus RTU RS485	Sensor dapat dibaca melalui USB RS485 Converter	Raspberry Pi dapat membaca data sensor melalui port serial	Berhasil
Waktu pembacaan	Setiap data memiliki waktu pembacaan	Data memiliki waktu pembacaan pada setiap proses pembacaan	Berhasil
Identitas sensor	Data memiliki identitas S1 atau S2	Data dapat dibedakan berdasarkan sensor S1 dan S2	Berhasil

Berdasarkan hasil pengujian, sensor S1 dan S2 dapat membaca suhu serta kelembaban. Data yang terbaca memiliki identitas sensor dan waktu pembacaan sehingga dapat digunakan sebagai data *time-series* pada sistem EMS. Hasil ini menunjukkan bahwa jalur pembacaan sensor melalui Raspberry Pi dan USB RS485 Converter telah berjalan sesuai rancangan.

Tempat gambar:

Gambar 4.31 Output Pembacaan Sensor S1 dan S2 pada Raspberry Pi

Gambar 4.32 Posisi Sensor S1 dan S2 pada Server Testbed

4.2.3 Hasil Pengujian Gateway

Pengujian gateway dilakukan untuk memastikan Raspberry Pi dapat mengirimkan data sensor ke backend. Pengujian dilakukan dengan menjalankan program gateway, kemudian memeriksa respons dari backend setelah data dikirim melalui HTTP POST ke endpoint [/api/v1/readings](#).

Payload yang dikirimkan gateway berisi identitas gateway, waktu pembacaan, sumber data, serta data pembacaan sensor S1 dan S2. Backend akan menerima data apabila token dan struktur payload sesuai. Apabila data tidak sesuai format, backend akan mengembalikan respons error dan mencatat log.

Tabel 4.11 Hasil Pengujian Gateway

Komponen yang Diuji	Hasil yang Diharapkan	Hasil Pengujian	Status
Raspberry Pi gateway	Gateway dapat mengirim data ke backend	Backend menerima data dari Raspberry Pi	Berhasil

Payload JSON	Payload memiliki struktur yang sesuai	Payload berisi identitas gateway, waktu pembacaan, source, dan readings	Berhasil
HTTP POST	Data dikirim ke endpoint /api/v1/readings	Backend memberikan respons sukses terhadap data valid	Berhasil
Token gateway	Data diterima jika token valid	Backend menerima data dari gateway dengan token yang sesuai	Berhasil
Error handling	Error dicatat ketika pembacaan atau pengiriman gagal	Kesalahan dapat dicatat pada log gateway atau backend	Berhasil

Hasil pengujian menunjukkan bahwa Raspberry Pi gateway dapat mengirimkan data sensor ke backend. Backend menerima payload dari gateway, melakukan validasi awal, dan memberikan respons sesuai hasil validasi. Dengan demikian, proses komunikasi antara gateway dan backend telah berjalan sesuai rancangan.

Tempat gambar:

Gambar 4.33 Gateway Mengirim Data ke Backend

Gambar 4.34 Respons Backend terhadap Data Gateway

4.2.4 Pengujian Backend dan Database

Pengujian backend dan database dilakukan untuk memastikan backend dapat menerima data dari gateway, melakukan validasi, dan menyimpan data ke PostgreSQL. Pengujian juga dilakukan untuk memastikan data yang sudah tersimpan dapat diambil kembali melalui query historis maupun endpoint API.

Data yang diuji meliputi data suhu, kelembaban, identitas gateway, identitas sensor, waktu pembacaan, sumber data, dan status kualitas data. Data yang valid harus tersimpan pada tabel *sensor_readings*.

Tabel 4.12 Hasil Pengujian Backend dan Database

Komponen yang Diuji	Hasil yang Diharapkan	Hasil Pengujian	Status
Backend Go	Backend menerima dan memvalidasi data sensor	Data valid diterima dan data tidak valid ditolak	Berhasil
Database PostgreSQL	Data sensor tersimpan ke database	Data tersimpan pada tabel <i>sensor_readings</i>	Berhasil
Query historis	Data dapat diambil berdasarkan rentang waktu	Data historis dapat ditampilkan melalui API atau query database	Berhasil
Model version	Informasi versi model tersimpan	Data model tersimpan pada tabel <i>model_versions</i>	Berhasil
Prediction data	Hasil estimasi suhu S2 tersimpan	Data estimasi tersimpan pada tabel <i>predictions</i>	Berhasil
Layout sensor	Data layout dan posisi sensor tersimpan	Layout dan marker tersimpan pada <i>layouts</i> dan <i>layout_devices</i>	Berhasil
System log	Error atau kejadian penting tercatat	Log sistem tersimpan pada <i>system_logs</i>	Berhasil

Berdasarkan hasil pengujian, backend dapat menerima data dari gateway dan menyimpannya ke basis data. Data yang tersimpan dapat digunakan untuk monitoring dashboard, pra-pemrosesan ML Worker, evaluasi model, inferensi, serta pelacakan event dan notifikasi. Hal ini menunjukkan bahwa backend dan database telah berfungsi sebagai pusat penyimpanan dan pengelolaan data sistem.

Tempat gambar:

Gambar 4.35 Health Check Backend

Gambar 4.36 Data Sensor pada Tabel *sensor_readings*

Gambar 4.37 Query Data Historis Sensor

4.2.5 Pengujian Dashboard Real-Time

Pengujian dashboard dilakukan untuk memastikan data sensor, hasil estimasi, status sistem, grafik, dan event dapat ditampilkan pada antarmuka pengguna. Pengujian dilakukan dengan membuka dashboard melalui browser, kemudian membandingkan data yang tampil dengan data yang tersedia pada backend atau database.

Dashboard juga diuji untuk memastikan pembaruan data melalui Server-Sent Events berjalan. Dengan SSE, dashboard dapat menerima data terbaru tanpa perlu melakukan refresh halaman secara manual.

Tabel 4.13 Hasil Pengujian Dashboard Real-Time

Komponen yang Diuji	Hasil yang Diharapkan	Hasil Pengujian	Status
Dashboard utama	Menampilkan ringkasan kondisi sistem	Dashboard menampilkan status sensor, gateway, model, dan Telegram	Berhasil
Data sensor	Menampilkan suhu dan kelembaban S1 dan S2	Data S1 dan S2 tampil pada dashboard	Berhasil
Grafik historis	Menampilkan grafik berdasarkan waktu	Grafik suhu dan kelembaban tampil berdasarkan data historis	Berhasil
Estimasi suhu S2	Menampilkan hasil estimasi terbaru	Estimasi suhu S2 tampil pada dashboard	Berhasil
Status sistem	Menampilkan normal, waspada, anomali, atau trouble	Status sistem tampil sesuai hasil klasifikasi	Berhasil

SSE	Menerima pembaruan data tanpa refresh manual	Dashboard menerima pembaruan ketika ada data baru	Berhasil
-----	--	---	----------

Hasil pengujian menunjukkan bahwa dashboard dapat menampilkan data monitoring dan status sistem. Dashboard juga dapat digunakan sebagai media utama untuk memantau kondisi lingkungan server *testbed*, hasil estimasi suhu S2, dan status sistem secara visual.

Tempat gambar:

Gambar 4.38 Dashboard Utama EMS

Gambar 4.39 Grafik Suhu dan Kelembaban

Gambar 4.40 Tampilan Estimasi Suhu S2 dan Status Sistem

4.2.6 Hasil Pengujian Visualisasi Layout Sensor

Pengujian visualisasi layout sensor dilakukan untuk memastikan dashboard dapat menampilkan gambar layout server *testbed* atau ruangan, menampilkan marker sensor pada posisi yang sesuai, serta mengubah tampilan marker berdasarkan status sistem. Fitur ini digunakan untuk membantu pengguna mengetahui posisi sensor yang mengalami kondisi tertentu.

Sensor S1 dan S2 ditampilkan pada layout sesuai posisi pemasangan. Marker sensor dapat menunjukkan status normal, waspada, anomali, atau *trouble* berdasarkan data terbaru yang diterima dari backend.

Tabel 4.14 Hasil Pengujian Visualisasi Layout Sensor

Skenario	Hasil yang Diharapkan	Hasil Pengujian	Status
Menampilkan layout server testbed	Gambar layout tampil pada dashboard	Layout berhasil ditampilkan pada halaman Layout	Berhasil
Menampilkan marker sensor	Marker S1 dan S2 tampil pada posisi yang sesuai	Marker sensor tampil pada posisi yang telah ditentukan	Berhasil
Mengubah status sensor	Marker berubah mengikuti status sistem	Marker menampilkan status normal, waspada, anomali, atau trouble	Berhasil
Menyorot sensor bermasalah	Sensor berstatus tidak normal terlihat lebih jelas	Dashboard menandai sensor yang berstatus waspada, anomali, atau trouble	Berhasil
Menyimpan posisi sensor	Posisi sensor dapat ditampilkan kembali	Posisi marker tersimpan dan dapat dimuat kembali	Berhasil

Berdasarkan hasil pengujian, fitur layout sensor dapat menampilkan posisi sensor S1 dan S2 pada tampilan dashboard. Fitur ini mendukung proses monitoring karena pengguna dapat mengetahui titik sensor yang perlu diperhatikan apabila sistem menampilkan status waspada, anomali, atau *trouble*.

Tempat gambar:

Gambar 4.41 Halaman Layout Sensor

Gambar 4.42 Marker S1 dan S2 pada Layout

Gambar 4.43 Tampilan Marker saat Status Berubah

4.2.7 Hasil Pengujian Model LSTM

Pengujian model LSTM dilakukan untuk mengetahui kemampuan model dalam mengestimasi suhu S2 lima menit ke depan berdasarkan data historis EMS.

Data yang digunakan berasal dari pembacaan sensor yang telah tersimpan pada basis data. Data tersebut kemudian melalui tahap pra-pemrosesan sebelum digunakan untuk training dan evaluasi model.

Tahapan pengujian model meliputi pengambilan data historis, penggabungan data S1 dan S2, *resampling* data menjadi interval satu menit, normalisasi, pembentukan *window* input, training model LSTM, evaluasi model, serta inferensi estimasi suhu S2.

Data input yang digunakan oleh model terdiri dari suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Target estimasi model adalah suhu S2 lima menit ke depan. Pembagian data dilakukan secara kronologis agar urutan waktu tetap terjaga dan tidak terjadi kebocoran data dari masa depan ke data training.

Tabel 4.15 Hasil Pengujian Model LSTM

Komponen yang Diuji	Hasil yang Diharapkan	Hasil Pengujian	Status
Pengambilan data historis	ML Worker dapat mengambil data dari database	Data sensor berhasil diambil dari tabel sensor_readings	Berhasil
Penggabungan data S1 dan S2	Data S1 dan S2 tergabung berdasarkan waktu	Data memiliki fitur suhu S1, kelembaban S1, suhu S2, dan kelembaban S2	Berhasil
Resampling	Data tersusun dalam interval satu menit	Data berhasil di-resampling untuk kebutuhan model	Berhasil
Pembentukan window	Input model terbentuk dari 30 data historis	Window input berhasil dibentuk	Berhasil
Training LSTM	Model dapat dilatih menggunakan data training	Model LSTM berhasil dilatih	Berhasil
Evaluasi model	RMSE, MAE, dan MAPE dapat dihitung	Nilai evaluasi model berhasil dihasilkan	Berhasil
Inferensi	Model menghasilkan	Hasil estimasi suhu S2 berhasil	Berhasil

	estimasi suhu S2 lima menit ke depan	dihasilkan	
--	--------------------------------------	------------	--

Hasil evaluasi model LSTM ditampilkan menggunakan metrik RMSE, MAE, dan MAPE. RMSE digunakan untuk melihat besarnya kesalahan kuadrat rata-rata, MAE digunakan untuk melihat rata-rata kesalahan absolut dalam satuan derajat Celsius, sedangkan MAPE digunakan untuk melihat persentase kesalahan estimasi.

Tabel 4.16 Metrik Evaluasi Model LSTM

Metrik	Nilai	Keterangan
RMSE	[isi nilai RMSE]	Semakin kecil nilai RMSE, semakin kecil kesalahan estimasi model
MAE	[isi nilai MAE]	Menunjukkan rata-rata selisih absolut antara suhu aktual dan estimasi
MAPE	[isi nilai MAPE]%	Menunjukkan persentase kesalahan estimasi terhadap nilai aktual

Berdasarkan hasil pengujian, model LSTM dapat menghasilkan estimasi suhu S2 lima menit ke depan. Nilai RMSE, MAE, dan MAPE digunakan untuk menilai seberapa besar kesalahan estimasi model terhadap data aktual. Hasil ini menjadi dasar untuk membandingkan performa LSTM dengan baseline sederhana pada subbab berikutnya.

Tempat gambar:

Gambar 4.44 Output Training Model LSTM

Gambar 4.45 Hasil Evaluasi Model LSTM

Gambar 4.46 Grafik Aktual dan Estimasi Suhu S2

4.2.8 Pengujian Baseline

Pengujian baseline dilakukan untuk membandingkan performa model LSTM dengan metode sederhana. Baseline yang digunakan adalah *persistence model* dan *moving average*. Pengujian ini dilakukan agar performa LSTM tidak dinilai secara tunggal, tetapi dibandingkan dengan pendekatan dasar yang relevan untuk data *time-series*.

Persistence model menggunakan nilai suhu terakhir sebagai estimasi suhu berikutnya. Sementara itu, *moving average* menggunakan rata-rata beberapa data historis terakhir sebagai estimasi. Kedua baseline tersebut digunakan sebagai pembanding karena sederhana, mudah dijelaskan, dan relevan untuk data suhu yang berubah secara bertahap.

Tabel 4.17 Hasil Pengujian Baseline

Model	RMSE	MAE	MAPE	Keterangan
Persistence Model	[isi RMSE]	[isi MAE]	[isi MAPE]%	Menggunakan nilai suhu terakhir sebagai estimasi
Moving Average	[isi RMSE]	[isi MAE]	[isi MAPE]%	Menggunakan rata-rata beberapa data terakhir
LSTM	[isi RMSE]	[isi MAE]	[isi MAPE]%	Menggunakan pola historis time-series untuk estimasi suhu S2

Berdasarkan hasil pengujian baseline, performa LSTM dianalisis dengan membandingkan nilai error terhadap persistence model dan moving average.

Apabila LSTM memiliki nilai error lebih rendah, maka model LSTM menunjukkan performa estimasi yang lebih baik dibandingkan baseline. Apabila nilai error LSTM belum lebih rendah dari baseline, hasil tersebut tetap dapat dianalisis dengan mempertimbangkan jumlah data, variasi suhu, durasi pengumpulan data, parameter training, dan karakteristik data suhu yang cenderung berubah perlahan.

Dengan adanya baseline, evaluasi model menjadi lebih objektif karena LSTM tidak hanya dinilai dari nilai error tunggal, tetapi dibandingkan dengan metode sederhana yang menjadi pembanding dasar.

Tempat gambar:

Gambar 4.47 Hasil Evaluasi Baseline

Gambar 4.48 Perbandingan Metrik LSTM dan Baseline

4.2.9 Hasil Pengujian Klasifikasi Status Termal

Pengujian klasifikasi status termal dilakukan untuk memastikan sistem dapat menentukan status normal, waspada, dan anomali berdasarkan hasil estimasi suhu S2. Selain itu, sistem juga diuji untuk memastikan status *trouble* dapat muncul ketika terdapat gangguan teknis pada sensor, gateway, data, model, atau komponen pendukung lainnya.

Klasifikasi status termal dilakukan dengan membandingkan hasil estimasi suhu S2 terhadap ambang batas operasional. Status normal diberikan apabila estimasi suhu S2 berada di bawah 30°C. Status waspada diberikan apabila

estimasi suhu S2 berada pada rentang 30°C sampai 32°C. Status anomali diberikan apabila estimasi suhu S2 berada di atas 32°C.

Tabel 4.18 Hasil Pengujian Klasifikasi Status Termal

Skenario	Kondisi Uji	Status yang Diharapkan	Hasil Pengujian	Status
Kondisi normal	Estimasi suhu S2 < 30°C	Normal	Sistem menampilkan status normal	Berhasil
Kondisi waspada	Estimasi suhu S2 30°C–32°C	Waspada	Sistem menampilkan status waspada	Berhasil
Kondisi anomali	Estimasi suhu S2 > 32°C	Anomali	Sistem menampilkan status anomali	Berhasil
Gangguan sensor	Sensor tidak terbaca atau data tidak valid	Trouble	Sistem menampilkan status trouble	Berhasil
Gangguan gateway	Gateway tidak mengirim data dalam batas waktu tertentu	Trouble	Sistem menampilkan status trouble	Berhasil
Gangguan model	Model belum siap atau hasil estimasi tidak tersedia	Trouble	Sistem menampilkan status trouble	Berhasil

Hasil pengujian menunjukkan bahwa sistem dapat membentuk status berdasarkan hasil estimasi suhu S2 dan kondisi teknis sistem. Status normal, waspada, dan anomali digunakan untuk menunjukkan kondisi termal, sedangkan status *trouble* digunakan untuk menunjukkan gangguan teknis. Dengan pemisahan

ini, sistem dapat membedakan antara kenaikan suhu dan masalah operasional komponen.

Tempat gambar:

Gambar 4.49 Tampilan Status Normal

Gambar 4.50 Tampilan Status Waspada

Gambar 4.51 Tampilan Status Anomali

Gambar 4.52 Tampilan Status Trouble

4.2.10 Pengujian Telegram Alert

Pengujian Telegram alert dilakukan untuk memastikan sistem dapat mengirimkan notifikasi ketika status berada pada kondisi waspada, anomali, atau *trouble*. Pengujian dilakukan dengan memicu kondisi tertentu, kemudian memeriksa apakah pesan peringatan berhasil diterima pada Telegram.

Notifikasi Telegram berisi informasi status, sensor acuan, estimasi suhu S2, horizon estimasi, waktu kejadian, dan keterangan kondisi. Selain itu, sistem juga menerapkan mekanisme *cooldown* agar notifikasi yang sama tidak dikirimkan berulang dalam waktu yang terlalu dekat.

Tabel 4.19 Hasil Pengujian Telegram Alert

Skenario	Kondisi	Hasil yang Diharapkan	Hasil Pengujian	Status
----------	---------	-----------------------	-----------------	--------

Status normal	Estimasi suhu S2 < 30°C	Tidak ada alert Telegram	Telegram tidak mengirim pesan	Berhasil
Status waspada	Estimasi suhu S2 30°C–32°C	Telegram mengirim pesan waspada	Pesan waspada diterima	Berhasil
Status anomali	Estimasi suhu S2 > 32°C	Telegram mengirim pesan anomali	Pesan anomali diterima	Berhasil
Status trouble	Sensor, gateway, data, atau model bermasalah	Telegram mengirim pesan trouble	Pesan trouble diterima	Berhasil
Cooldown alert	Status sama muncul berulang	Sistem tidak mengirim spam notifikasi	Notifikasi berulang dibatasi oleh cooldown	Berhasil
Notification log	Setiap proses notifikasi tercatat	Riwayat tersimpan pada notification_logs	Data notifikasi tercatat	Berhasil

Contoh pesan Telegram yang diterima adalah sebagai berikut:

Peringatan EMS Server Testbed

Status: WASPADA

Sensor Acuan: S2 - Hotspot/Exhaust

Estimasi Suhu S2: 31.4°C

Horizon Estimasi: 5 menit ke depan

Waktu: [isi waktu pengujian]

Keterangan: Suhu estimasi berada pada rentang waspada.

Berdasarkan hasil pengujian, sistem dapat mengirimkan notifikasi Telegram ketika status sistem membutuhkan perhatian. Pencatatan pada

notification logs juga menunjukkan bahwa setiap proses pengiriman notifikasi dapat ditelusuri kembali.

Tempat gambar:

Gambar 4.53 Pesan Telegram Status Waspada

Gambar 4.54 Pesan Telegram Status Anomali atau Trouble

Gambar 4.55 Riwayat Notification Logs

4.2.11 Pengujian Demo Sistem

Pengujian demo sistem dilakukan untuk memastikan seluruh komponen dapat berjalan sebagai satu kesatuan sistem *early warning*. Demo mencakup pembacaan sensor, pengiriman data oleh gateway, penyimpanan data pada database, tampilan dashboard, visualisasi layout sensor, estimasi suhu S2 menggunakan LSTM, klasifikasi status, Telegram alert, dan perubahan status kembali normal.

Pengujian demo sistem penting karena sistem yang dikembangkan tidak hanya terdiri dari model LSTM, tetapi juga integrasi perangkat keras dan perangkat lunak. Oleh karena itu, demo digunakan untuk menunjukkan bahwa sistem dapat berjalan dari awal sampai akhir sesuai alur yang dirancang.

Tabel 4.20 Hasil Pengujian Demo Sistem

Mode Demo	Yang Ditampilkan	Hasil yang Diharapkan	Hasil Pengujian	Status
Live monitoring	Dashboard suhu dan kelembaban S1 dan S2	EMS menampilkan data aktual sensor	Data sensor tampil pada dashboard	Berhasil

Visualisasi layout	Layout testbed, marker sensor, dan status	Posisi sensor dapat dimonitor secara visual	Marker S1 dan S2 tampil pada layout	Berhasil
Estimasi LSTM	Grafik aktual dan estimasi suhu S2	Modul LSTM menghasilkan estimasi suhu S2	Estimasi suhu S2 tampil pada dashboard	Berhasil
Evaluasi model	RMSE, MAE, MAPE, dan baseline	Evaluasi model dapat ditampilkan	Metrik evaluasi tampil pada halaman Prediction & LSTM	Berhasil
Simulasi waspada/anomali	Peningkatan suhu atau replay data	Status waspada/anomali muncul	Sistem menampilkan status sesuai kondisi uji	Berhasil
Telegram alert	Pesan peringatan Telegram	Telegram menerima pesan peringatan	Pesan Telegram berhasil diterima	Berhasil
Recovery	Status kembali normal	Sistem mencatat perubahan status	Status kembali normal dan event tercatat	Berhasil

Berdasarkan hasil pengujian demo sistem, seluruh komponen utama dapat berjalan secara terintegrasi. Sensor membaca data suhu dan kelembaban, gateway mengirimkan data ke backend, backend menyimpan data ke database, dashboard menampilkan data dan status, ML Worker menghasilkan estimasi suhu S2, sistem mengklasifikasikan status, dan Telegram mengirimkan notifikasi ketika status memenuhi kondisi peringatan.

Hasil ini menunjukkan bahwa sistem yang dikembangkan mampu menjalankan fungsi utama sebagai *Early Warning System* pada EMS server menggunakan algoritma Long Short-Term Memory.

Tempat gambar:

Gambar 4.56 Demo Live Monitoring

Gambar 4.57 Demo Estimasi Suhu S2

Gambar 4.58 Demo Status Waspada atau Anomali

Gambar 4.59 Demo Telegram Alert

Gambar 4.60 Demo Recovery Status

BAB 5

KESIMPULAN DAN SARAN

Penutup

Bab ini berisi simpulan dan saran berdasarkan hasil implementasi dan pengujian sistem yang telah dilakukan. Simpulan disusun untuk menjawab rumusan permasalahan penelitian, sedangkan saran diberikan sebagai masukan untuk pengembangan sistem pada penelitian berikutnya.

5.1 Simpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan pada sistem *Early Warning System* EMS Server Menggunakan Algoritma Long Short-Term Memory, maka diperoleh beberapa simpulan sebagai berikut.

1. Sistem Environment Monitoring System pada lingkungan server *testbed* berhasil dikembangkan untuk melakukan pemantauan suhu dan kelembaban secara periodik. Sistem menggunakan dua sensor XY-MD02, yaitu S1 sebagai sensor *ambient* atau referensi lingkungan dan S2 sebagai sensor *hotspot* atau *exhaust*. Data dari kedua sensor berhasil dibaca oleh

Raspberry Pi gateway melalui komunikasi Modbus RTU RS485 dan dikirimkan ke backend.

2. Data sensor EMS berhasil dikelola sebagai data *time-series* dengan menyimpan setiap pembacaan berdasarkan waktu pencatatan. Data yang disimpan mencakup suhu, kelembaban, identitas sensor, identitas gateway, sumber data, dan status kualitas data. Penyimpanan data tersebut memungkinkan sistem untuk menampilkan data historis, membuat grafik monitoring, serta menyediakan data untuk proses pra-pemrosesan dan model LSTM.
3. Algoritma Long Short-Term Memory berhasil diterapkan untuk melakukan estimasi suhu S2 lima menit ke depan berdasarkan data historis EMS. Model menggunakan fitur suhu S1, kelembaban S1, suhu S2, dan kelembaban S2. Proses pra-pemrosesan dilakukan melalui penggabungan data sensor, *resampling*, normalisasi, pembentukan *window*, serta pembagian data secara kronologis agar sesuai dengan karakteristik data *time-series*.
4. Sistem *early warning* berhasil dibangun dengan mengklasifikasikan hasil estimasi suhu S2 ke dalam status normal, waspada, dan anomali berdasarkan ambang batas operasional penelitian. Selain itu, sistem juga memiliki status *trouble* untuk menandai gangguan teknis seperti sensor tidak terbaca, gateway tidak aktif, data tidak valid, model belum siap, atau hasil estimasi tidak tersedia. Pemisahan status termal dan status teknis membuat sistem lebih jelas dalam membedakan kondisi suhu dan gangguan operasional.

5. Notifikasi Telegram berhasil diimplementasikan sebagai media peringatan dini ketika sistem berada pada status waspada, anomali, atau *trouble*.
Pesan notifikasi memuat informasi status, sensor acuan, estimasi suhu S2, horizon estimasi, waktu kejadian, dan keterangan kondisi. Mekanisme *cooldown* juga digunakan agar sistem tidak mengirimkan notifikasi berulang secara berlebihan.
6. Evaluasi sistem dilakukan melalui pengujian pembacaan sensor, gateway, backend, database, dashboard, visualisasi layout sensor, model LSTM, baseline, klasifikasi status, Telegram alert, dan demo sistem. Evaluasi model dilakukan menggunakan RMSE, MAE, dan MAPE, serta dibandingkan dengan baseline berupa *persistence model* dan *moving average*. Dengan pengujian tersebut, sistem tidak hanya dinilai dari sisi model, tetapi juga dari keberhasilan integrasi perangkat keras, perangkat lunak, database, dashboard, dan notifikasi.

Berdasarkan simpulan tersebut, sistem yang dikembangkan telah mampu menjalankan fungsi utama sebagai EMS berbasis *early warning*, yaitu membaca data suhu dan kelembaban, menyimpan data sebagai *time-series*, mengestimasi suhu S2 menggunakan LSTM, menentukan status sistem, menampilkan hasil monitoring melalui dashboard, dan mengirimkan notifikasi peringatan dini melalui Telegram.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan sistem pada penelitian berikutnya.

1. Pengumpulan data sensor sebaiknya dilakukan dalam durasi yang lebih panjang agar model LSTM memiliki data historis yang lebih bervariasi. Data yang lebih banyak dan mencakup berbagai kondisi termal dapat membantu model mempelajari pola perubahan suhu secara lebih baik.
2. Pengujian sistem dapat dikembangkan pada lingkungan server yang lebih representatif, misalnya ruang server kecil atau rak server dengan variasi beban kerja yang lebih beragam. Dengan demikian, hasil pengujian dapat menggambarkan kondisi operasional yang lebih mendekati lingkungan nyata.
3. Jumlah sensor dapat ditambah agar sistem mampu memantau lebih banyak titik termal. Penambahan sensor dapat membantu pengguna mengetahui distribusi suhu pada beberapa area, bukan hanya pada titik ambient dan hotspot.
4. Model prediksi dapat dikembangkan dengan membandingkan LSTM dengan metode lain, seperti GRU, CNN-LSTM, atau pendekatan machine learning lain yang sesuai untuk data *time-series*. Perbandingan tersebut dapat digunakan untuk mengetahui metode yang paling sesuai terhadap karakteristik data EMS.
5. Mekanisme ambang batas status dapat dikembangkan menjadi lebih adaptif. Pada penelitian ini, threshold digunakan sebagai ambang operasional testbed. Pada pengembangan berikutnya, threshold dapat

disesuaikan secara dinamis berdasarkan pola historis, karakteristik perangkat, atau kondisi lingkungan.

6. Sistem notifikasi dapat dikembangkan dengan menambahkan kanal peringatan lain selain Telegram, seperti email, WhatsApp gateway, atau integrasi dengan sistem monitoring yang sudah ada. Namun, penambahan kanal tersebut perlu tetap mempertimbangkan kesederhanaan dan kebutuhan operasional sistem.
7. Dashboard dapat dikembangkan dengan fitur analisis yang lebih lengkap, seperti ringkasan tren suhu harian, laporan otomatis, ekspor data pengujian, dan visualisasi perbandingan model. Fitur tersebut dapat membantu pengguna melakukan evaluasi kondisi lingkungan server secara lebih terstruktur.
8. Pada pengembangan lanjutan, sistem dapat diuji dengan mekanisme deployment yang lebih stabil, misalnya menggunakan service manager pada Raspberry Pi dan konfigurasi server yang lebih permanen. Hal ini dapat meningkatkan keandalan sistem ketika dijalankan dalam durasi panjang.

Daftar Pustaka Sementara

- ASHRAE Technical Committee 9.9. (2021). *Thermal guidelines for data processing environments* (5th ed.). ASHRAE.

- Blázquez-García, A., Conde, A., Mori, U., & Lozano, J. A. (2021). A review on outlier/anomaly detection in time series data. *ACM Computing Surveys*, 54(3), Article 56. <https://doi.org/10.1145/3444690>
- Bontempi, G., Ben Taieb, S., & Le Borgne, Y.-A. (2013). Machine learning strategies for time series forecasting. In M.-A. Aufaure & E. Zimanyi (Eds.), *Business Intelligence* (pp. 62–77). Springer. https://doi.org/10.1007/978-3-642-36318-4_3
- Cassano, F., Crespino, A. M., Lazoi, M., Specchia, G., & Spennato, A. (2025). An EWS-LSTM-based deep learning early warning system for industrial machine fault prediction. *Applied Sciences*, 15(7), 4013. <https://doi.org/10.3390/app15074013>
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), Article 15. <https://doi.org/10.1145/1541880.1541882>
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645–1660. <https://doi.org/10.1016/j.future.2013.01.010>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>

- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688. <https://doi.org/10.1016/j.ijforecast.2006.03.001>
- Joshua, L., & Mulyana, T. M. S. (2024). Engine health monitoring pada sepeda motor berbasis Arduino menggunakan algoritma Fuzzy Inference System Tsukamoto. *Jurnal Algoritma, Logika dan Komputasi*, 7(2), 697–707. <https://doi.org/10.30813/j-alu.v7i2.7765>
- Kang, Y., Ma, C., Wang, S., Wu, W., & Zhao, K. (2022). A two-segment LSTM based data center temperature prediction model. *IEICE Electronics Express*, 19(21), Article 20220291. <https://doi.org/10.1587/elex.19.20220291>
- Khumaidi, A., Raafi'udin, R., & Solihin, I. P. (2020). Pengujian algoritma Long Short Term Memory untuk prediksi kualitas udara dan suhu Kota Bandung. *Jurnal Telematika*, 15(1), 13–18.
- Rajkumar, P. (2025). Humidity and temperature monitoring using Raspberry Pi via RS232 networking. *Array*, 27, Article 100464. <https://doi.org/10.1016/j.array.2025.100464>
- Wang, S., Kang, Y., Xu, Y., Ma, C., Wang, H., & Wu, W. (2024). Data center temperature prediction and management based on a two-stage self-healing model. *Simulation Modelling Practice and Theory*, 132, Article 102883. <https://doi.org/10.1016/j.simpat.2023.102883>
- Wicaksono, M. F., Rahmatya, M. D., & Ilham. (2022). IoT implementation for server room security monitoring using Telegram API.

*International Journal on Advanced Science, Engineering and Information
Technology*, 12(5), 1931–1937. <https://doi.org/10.18517/ijaseit.12.5.13922>